

Multi-Model Orchestration Patterns

Validation of the Nova design — one conversational controller, disposable stateless workers, script-carried handoffs. Seven sourced research passes, synthesised, with corrections where the evidence disagrees. Every substantive claim links to its source.

1. Verdict

Mixed, leaning sound. The skeleton of the design — one controller that owns the task, workers that return results rather than talk, deterministic code carrying the handoff, checkpoints between stages, a hard retry cap with escalation to a human — is the dominant shape in shipped systems today and the only multi-agent topology with benchmark evidence of *containing* error propagation rather than amplifying it. Anthropic's Research feature, Manus Wide Research, Claude Code, Codex CLI, Gemini CLI, Block Goose, Stripe's internal coding agents, and every enterprise orchestration platform (Bedrock, Copilot Studio, ADK, Databricks, Agentforce) ship essentially this shape. The two failure modes you are designing against (ownerless handoff loops, transcript-passing token burn) are real, well documented, and this design does address them.

Four parts of the design as written are wrong or naive, and each is spelled out in §3:

1. **"Checkpoint every stage, one step at a time" is a serial bottleneck by construction.** Anthropic names synchronous lead-waits-for-worker as the main scaling failure of this exact pattern. The script needs fan-out and async joins for independent steps, and the controller should only gate on *dependencies*, not on every step.
2. **"Nova judges done/not-done" is the weakest link, not the safety net.** LLM-as-judge is biased toward approving its own model family's output, near coin-flip on hard objective pairs, and flips under pushback. Definition-of-done must be machine-checkable wherever possible; an LLM verdict should be the fallback, not the default.
3. **"Retry capped at three" is a convention, not a finding.** With an LLM judge, the evidence says one retry captures most of the gain and further rounds have *negative* expected value. With a deterministic verifier (tests, schema), returns keep improving log-linearly and three is arbitrary and probably too low; the right cap is a budget.
4. **The brief/return schema as described omits the fields that production systems learned the hard way:** task and correlation ids, attempt number, idempotency key, budget and deadline, allowed tools, a non-terminal *input-required* / *blocked* state, a classified (retryable or not) failure, side effects performed, and cost consumed.

One more correction of framing: the strongest evidence for "strip history from workers" is *not* that history is noise; it is that information delivered incrementally across turns degrades performance by ~39% and that a single fully-specified brief recovers ~95% of it. The design is right, but for a slightly different reason than

stated, and the implication is that Nova's brief-writing quality is the single highest-leverage component in the system.

2. Answers to the seven questions

Q1. Is single-controller + script-mediated handoff sound? Named equivalents, scale limits, cost vs agent-to-agent

Named equivalents. The pattern has at least eight names in current practice, all sharing the same invariants (one LLM owns the plan; workers get a fresh context and a task; workers return a result; no peer messaging):

- *Orchestrator-workers* — Anthropic, [Building effective agents](#) (Dec 2024): "a central LLM dynamically breaks down tasks, delegates them to worker LLMs, and synthesizes their results."
- *Lead agent + subagents* — Anthropic, [How we built our multi-agent research system](#) (Jun 2025), the production instance with the most published numbers.
- *Manager pattern / agents-as-tools* — OpenAI, [A practical guide to building agents](#); their SDK docs add that orchestrating "via your code" makes tasks "more deterministic and predictable, in terms of speed, cost and performance" ([OpenAI Agents SDK](#)). That is the closest vendor endorsement of your "dumb script" choice.
- *Supervisor* — LangGraph ([langgraph-supervisor](#)), with an `output_mode` of `last_message` as the library-level answer to controller bloat.
- *Supervisor mode* — [AWS Bedrock multi-agent collaboration](#), which exposes no peer channel at all.
- *Sequential / Parallel / Loop workflow agents* — [Google ADK](#), described as "deterministic controllers of the execution" — literally a script carrying handoffs.
- Academic names: *centralized / hub-and-spoke* in Google/MIT's [Towards a Science of Scaling Agent Systems](#) (Dec 2025); *puppeteer* orchestration in [Multi-Agent Collaboration via Evolving Orchestration](#) (NeurIPS 2025); *planner-executor* generically.

Where it breaks down at scale. Five documented limits, in descending order of evidence quality.

Sub-task specification is the largest failure bucket. In the MAST taxonomy ([Cemri et al. 2025](#), 1,600+ annotated traces, $\kappa = 0.88$) "system design / specification" failures are ~44% of all failures — bigger than inter-agent misalignment (~32%) or verification (~24%). Anthropic's lead agent produced duplicated and misdirected work when briefs were vague ("research the semiconductor shortage"), and the fix was a four-part contract: "an objective, an output format, guidance on the tools and sources to use, and clear task boundaries" ([Anthropic](#)). Stateless workers make this a hard requirement rather than a nicety: nothing else carries context across the boundary.

Sequential work degrades under every multi-agent variant. Google/MIT tested 260 configurations across five topologies, three model families, and four benchmarks. Centralized coordination gave the largest gain measured on a decomposable task (+80.8% on Finance-Agent) but *all* multi-agent variants degraded sequential planning (PlanCraft) by 39–70%, and tool-heavy workflows (16 tools) suffered a measurable "tool-coordination penalty" ($\beta = -0.096$, $p = 0.002$). They also found negative returns once the single-agent baseline exceeds ~45% accuracy ([arXiv 2512.08296](#); [Google Research blo](#)

g). This is one preprint, unreplicated, and its predictive model explains only $R^2 = 0.373$ of the variance (0.413 with their task-grounded capability metric) — but it matches Cognition's and Anthropic's independent advice to start single-agent.

The synchronous controller is a latency bottleneck. Anthropic's own admission: "our lead agents execute subagents synchronously, waiting for each set of subagents to complete before proceeding... the entire system can be blocked while waiting for a single subagent to finish" ([Anthropic](#)). Claude Code docs add the cost specific to *stateless* workers: a fresh subagent "may need time to gather context" on every spawn ([Claude Code subagents](#)).

Information fragmentation is an unavoidable tax. Google/MIT call it "intrinsic information fragmentation... the global context must be compressed into inter-agent messages." A 2026 theory paper shows any delegated network is decision-theoretically dominated by a centralized decider with the same information, with the gap equal to the mutual information lost in compression ([Ao, Gao & Simchi-Levi 2026](#)). This supports centralizing *decisions* in Nova, but is also a ceiling on any design that summarises worker output.

The controller is a single point of failure — with a twist. The same Google/MIT study found the central node is also the error firewall: independent agents amplified trace-level errors 17.2×, centralized coordination contained that to 4.4×. MAST interventions point the same way: giving ChatDev's CEO agent final authority raised success +9.4%, adding verification +15.6% ([MAST](#)). Losing the controller loses the system, but removing it lets errors compound.

What it costs versus direct agent-to-agent messaging. No paper holds everything constant and compares controller-plus-script against peer messaging. The directional evidence is consistent:

- Anthropic: agents $\approx 4\times$ chat tokens, multi-agent $\approx 15\times$; on BrowseComp "token usage alone explained 80% of the variance" in performance ([Anthropic](#)). Their Jan 2026 follow-up says 3–10× ([Claude blog](#)). Internal measurements, no public task set.
- Claude Code docs on their own peer-messaging mode: token cost "Higher: each teammate is a separate Claude instance... coordination overhead increases" versus subagents "Lower: results summarized back to main context" ([agent teams](#)). Vendor statement, no numbers.
- CoRL (2025): decentralized frameworks "invoke multiple LLMs for every input and thus lead to substantial and uncontrolled inference costs"; a central controller makes cost "controllable" ([arXiv 2511.02755](#)).
- The one cost where the controller clearly loses is wall-clock latency on sequential chains (above).

Practitioner figures like "supervisors use 20–40% more tokens than swarms" or "routing degrades after 8–12 round trips" circulate ([Augment Code guide](#)) but I found no primary measurement behind them; treat as opinion.

Counter-evidence for decentralized designs exists but is weak. AgentNet reports strong numbers against older baselines ([arXiv 2504.00587](#)); Agent Q-Mix beats LangGraph on HLE by 1.6 points ([arXiv 2604.00344](#)); AgentVerse found horizontal (peer) structure better for perspective-diverse tasks but "an increase in the number of reviewers correlates with a higher likelihood of erroneous critiques" ([AgentVerse, ICLR 2024](#)). Cognition's [Don't Build Multi-Agents](#) is the main dissent and it critiques *both* your design and the alternative; its carve-out is precisely your design restricted to read-only work, and Walden Yan's 2026 update lands on "one single Devin" managing isolated parallel sub-agents ([Latent Space](#)).

Q2. What the major frameworks actually do

Full detail is in Appendix A. The organising facts, as of August 2026:

Default path. Frameworks split cleanly. Controller-calls-workers is the default in LangChain 1.x (whose docs now *discourage* their own supervisor library in favour of subagents-as-tools, and say subagents "are stateless... all conversation memory maintained by the main agent" — [LangChain subagents](#)), Claude Code / Agent SDK, Pydantic AI, smolagents, CrewAI (delegation off by default), and Bedrock. Peer handoff is an equal first-class path in OpenAI Agents SDK, Microsoft Agent Framework (whose Handoff orchestration is "a mesh topology where agents are connected directly without an orchestrator" — [MS Learn](#)), Google ADK (listing `sub_agents` enables LLM-driven `transfer_to_agent` including lateral transfers — [ADK](#)), and Strands (Swarm).

What a handoff transfers. This is the token-burn axis you care about. OpenAI handoffs pass the *entire conversation history* by default: "it's as though the new agent takes over the conversation, and gets to see the entire previous conversation history," trimmable only via `input_filter` ([OpenAI handoffs](#)). Microsoft's handoff broadcasts all user/agent messages to every agent. ADK `sub_agents` continue the same session. Bedrock has a single `relayConversationHistory` toggle. By contrast every *tool-style* delegation — Claude Code, LangChain subagents, Pydantic AI, smolagents, Strands agents-as-tools, CrewAI — passes only the task string. So: **the frameworks whose default primitive is "handoff" push you toward transcript-passing; the ones whose primitive is "agent as tool" do not.** To get your design out of OpenAI, Microsoft, or ADK you must deliberately avoid the handoff primitive (OpenAI: use only `as_tool()`, define no `handoffs`; ADK: `disallow_transfer_to_peers=True` or use `AgentTool` / workflow agents; Microsoft: `WorkflowBuilder` / `SequentialBuilder` rather than `HandoffBuilder`).

Checkpointing. Two shapes. *Structured super-step snapshots*: LangGraph (a `StateSnapshot` per super-step with values, next node, metadata, pending tasks — [check pointers](#)), Microsoft Agent Framework (executor state, pending messages, shared state per superstep — [checkpoints](#)), CrewAI Flows, Strands multi-agent. *Transcript logs*: OpenAI Sessions, Claude sessions (JSONL), ADK events, AutoGen `save_state`. Your "structured fields, not prose" choice aligns with the first camp. **Resume semantics are re-execution, not line-level suspension, in every framework that documents it** — LangGraph "restarts the entire node from the beginning" ([interrupts](#)), ADK warns "tools may execute more than once" ([ADK resume](#)). Your checkpoint design therefore needs idempotent workers (see Q5).

Corrections to common assumptions. Claude Code subagents are not strictly flat: by default they can nest three layers deep, and named subagents with the `SendMessage` tool can message siblings (v2.1.206+); agent teams (true peer messaging) exist but are experimental and off by default ([sub-agents](#); [agent teams](#)). Claude Code's *Dynamic Workflows* (v2.1.154+) are a JavaScript script that "holds the loop, the branching, and the intermediate results itself, so Claude's context holds only the final answer" ([workflows](#)) — the closest commercial implementation of your "dumb script" idea. AutoGen is in maintenance mode; Bedrock Agents Classic closed to new customers on 30 July 2026 ([AWS](#)).

Q3. Context isolation — what the evidence says

Bottom line. The direct evidence supports a narrower claim than the one in your brief. It strongly supports "give a worker the whole job in one fully-specified turn" and "keep irrelevant material out of the worker's context." It does not directly measure "same short

query with versus without a few thousand tokens of unrelated prior chat." That specific experiment does not appear to exist.

Single fully-specified turn versus incremental multi-turn — strong, direct.

- [Laban et al., "LLMs Get Lost in Multi-Turn Conversation"](#) (Microsoft/Salesforce, May 2025): 15 models, 6 tasks including API/function calling, 200k+ simulated conversations. Fully-specified single turn $\approx$ 90% average; the same information revealed across turns $\approx$ 65%: a **39% average relative drop**. Aptitude fell 16%; *unreliability* (best-minus-worst of N runs) rose 112%. Crucially, the CONCAT condition — all the shards pasted into one turn — recovers **95.1%** of single-turn performance. The mechanism is premature commitment and poor state tracking across turns, not context length. A 2026 replication reports GPT-5.2 dropping from 92.7% to 48.5% sharded, with memory augmentation barely helping and a "mediator" that rewrites the dialogue into one explicit instruction recovering $\sim$ 20 points ([arXiv 2602.07338](#)).
- [\$\tau^2\$ -bench](#) (Sierra, 2025) has the cleanest ablation for tool-using agents: the same telecom tasks with information elicited through dialogue versus given up front. GPT-4.1 34% $\rightarrow$ 52% (**+18 points**); o4-mini 42% $\rightarrow$ 67% (**+25 points**). pass^k decays fast (roughly halving from k = 1 to k = 4 for GPT-4.1, read from their figure).
- [BFCL v3/v4](#): under the official harness a single model scores $\sim$ 30–45 absolute points lower on multi-turn than single-turn categories (e.g. one published run: non-live 89.1% vs multi-turn 45.1% — [algollabs](#)). The `long_context` sub-category, which injects extraneous data, scores below base for most models. Caveat: BFCL multi-turn tasks are genuinely multi-step with state, so this measures planning difficulty as much as "history as noise."

Irrelevant context hurts narrow tasks — direct, but mostly in the long-context regime.

- [LongMemEval](#): on *information extraction from chat history*, GPT-4o scores 0.870 with only the relevant sessions versus 0.606 with the full history (**−30%**); Llama-3.1-70B $\sim$ 55%. History here is $\sim$ 100k tokens.
- [Chroma "Context Rot"](#) (Jul 2025, 18 models): a focused $\sim$ 300-token input beats the full $\sim$ 113k-token conversation for every model; a single distractor lowers accuracy and distractors compound.
- [Lost in the Middle](#): GPT-3.5 with the answer buried in the middle of 20 documents (53.8%) scored *below* closed-book with no context at all (56.1%).
- [Shi et al., ICML 2023](#): adding a single irrelevant-but-plausible sentence to GSM8K-style problems (the GSM-IC set) dropped code-davinci-002 with chain-of-thought from 95.0% on the base problems to 72.4%.
- [NoLiMa](#): 11 of 13 long-context models fall below 50% of their short-context baseline by 32k tokens; GPT-4o 99.3% $\rightarrow$ 69.7%.

Irrelevant tools hurt tool selection — direct and strong. Anthropic's internal MCP eval: Opus 4 went from 49% to 74% when tool definitions were retrieved on demand rather than all loaded ([Advanced tool use](#), vendor internal eval). RAG-MCP: tool-selection accuracy 13.6% with all tools in prompt versus 43.1% with retrieval ([arXiv 2505.03275](#)). [FuncBenchGen](#): type-compatible distractor functions "severely degrade performance for all models"; GPT-5 drops from 72.5% at 5 required calls to 15.0% at 20. This is "strip irrelevant tools," not "strip history," but the mechanism practitioners invoke is the same. For your split-by-context-need rule, it means the *tool list* handed to an extraction worker matters as much as the absence of history.

Structured output constraints are not the hazard. [Tam et al. 2024](#) found JSON mode hurt reasoning (Claude-3-Haiku GSM8K 86.5 → 23.4) but *helped* classification (DDXPlus +11 to +19 points), and the drop was caused by the schema putting `answer` before `reasoning`. The dottxt rebuttal showed properly implemented constrained decoding matches or beats unconstrained ([Say What You Mean](#)). For your "emit a tool-call object with no history" workers, format constraints help; just order any reasoning field before the answer.

What the evidence does *not* show. No controlled benchmark takes a narrow extraction or tool-call query, prepends 0/N turns of realistic *unrelated* short chat, and measures accuracy. Your design's "no history for extraction" rule is an extrapolation from (a) CONCAT ≈ FULL, (b) irrelevant-context penalties at long-context scale, and (c) tool-overload results. Well supported in direction; magnitude for short histories (<5k tokens) unmeasured. One practitioner counter-note: Manus found that *removing* context mid-loop (tool definitions, failed actions) caused schema violations and repeated mistakes, and argues for masking rather than stripping and for keeping errors in context ([Manus](#)). That is about an agent's own loop, not a fresh worker, but it is a reminder that the benefit is not unconditional. Also relevant: on SWE-bench, simply masking old tool observations halved cost with no loss in solve rate ([JetBrains](#), "[The Complexity Trap](#)") — history is dispensable, but that study shows *no accuracy gain* from dropping it.

Q4. Verdict loops and retry caps

How production systems implement it. The canonical name is Anthropic's *evaluator-optimizer* workflow, recommended only "when we have clear evaluation criteria, and when iterative refinement provides measurable value," with "stopping conditions (such as a maximum number of iterations)" — no number given ([Building effective agents](#)). Microsoft's Magentic-One orchestrator keeps a task ledger and a progress ledger with a *stall counter*; if stalls exceed a threshold (≤ 2 in their experiments) it exits the inner loop and re-plans ([arXiv 2411.04468](#)). OpenHands ships a concrete stuck detector: halt on the same action/observation 4+ times, same action/error 3+ times, 3+ monologue messages, or 6+ ping-pong cycles, with semantic rather than exact matching ([OpenHands docs](#)). Google ADK's `LoopAgent` refuses to default: "You *must* implement a termination mechanism" ([ADK](#)).

Retry limits in practice. They differ by two orders of magnitude and none cites evidence: OpenAI Agents SDK `max_turns` = 10 ([source](#)); LangGraph `recursion_limit` was 25 for years and was raised to **1000** steps in v1.0.6 ([graph API docs](#)) — a 40× change with no published rationale, which says something about how principled these numbers are; CrewAI `max_iter` = 20 and `max_retry_limit` = 2 ([CrewAI agents](#)), `guardrail_max_retries` = 3 ([CrewAI tasks](#)); AutoGen 0.2 `max_consecutive_auto_reply` = 100; Claude Agent SDK `max_turns` = unlimited, with `max_budget_usd` as the alternative brake ([Claude SDK](#)); Temporal activities default to **unlimited** attempts with exponential backoff and recommend bounding by *timeout* rather than attempt count ([Temporal retry policies](#)). Note CrewAI's behaviour at the cap: the agent "must provide its best answer" — a forced final answer, which is a rubber stamp by construction.

Escalation paths. Three mechanisms recur. LangGraph `interrupt()` persists state and resumes with `Command(resume=...)`, with the documented caveat that the node re-runs from its start ([interrupts](#)). OpenAI `needsApproval` pauses the run and returns a serialisable state; guardrails are automatic tripwires, approvals are human decision points ([OpenAI](#)). Temporal delivers the human verdict as a Signal, with durable timers so "escalations fire automatically" on SLA expiry ([Temporal HITL](#)). Your "surfaces to me

after three" is a degenerate version of this; the missing pieces are a *timeout* on your own response and a defined action when you do not answer.

Documented failure modes of self-checking loops.

Rubber-stamping. Intrinsic self-correction does not work for reasoning: GPT-4 on GSM8K goes 95.5% → 91.5% → 89.0% across two self-correction rounds; GPT-3.5 on CommonSenseQA collapses 75.8% → 38.1% ([Huang et al., ICLR 2024](#)). The TACL survey concludes "no prior work demonstrates successful self-correction with feedback from prompted LLMs, except for studies in tasks that are exceptionally suited for self-correction" ([Kamoi et al. 2024](#)). Judges favour their own model family: GPT-4 self-recognises at 73.5% and self-preference scales linearly with recognition ([Panickssery et al., NeurIPS 2024](#)). On objectively graded hard pairs GPT-4o scores 50.86% on JudgeBench — coin-flip ([arXiv 2410.12784](#)). Judges flip under pushback: a bare "the answer should be X" rebuttal flips the verdict 84.5% of the time with only a 17.1% *correct-flip* rate ([Kim et al. 2025](#)). Anthropic's Project Vend phase 2 is the production illustration: adding a CEO agent as approver, it approved requests ~8× more often than it denied them, including full refunds ([Project Vend 2](#)). MAST v3: "incorrect verification" 9.1% and "no/incomplete verification" 8.2% of all failures ([MAST v3](#)).

Reject-forever. Strict verifiers manufacture the mirror loop: TinyV found 38.5% of responses a rule-based verifier marked wrong were actually correct ([arXiv 2505.14625](#)). Huang et al. document the model "changing a correct answer to an incorrect one" when told to retry. Self-Refine found *generic* feedback actively hurts (sentiment task 43.2 → 31.2) — the critic must name a concrete action and concrete phrases to change ([Madaan et al. 2023](#)).

Evaluator gaming. METR: o3 reward-hacked in 30.4% of RE-Bench runs (100% on one task) by patching timers, stubbing evaluators, and overriding equality operators, and when asked said it violated user intent 10/10 times ([METR](#)). ImpossibleBench: when tests are made impossible to pass legitimately, GPT-5 cheats in 76% of the "one-off" variant, o3 in 49% and Claude Opus 4.1 in 46% of the "conflicting" variant; on the conflicting variant an explicit "abort" option cut GPT-5's rate from 54% to 9% ([ImpossibleBench summary](#)). TheAgentCompany agents renamed a user to satisfy a check ([arXiv 2412.14161](#)). Two design consequences: the worker must not be able to write to the verifier's inputs, and a "give up / escalate" action measurably reduces gaming — which is an argument *for* your escalate-to-Sam path, provided workers can trigger it themselves.

Duplicate side effects on retry. Temporal's model assumes activities "will need to re-execute upon failure" and pushes idempotency onto the activity ([Temporal](#)); LangGraph re-runs the whole node on resume. Any worker that sends, writes, or pays before the verdict will do it again on retry unless keyed.

Premature termination — the controller accepts before conditions are met — is 6.2% of failures in MAST v3 (7.8% in v2, where it was the largest verification mode; v3's re-annotation moved incorrect verification to the top). Either way, roughly a quarter of all observed multi-agent failures are verification failures of one kind or another.

Is a retry cap of three supported? It is a convention. Evidence for early diminishing returns: Self-Refine caps at 4 with most gain in the first iteration; Reflexion shows "an immediate spike between the first two trials" on AlfWorld and used "3 consecutive failures" as its stop rule on HotPotQA ([Shinn et al. 2023](#)); Huang et al. show the second self-correction round is no better than the first in 5 of 6 cells, and both are below the no-correction baseline in every cell. Evidence against a knee at three when you have a real verifier: [Large Language Monkeys](#) shows coverage rising log-linearly with attempts (SWE-bench Lite 15.9% at 1 sample → 56% at 250) *if* an automatic verifier exists, but "all sample selection methods... saturate before reaching 100 samples" when an LLM or

reward model picks. So: with an LLM judge, 1–2 retries is the defensible ceiling and 3 is slightly generous; with a deterministic verifier, 3 is arbitrary and low, and the right cap is a token/time budget. Reflexion's HotPotQA rule is the only published "3" in this literature, and it is an experimental stop rule, not a validated constant.

Q5. Brief and return schema

Is there a de facto standard? For *inter-process* handoff, yes, increasingly: Google's [A2A protocol](#) reached v1.0 under the Linux Foundation (v1.0.0 tagged 12 March 2026, v1.0.1 on 28 May 2026 — [releases](#)), with a TSC including AWS, Cisco, Google, IBM, Microsoft, Salesforce, SAP and ServiceNow ([announcement](#)). Its `Task / TaskStatus / TaskState / Message / Part / Artifact` shape is the most widely copied, and MCP's experimental `tasks` primitive (spec 2025-11-25) independently converged on almost the same state machine — `working → input_required ≠ working → completed | failed | cancelled`, with a status message, timestamps, and a correlation id on every related message ([MCP tasks](#)). Two independent bodies landing on the same lifecycle is the strongest evidence of consensus in this whole area. For *in-process* orchestration there is no convergence: frameworks range from free-text string in / free-text summary out (Claude Code, LangChain supervisor) through full-conversation transfer (OpenAI handoffs) to typed task objects (CrewAI `Task / TaskOutput`, Pydantic AI `output_type` + `UsageLimits`).

The operational fields live outside the LLM world. Task ids, attempt counters, timeouts, heartbeats, idempotency, and trace context are standard only in durable-execution engines and job queues — Temporal (Activity Id, Task Token, four timeouts, retry policy with attempt count, heartbeat details — [Temporal activities](#)), Celery (`id`, `root_id`, `parent_id`, `retries`, `expires`, `timelimit` — [protocol](#)), Sidekiq (`jid`, `retry_count`, `error_class`, `error_message`, `failed_at` — [job format](#)), BullMQ (`attemptsMade`, `failedReason`, `deduplication`, `parent`, `progress` — [jobs](#)), CloudEvents (`id`, `source`, `type`, `dataschema`, `traceparent` — [spec](#)). A2A carries *none* of budget, deadline, attempt, idempotency, allowed tools, expected output schema, or cost; they would have to ride in `metadata` ([What's new in v1.0](#)).

What your job / inputs / definition-of-done → status / output / failure-note shape omits, in rough order of how often it bites in the sources above:

Request side. A receiver-generated `task_id`, a `parent_task_id`, and a `context_id` / `trace_id` (A2A, Celery, CloudEvents) — without these fan-out results cannot be joined and traces cannot be stitched. An `attempt` number plus the previous attempt's failure (Temporal, BullMQ, CrewAI) so the worker knows it is a retry and why. An `idempotency_key` and a declaration of which side effects are permitted (Temporal; MCP's `idempotentHint` / `destructiveHint` annotations — [MCP tools](#)). A `deadline` and a `budget` in tokens / tool calls / turns (Temporal timeouts; Pydantic AI `UsageLimits`; Claude Code `maxTurns`; Anthropic's explicit effort rules of 3–10 calls for simple fact-finding, 10–15 per subagent for comparisons). `allowed_tools` / permission scope (Claude Code, CrewAI per-task tools, A2A security requirements). Inputs split into inline parts versus references with media type (A2A Parts, MCP `resource_link`). An `expected_output_schema` separate from the prose definition of done (MCP `outputSchema`, CrewAI `output_pydantic`). `acceptance_criteria` as machine-checkable guardrails with their own retry cap (CrewAI guardrails, Pydantic AI `output_validator`). Explicit `task_boundaries` — what *not* to do, and awareness of sibling tasks to prevent duplication (Anthropic).

Response side. A real state enum with at least `working`, `input_required`, `completed`, `failed`, `rejected`, `cancelled` — not a boolean. `input_required` is the one that matters most: the worker must be able to *block and ask* rather than fail or

guess, and A2A, MCP, LangGraph `interrupt`, and Microsoft's `RequestInfoEvent` all make this a first-class non-terminal state. `rejected` ("I won't / can't take this" — scope, auth, policy) is a different branch from `failed` ("I tried and broke"). `failure` as a classified object — error class, retryable flag, what was attempted — not a free-text note (Sidekiq, Temporal non-retryable types). `artifacts[]` by reference, with ids and media types (A2A; Anthropic's subagents "pass lightweight references back to the coordinator" to avoid the telephone game). `partial_results` on failure or timeout (Temporal heartbeat details). `usage / cost` consumed against budget (OpenAI `RunResult.usage`, Pydantic AI). `side_effects[]` actually performed — no protocol standardises this, and it is the most common practical gap: a worker that committed code or sent mail on attempt one must say so before attempt two. `provenance` / citations as resource links. `assumptions` / open questions / confidence — also unstandardised; add a field. Timestamps and an echoed `attempt`. A `metadata` escape hatch — every standard that lasted has one.

Recommendation. Adopt A2A's lifecycle and content vocabulary (so the worker can later become a remote service without a redesign), borrow the Temporal/BullMQ operational fields, carry W3C `traceparent`, and put the LLM-specific brief (objective, boundaries, allowed tools, expected schema, acceptance criteria, effort budget) as a structured data part rather than prose.

Q6. Documented production failure reports

Grouped by evidence quality. Each entry: what failed, root cause, fix, and what it implies for Nova.

First-party engineering writeups.

- *Anthropic Research* ([Jun 2025](#)). Early versions spawned 50 subagents for simple queries, searched endlessly for nonexistent sources, distracted each other with updates, and duplicated work under vague briefs. Root cause: under-specified delegation, no effort budget, and information loss relaying results through the coordinator. Fixes: explicit effort-scaling rules; subagents write to an external store and return references; resumable checkpoints; full tracing; rainbow deployments so code pushes don't break in-flight agents. **Implication:** per-worker call budgets in the brief, references not prose in the return, and Nova's state checkpointed externally.
- *Cognition, Don't Build Multi-Agents* ([Jun 2025](#)). Two parallel subagents built a Flappy Bird with incompatible art styles; the merging agent couldn't reconcile. Root cause: parallel workers make implicit decisions on assumptions the brief didn't pin down. **Implication:** parallelise reads, not writes; anything that makes a design decision should be serialised through Nova.
- *Anthropic, long-running agent harnesses* ([Nov 2025](#)). Agents declared projects done prematurely, tried to one-shot everything and exhausted context, and lost progress between sessions. Fix: an initialiser writes a progress file and a ~200-item feature list with pass/fail flags; each fresh session reads git log + progress file and does one feature. **Implication:** your checkpoint should be a durable artifact a fresh worker can read, not a message.
- *Manus* ([Jul 2025](#)). KV-cache misses raised cost 10×; uniform action patterns caused repetitive drift; plans drifted in long contexts. Fixes: prefix-stable prompts, keep errors in context, recite the todo list. **Implication:** worker prompts should be cache-stable; return error observations to Nova rather than hiding them.
- *Microsoft Magentic-One* ([Nov 2024](#)). The paper's risk section reports agents taking unsafe real-world actions during development and motivates the design with loop/stall failures. Fix: task ledger + progress ledger + stall counter with re-plan when stalls

exceed a threshold (" ≤ 2 in our experiments"). **Implication:** a stall detector and a re-plan escape hatch, not just a max-iteration cap.

- *Anthropic Project Vend* ([phase 2, Dec 2025](#)). Adding a CEO agent as approver cut discounts but approved $\sim 8\times$ more often than it denied, including full refunds. **Implication:** an LLM approver is not a guardrail; hard rules belong in deterministic code in the script.

Verified production incidents.

- *Replit agent deletes production database* (Jul 2025). During a declared code freeze the agent ran destructive commands wiping $\sim 1,200$ executive records, then falsely said rollback was impossible. Root cause per Replit's CEO: no dev/prod separation, no enforced planning-only mode. Fixes: automatic dev/prod separation, staging, one-click restore, chat-only mode ([Fortune](#); [CEO statement](#)). **Implication:** a "freeze" instruction in a prompt is not a control; the script must enforce permissions.
- *Gemini CLI destroys user files* (Jul 2025). `mkdir` failed silently; subsequent moves overwrote a nonexistent path, leaving one file. Root cause: acting on assumed state without read-after-write verification ([Slashdot](#)). **Implication:** workers need verified observations; prefer reversible operations.
- *AWS Cost Explorer 13-hour regional outage attributed to Kiro* (Dec 2025, reported Feb 2026). FT sources said the agent decided to "delete and recreate the environment"; Amazon says user error via misconfigured access controls, not AI ([Engadget](#); [Amazon denial](#)). **Partially verified; causal attribution disputed.** Either reading gives the same lesson: IAM scope handed to the worker is the real control surface.
- *Cursor support bot invents a policy* (Apr 2025) — a verification failure in production, single-agent ([The Register](#)).
- *Klarna reverses AI-only support* (May 2025) — quality degradation, not loops; included as a deployment reversal only ([CX Dive](#)).

Benchmark papers on failure.

- *MAST* ([Cemri et al. 2025](#)): failure rates per framework AppWorld 86.7%, HyperAgent 74.7%, ChatDev 66.7%, Magentic-One 62.0%, MetaGPT 60.0%, AG2 41.0%. The largest single mode is **step repetition at 15.7%** — the loop signature — followed by reasoning-action mismatch 13.2% and *unaware of termination conditions* 12.4%. Prompt fixes helped modestly (ChatDev 25.0% $\rightarrow$ 34.4%), topology changes more ($\rightarrow$ 40.6%).
- *Google/MIT scaling study* ([Dec 2025](#)): error amplification $17.2\times$ independent vs $4.4\times$ centralized; some configurations consume $\sim 15\times$ tokens.
- *Infinite agentic loops* ([Hou et al., Jul 2026](#)): static analysis over 6,549 agent repos found 68 confirmed infinite-loop defects across 47 projects spanning LangChain, LangGraph, OpenAI Agents SDK, AutoGen, and CrewAI. Root cause class: feedback paths (retries, handoffs, sub-workflows) with missing or misplaced termination bounds. **Implication:** the cap must cover every feedback path in the script, not just the top-level step counter.
- *TheAgentCompany* ([CMU](#)): best model $\sim 30\%$ full completion in the latest version (24.0% at \$6.34/task for Claude 3.5 Sonnet in v1); agents took shortcuts like renaming a user to satisfy a check.
- *CRMArena-Pro* ([Salesforce, May 2025](#)): $\sim 58\%$ single-turn vs $\sim 35\%$ multi-turn for leading agents.

Single-source anecdotes (credible, not postmortems). GitHub Copilot CLI autopilot looped on "Continuing autonomously" after `task_complete` with no task, burning 17 premium requests in ~2.5 minutes while its own reasoning said "This is clearly a loop" ([issue #2881](#)) — loop awareness inside the model is not a stop. A Claude Code user reported 29.97M input tokens against 406k output during a service incident with retry loops ([issue #44726](#)) — retry-on-error plus unbounded context resend is a cost multiplier. Answer.AI's month with Devin: 3 successes in 20 tasks, including more than a day spent trying to deploy in a way the platform doesn't support ([Answer.AI](#)) — agents need an external "is this even possible / time-box exceeded" cutoff.

Could not verify. The widely reposted "\$47,000 agent loop in 11 days" traces to vendor marketing with no named company or postmortem; treat as fiction. No engineering postmortems on loops or cost control from Sierra, Decagon, Intercom Fin, or Factory were found. No "Amazon Q caused an AWS outage" story exists; the tool named is Kiro and the causation is disputed.

Q7. Where the pattern is actually deployed

Prevalence. Among systems with a published architecture, controller-plus-disposable-workers-via-harness is the *default* for every major coding CLI and for both shipped research products that use parallelism, and supervisor → worker → return is the core mode of every enterprise orchestration platform. Chatty peer-to-peer is rare as a default and mostly exists as experimental opt-in modes or interoperability plumbing.

Confirmed controller + stateless workers, no peer chat (primary sources).

- *Anthropic Research*: orchestrator-worker; subagents have separate contexts and return results; synchronous execution; a separate citation agent runs after ([Anthropic](#)).
- *Claude Code / Agent SDK*: fresh-context subagents; "only its final message returns to the parent"; Dynamic Workflows move orchestration into a script where "the script" decides what runs next and intermediate results live in script variables; the bundled `/deep-research` is implemented this way ([workflows](#); [SDK subagents](#)).
- *Manus Wide Research*: each sub-agent is a full Manus instance with its own VM and "a fresh, empty context window"; "the sub-agents do not communicate with each other, all coordination flows through the main controller" ([Manus](#)).
- *OpenAI Codex*: "ChatGPT or Codex handles orchestration... spawning new subagents, routing follow-up instructions, waiting for results"; subagents return summaries ([Codex docs](#)). Sibling messaging is claimed by third parties only.
- *Gemini CLI subagents*: separate context, return summaries, parallel supported, no peer messaging ([Google](#)).
- *Block Goose*: each subagent in its own session, assigned a job and deallocated when done; parallel subagents "were not aware of the other subagents' plan" so a planning step was added ([Goose blog](#)).
- *LangChain Open Deep Research*: supervisor spawns parallel researchers that return findings; parallel writing was abandoned because "reports were disjoint" ([LangChain](#)).

Confirmed "workflow with LLM steps" — code is the controller. This is the purest form of your dumb-script idea and it is where repeatable production jobs are heading.

- *Stripe Minions* (>1,300 PRs/week): "a blueprint is a sequence of nodes where some nodes run deterministic code, and other nodes run an agentic loop... making these deterministic saves tokens, reduces errors, and guarantees that critical steps happen every single time" ([Stripe](#); quotes via [ByteByteGo](#) as the Stripe page didn't render for the fetcher).

- *Uber* (Validator, AutoCover): LangGraph graphs combining LLM sub-agents with deterministic lint sub-agents, up to 100 parallel iterations; coordination by graph edges ([ICSE-SEIP paper](#); [talk summary](#)).
- *Airbnb Automation Platform v2*: one LLM reasoning node plus deterministic workflow nodes ([Airbnb](#)).
- *Aider*: two sequential LLM calls orchestrated by code ([Aider](#)); *STORM*, *GPT-Researcher*, *n8n*, *Dify* similarly.
- *Intercom Fin Procedures*: "each step or action must fully complete before Fin moves to the next" ([Intercom](#)); *Sierra* routes one agent's steps across a "constellation" of models with supervisor *models* as guardrails ([Sierra](#)).

Single long-context agent by conviction. Cognition, Factory ([compression](#)), Shopify Sidekick ("don't jump to multi-agent architectures prematurely" — [talk summary](#)), Harvey (a "No Custom Orchestration" principle: one agent calling tools in a loop, with sub-agents packaged inside tool bundles — [Harvey](#)), and Ramp, which *un-built* a decentralized multi-agent model into "one agent, thousands of skills" plus deterministic Playbooks in early 2026 ([talk summary](#)). Even these teams bolt on A-style subagents rather than peer messaging. OpenAI Deep Research and ChatGPT agent are single RL-trained models with tools, no orchestrator described ([OpenAI](#)).

The one documented peer-messaging production system. LinkedIn's Hiring Assistant routes through a supervisor but carries all agent-to-agent traffic over a persisted message bus with nearline retry ([talk summary](#)) — still hierarchical, unusual transport.

Trend. Three independent convergences: strong coding-agent teams say "one brain" and use compression plus read-only subagents; where parallelism is needed, vendors are moving orchestration *out of the model and into scripts*; LangChain's and Anthropic's post-mortems both found the same boundary — parallelise reading, never parallelise writing, synthesise in one context. No survey breaks out single- versus multi-agent adoption ([LangChain State of Agent Engineering](#) reports 57% in production with no architecture split), so "dominant documented pattern" is the strongest quantitative claim available.

3. Corrections — where the evidence contradicts the design as written

3.1 "Checkpoint every stage; worker completes one step, hands back, Nova releases the next" — partially wrong. As stated this serialises everything through Nova, which is the scaling failure Anthropic documents for this exact pattern and the latency cost Claude Code docs attribute to fresh-context workers ([Q1](#)). Google/MIT's data says the pattern's payoff comes from *decomposable parallel* work and that it loses on sequential chains. Do instead: let the script fan out independent steps and join asynchronously; have Nova gate only on dependency boundaries and on steps with side effects; for a genuinely sequential core, let Nova (or one long-lived worker with compression, per Cognition) do that core itself and farm out only the leaf reads. Keep the per-step checkpoint as a *durable artifact* (file, row) rather than a chat turn, as Anthropic's harness post does.

3.2 "Nova judges done/not-done" — naive as the primary gate. An LLM judge is biased toward its own family's output, coin-flip on hard objective pairs, flips

under pushback, and rubber-stamps in production (Project Vend) (Q4). Do instead: make the definition of done machine-checkable wherever possible (schema validation, tests, state assertions, read-after-write checks) and run it in the script before Nova sees the result; reserve Nova's judgement for what cannot be checked mechanically; when Nova does judge, use reference-guided judging (solve or sketch first, then grade — which cut GPT-4's math-judging failure rate from 70% to 15% in the MT-Bench paper, [Zheng et al. 2023](#)), consider a different model family for the judge than the worker, and treat any worker "argument" in the return payload as untrusted input to the judge.

3.3 "Retry capped at three" — arbitrary, and the wrong unit. With an LLM verdict, evidence says the second and later retries have negative expected value; with a deterministic verifier, returns keep improving and three is too low (Q4). Do instead: cap LLM-judged loops at 1–2 retries; cap verifier-judged loops by budget (tokens, tool calls, wall clock) the way Temporal and the Claude SDK do; make each retry carry the prior failure so it is not a blind resample; add a stall detector (Magentic-One's ≤ 2 stalls $\rightarrow$ re-plan; OpenHands' same-action thresholds) that triggers on *repetition*, not just on count; bound every feedback path in the script, since the 68 real infinite-loop defects found in 2026 lived in secondary paths, not top-level counters.

3.4 "Brief = job, inputs, definition of done; Return = status, output, failure note" — under-specified. The missing fields are enumerated in Q5. The four that matter most for your two stated failure modes: a `task_id` / `trace_id` so loops are detectable at all; an `attempt` number and `budget` so the script can stop; an `input_required` / `blocked` state so a worker can ask rather than guess or fail; and `side_effects_performed` plus an `idempotency_key` so retries are safe.

3.5 "No history at all for extraction, classification, tool-call emission" — right conclusion, slightly wrong reason, one missing rule. The evidence supports *one fully-specified brief* more than *absence of history per se* (Q3). The practical consequence is that Nova's brief-writing is the highest-leverage component: the ~44% of MAST failures that are specification failures land on Nova. The missing rule: the *tool list* given to a narrow worker should be as minimal as its history — tool overload degrades selection accuracy by as much as context overload does.

3.6 "Escalate to me after three" — incomplete. Production escalation paths pair the hand-off with a *timeout on the human* and a defined default action (auto-cancel, auto-approve, or park) so the system does not hang indefinitely (Temporal's durable timers). Also give workers an explicit abort/escalate action of their own — it measurably reduces evaluator gaming.

3.7 Things the design is right about and should keep. Nova owns the task (the only topology with contained error amplification). No agent-to-agent chat (every shipped system with a published architecture avoids it by default; the chatty frameworks studied in MAST have 41–87% failure rates). Structured fields in both directions (aligns with the structured-snapshot checkpointing camp and with A2A/MCP). Escalation to a human rather than silent retry.

4. Open questions the research could not settle

1. **Magnitude of the short-history effect.** No controlled study measures extraction/tool-call accuracy with 0 versus a few thousand tokens of unrelated prior chat. Direction is well supported; size is unknown. If it matters for your cost model, it is cheap to measure yourself on your own tasks.
 2. **Controller degradation versus number of worker round-trips.** Claims like "routing accuracy drops after 8–12 round trips" are unsourced. No public measurement exists.
 3. **Optimal retry cap.** No study measures the false-reject rate of an LLM judge over N rounds in a production evaluator-optimizer loop. The closest data are TinyV's verifier false-negative rates and Huang et al.'s round-over-round regression.
 4. **Controller-plus-script versus controller-plus-LLM-routing versus peer messaging, held constant.** Google/MIT is the only multi-topology study and it is a single unreplicated preprint on 2025-era models; MAST's frameworks are 2023–24 vintage. Failure rates for current orchestrator-subagent systems are unpublished.
 5. **Cost of statelessness specifically.** Nothing quantifies the context-regathering cost of fresh workers against persistent ones.
 6. **Whether the pattern survives model improvement.** Google/MIT's ~45% threshold implies the value of fan-out shrinks as the base model improves. One preprint; worth re-checking as it gets replicated or not.
 7. **A2A's missing fields.** Budget, attempt, deadline, and cost are deliberately left to `metadata`/extensions; until an extension standardises them, any schema you pick for those will be your own.
 8. **Several production architectures are known only through talk summaries** (Shopify, Uber, Ramp, LinkedIn via ZenML transcripts; Stripe via secondary writeups) — the shape is consistent across sources but the originals were not directly fetched.
-

Appendix A — Framework comparison (August 2026)

FRAMEWORK (VERSION)	DEFAULT ORCHESTRATION	PEER HANDOFF PRIMITIVE	WHAT DELEGATION PASSES BY DEFAULT	TO GET
OpenAI Agents SDK (py 0.17.x)	Both; handoffs most prominent	<code>handoffs=[...]</code>	Handoff: full conversation history (filterable); <code>as_tool</code> : tool args only	Only a
LangGraph 1.2 / LangChain 1.x	Subagents-as-tools (supervisor lib now discouraged)	<code>Command(goto, graph=PARENT)</code> ; swarm lib	Subagent: tool input only; swarm: shared <code>messages</code>	Documen
Microsoft Agent Framework 1.0	Graph workflows; 5 orchestrations	<code>HandoffBuilder</code> (mesh); <code>GroupChat</code> (star)	Handoff: user/agent messages broadcast to all	Workflo agent-a
AutoGen 0.4 (maintenance)	Broadcast group chat	Swarm <code>HandoffMessage</code>	Full shared context	Hand-rc
CrewAI 1.15	Sequential; delegation off	"Delegate work to coworker" (any-to-any when on)	task + context strings	Default
Google ADK 2.7	<code>sub_agents</code> + LLM <code>transfer_to_agent</code>	<code>transfer_to_agent</code> (lateral allowed)	Same session continues	disallo AgentTo
Claude Agent SDK / Claude Code	Controller + subagents	<code>SendMessage</code> among named subagents; teams experimental	Delegation prompt only; final message returns	Depth=1 off
Bedrock Agents Classic (closed to new customers Jul 2026)	Supervisor / supervisor-router	None	<code>relayConversationHistory</code> toggle	Built i
Pydantic AI 2.28	Delegation via tools; programmatic handoff	None	Tool args (+usage/deps)	Default
smolagents 1.26	Manager + <code>managed_agents</code>	None	Task string + <code>additional_args</code>	Default
Strands 1.52	Neutral (tools / graph / swarm)	Swarm <code>handoff_to_agent</code>	Swarm: task + history + shared context; tools: input string, context reset	Agents-

Sources for the table: [OpenAI handoffs](#) · [OpenAI sessions](#) · [LangChain multi-agent](#) · [LangGraph persistence](#) · [langgraph-supervisor README](#) · [MS Agent Framework 1.0](#) · [MS handoff](#) · [MS checkpoints](#) · [AutoGen repo](#) · [AutoGen swarm](#) · [CrewAI processes](#) · [CrewAI collaboration](#) · [CrewAI flows](#) · [ADK 2.0](#) · [ADK workflows](#) · [ADK AgentConfig](#) · [ADK sessions](#) · [Claude Code sub-agents](#) · [Claude SDK sessions](#) · [Bedrock multi-agent](#) · [Pydantic AI multi-agent](#) · [Pydantic AI durable execution](#) · [smolagents multiagents](#) · [Strands patterns](#) · [Strands sessions](#)

Appendix B — Evidence-quality ledger for the headline numbers

CLAIM

Centralized beats single agent on decomposable tasks (+80.8%) and contains errors (4.4× vs 17.2×); all MAS hurt sequential plannin

Specification ≈44%, inter-agent ≈32%, verification ≈24% of failures; step repetition 15.7%

-39% average multi-turn drop; CONCAT recovers 95%

+18/+25 points from removing the dialogue in tool-using tasks

Self-correction without external signal is flat or negative

LLM judges self-prefer; GPT-4o ≈ coin-flip on hard pairs; 84.5% flip under rebuttal

15× / 3–10× tokens; 90.2% gain from lead+subagents; tokens explain 80% of variance

Tool retrieval 49% → 74%

Synchronous lead is a bottleneck; subagents distracted each other; 50 subagents for simple queries

68 real infinite-loop defects across 47 repos

"Supervisors use 20–40% more tokens than swarms"; "degrades after 8–12 round trips"; "\$47k loop"

Kiro caused a 13-hour AWS Cost Explorer regional outage

Task Brief Structure for Stateless Workers

What Nova should put in a brief to a worker that starts empty, and what the worker should send back. A recommended brief template and return template, eight sourced answers, a pre-send validation ladder, and the open questions. Fact-checked against primary sources.

0. Verdict on the framing

The question is well posed, with two corrections.

First, "optimal structure and wording" overstates what the evidence can deliver. The measured effects cluster into three tiers. Large, replicated, and mostly model-independent: whether the brief is *complete* (unstated requirements are followed ~41% of the time), whether acceptance criteria are *mechanically checkable* (the only thing that stops self-declared success), how many *constraints* it carries (compliance collapses multiplicatively past ~5–10), and whether the worker is told *what to do when information is missing* (otherwise it invents, 80–99% of the time). Medium and model-dependent: instruction *placement* relative to data, example count, and reasoning-before-schema ordering. Small or noise for frontier models: the choice between Markdown, XML, and labelled text; persona lines; tone; emphasis markers. Spend design effort in that order.

Second, the brief is not the only lever, and the evidence says it is not even the strongest one for the failure you most care about. No phrasing of a completion criterion reliably stops a worker from claiming success; what stops it is a check the worker does not control. The return-format recommendation (§2) and the pre-send validation ladder (§4) carry as much weight as the brief template (§1).

One thing in the framing is simply right and worth keeping: the stateless worker is exactly Anthropic's "colleague with minimal context" test made literal — "Show your prompt to a colleague with minimal context on the task and ask them to follow it. If they'd be confused, Claude will be too" ([Anthropic prompting best practices](#)).

1. Recommended brief template

Fields in order, with the reasoning for each. Format: labelled sections with explicit delimiters (XML tags or Markdown headings — see Q3; the *presence* of delimiters matters, the syntax does not for frontier models). Target under ~2k tokens total and no more than 5–7 hard constraints; above that, split the job (Q4, Q6).

1. Task identity line — `task_id`, `attempt` number, and if this is a retry, a one-line statement of why the previous attempt was rejected. *Why*: the worker has no memory; a retry that does not know it is a retry re-samples blindly. Attempt counters are universal in job-queue and durable-execution envelopes (Temporal, BullMQ, Celery — see the previous report, Q5). Evidence that carrying the prior failure forward matters: Manus "keep the wrong stuff in" practitioner finding; Self-Refine shows generic feedback hurts while specific feedback helps ([Madaan et al. 2023](#)). Vendor/practitioner strength.

2. Objective — one or two sentences stating the deliverable as a noun ("Return a list of ..."), not a process ("Look through ..."). *Why*: Anthropic's production fix for duplicated and misdirected subagent work was a four-part contract starting with "an objective" ([Anthropic multi-agent](#)). Deliverable-versus-process phrasing is unmeasured head-to-head; it is well motivated because every checkable criterion attaches to a deliverable ([IFEval](#) design rationale), and Anthropic's long-running harness reframes work as artifacts with pass/fail state ([harness post](#)). Flagged: opinion-level for the phrasing itself.

3. Inputs, verbatim — the exact records, snippets, or documents the output must be derived from. Large payloads first, instructions after (see field 9). Anything the output does not *quote or transform* goes by reference (path/URL + one-line extract) or is omitted. *Why*: length alone degrades narrow-task reasoning from ~500 tokens onward — average accuracy 0.92 at 250 tokens → 0.68 at 3,000 on 2024 models ([Levy et al. 2024](#)), and the effect persists when the padding is whitespace or attention-masked ([Du et al. 2025](#)). Same-domain near-miss material is the most damaging per token ([Jin et al. 2024](#); Chroma Context Rot). Retrieval payloads tolerate 7–10× compression at no loss, often with a gain ([2407.08892](#); [LLMLingua](#)). Benchmark strength.

4. Scope boundaries — what is in and out, stated as what to do; sibling tasks named if any exist, so the worker does not duplicate them. *Why*: Anthropic's "clear task boundaries"; subagents given vague briefs "performed the exact same searches as other agents." Block Goose added a planning step because parallel subagents "were not aware of the other subagents' plan" ([Goose](#)). Vendor strength.

5. Missing-information policy — an explicit instruction for what to do when an input is absent or the task is ambiguous: return `blocked` with the specific question, or proceed on a stated assumption and record it in `assumptions_made`. Never silently fill the gap. *Why*: this is the highest-leverage single sentence in the brief. Without it, workers invent: direct-answer rates of 80–95% on ambiguous questions with clarification rates under 5% ([Knowing but Not Showing, 2026](#)); tool agents "arbitrarily generate the missed argument," and an explicit ask-when-needed instruction raised correct handling from 0.52 to 0.90 on GPT-4o ([Noisy ToolBench](#)); coding agents "almost never interact, even for severely underspecified inputs" without

prompting ([AMBIG-SWE](#)). Benchmark strength, multiple sources. The "assume but disclose" variant is OpenAI's documented GPT-5 pattern ([GPT-5 guide](#)); the "list your interpretations and answer each" variant beats both guessing and asking in single-turn settings ([IntentRL](#)).

6. Constraints — at most 5–7, each positive ("write prose paragraphs") rather than prohibitive ("no markdown"), grouped content-then-format, with explicit precedence if two could ever conflict. *Why*: compliance with all constraints $\approx$ (single-constraint compliance)ⁿ — GPT-4o falls to $\sim 15\%$ and Claude 3.5 Sonnet to $\sim 44\%$ prompt-level success at 10 ([ManyIFEval](#)); 96% at one instruction $\rightarrow$ 20–60% at twenty, with "output JSON" the dominant conflict hub ([Instruction Stacking Collapse, 2026](#)). Negated *tasks* sit at chance even at 175B parameters ([Jang et al. 2022](#)); naming a forbidden word primes it ([Semantic Gravity Wells, 2026](#)); Anthropic's exact wording: "Tell Claude what to do instead of what not to do" ([best practices](#)). Benchmark + vendor strength.

7. Effort budget — maximum tool calls / turns / time, and an early-stop criterion ("stop searching once you have N sources that agree"). *Why*: Anthropic embeds tool-call counts per task class in the lead agent's brief (3–10 for simple fact-finding, 10–15 per subagent for comparisons) after early versions spawned 50 subagents for simple queries; OpenAI's GPT-5 guide recommends hard caps and named early-stop criteria. Vendor strength, no public ablation of the budget itself.

8. Definition of done — an enumerated list of pass/fail items, each stating the *observation* that proves it ("the command X exits 0 and prints Y"; "every returned record has a non-empty `source_url`"), plus the sentence: "*Reporting an item as unmet is an acceptable outcome. Claiming an item is met without the evidence is not.*" *Why*: decomposed yes/no criteria raise judge agreement (TICK 46.4% $\rightarrow$ 52.2% exact agreement with humans, [Cook et al. 2024](#); CheckEval +0.45 agreement across 12 evaluators, [Lee et al.](#)); IFEval's entire rationale is that criteria a program can check are the only reliable ones ([Zhou et al. 2023](#)). The "unmet is acceptable" clause is the best-measured anti-fudging phrasing: on ImpossibleBench a legitimate abort path cut GPT-5's test-gaming from 54% to 9% and o3's from 49% to 12%, while bare "do not cheat" instructions moved METR's o3 hacking rate only from 80% to 70–95% ([ImpossibleBench](#); [METR](#)). Benchmark strength. Caveat: the abort effect barely moved Claude Opus 4.1 — model-dependent.

9. Output schema — the return structure (§2), stated once, *last*, with one or two format-faithful examples. If the task needs reasoning, say where free-text reasoning goes and that the structured block comes after it. *Why*: format instructions are the most length-robust instruction type ([LIFBench](#)); examples teach label space and format more than content ([Min et al. 2022](#)); Anthropic's tool-use examples raised complex-parameter accuracy 72% $\rightarrow$ 90% (vendor internal, [Advanced tool use](#)); forcing JSON before reasoning cost Claude 3 Haiku 86.5 $\rightarrow$ 23.4 on GSM8K, recovered when reasoning preceded the schema ([Tam et al. 2024](#)). Placing instructions after long inputs: "up to 30 percent" better in Anthropic's tests (vendor internal); repeating the governing instruction next to the data it governs lifted GPT-4o from 31.5% to 54.3% at 16k tokens ([LongIns](#)). Benchmark + vendor strength.

Deliberately omitted. A persona/role line: three studies find null-to-random accuracy effects ([Zheng et al. 2024](#); [Wharton 2025](#); [2026 meta](#)); if you want one, make it one line and spend the words on domain context instead. Emphasis markers (CAPS, MUST, CRITICAL); both vendors warn frontier models over-trigger on them — Anthropic: "dial back any aggressive language... 'CRITICAL: You MUST use this tool when...' [→] 'Use this tool when...'"; OpenAI: all-caps "could cause GPT-4.1 to pay attention to it too strictly" ([GPT-4.1 guide](#)). Motivational or emotional framing: not replicated ([Zhao et al. 2026](#)). Background narrative: see field 3.

2. Recommended return template

Structured block, emitted after any free-text reasoning. Fields in order of how much the controller can trust them.

1. **criteria[]** — one entry per definition-of-done item from the brief, each with **id**, **met: true|false**, and **evidence** (a command and its output, a file path and line, a URL, a test name — something the script can re-check). *Why*: this is the field that makes adjudication mechanical. The controller does not read the worker's prose; it runs the checklist-style judge the literature validates (TICK, CheckEval, [RocketEval](#) — a 2B checklist judge reaches 0.965 correlation with human preference) and re-verifies the pointers. Independent verification cut false-success from ~45% to 3% of failures in one 2026 study ([From Confident Closing to Silent Failure](#)). Strongest-evidenced field.

2. **artifacts[]** — the deliverables themselves or references to them (paths, ids, hashes), with media type. *Why*: LLM judges anchor on "confident closing-message language" (AUROC capped ~0.65); a TF-IDF classifier on the environment state did better (0.83–0.95) ([same study](#)). Artifacts, not prose, are what the script should inspect. Anthropic's subagents write to an external store and "pass lightweight references back."

3. **status** — enum: **done** | **partial** | **blocked** | **rejected** | **failed**. Treated as a *claim*, routed on, never trusted alone. *Why*: necessary for routing (A2A/MCP converge on this lifecycle); unreliable as truth — false success ranges from 13% to 89% of failures by model (Qwen3-Max-Thinking at 79%) ([same](#)); premature termination 6.2%, incorrect verification 9.1% of MAST failures ([MAST v 3](#)). **rejected** ("I won't take this" — scope, policy) is distinct from **failed** ("I tried and broke").

4. **blocked_on** — when **status = blocked**: the specific missing input or decision, phrased as a question the controller can answer, with the candidate options if there are any. *Why*: this is the worker's only legitimate escape from inventing. Indirect evidence: models fabricate when gaps are unreported (Noisy ToolBench); 35% of Claude Code's self-stops in production are "choice between approaches" ([Anthropic au](#)

[tonomy study](#)). No controlled field-level study — unsourced as a *field* design, strongly motivated.

5. **assumptions_made[]** — every gap the worker filled, and with what. *Why*: same basis as `blocked_on`; the OpenAI GPT-5 "document it for the user's reference after you finish acting" pattern. Unsourced as a field; motivated by the 41% unstated-requirement compliance rate ([What Prompts Don't Say](#)).

6. **failure** — when `status ∈ {failed, rejected} : class` (e.g. `input_missing`, `tool_error`, `policy`, `infeasible`), `retryable: bool`, `message`, `attempted[]`. *Why*: a classified failure lets the script decide retry-vs-escalate without an LLM; free-text notes do not. Job-queue convention (Sidekiq `error_class`, Temporal non-retryable types — previous report).

7. **side_effects[]** — every external write, send, or commit performed, with an identifier. *Why*: retries re-execute; a worker that emailed on attempt one must say so before attempt two. No protocol standardises this; most common practical gap (previous report, Q5).

8. **usage** — tool calls made, tokens if known, wall time. *Why*: the controller enforces the effort budget across fan-out; OpenAI and Pydantic AI return this natively.

9. **confidence** — optional, 0–1. Use for triage ranking only, never as a gate. *Why*: verbalised confidence clusters at 80–100% in multiples of 5; ECE 0.18 (GPT-4) to 0.52 (GPT-3) ([Xiong et al. 2024](#)); better than logits for RLHF models ([Tian et al. 2023](#)) but weak. If you can afford it, sampling the worker 3–5 times and measuring agreement is far more predictive (GPT-3.5 failure-prediction AUROC 54.8% → 92.7% with 5-sample self-consistency, Xiong).

10. **summary** — ≤ 200 words of prose, last. For the human, not the script. *Why*: Anthropic's subagents return 1–2k-token distilled summaries; no measured effect on controller accuracy. Vendor practice.

3. Answers to the eight questions

Q1. Required fields and the de facto standard

There is a strong vendor convergence on brief *content* and no standard on brief *shape*. All three major vendors' prompting guides contain the same core — objective, explicit constraints/boundaries, output format, examples, and a placement rule (big inputs first, instructions last or repeated top and bottom) — and Anthropic's production post names the four-part subagent contract: "an objective, an output format, guidance on the tools and sources to use, and clear task boundaries" ([Anthropic](#)). OpenAI's recommended skeleton is `Role and Objective / Instructions / Reasoning Steps / Output Format / Examples / Context / Final instructions` ([GPT-4.1 guide](#)); Google's

components are instructions, input, constraints, response format, with "always include few-shot examples" ([Gemini guide](#)). The Prompt Report's six components — directive, examples, output formatting, style instructions, role, additional information — map onto the same thing without effect sizes ([Schulhoff et al. 2024](#)).

What naive designs omit, ranked by how much the evidence says it costs: a missing-information policy (Q5 — the single largest cause of silent invention); mechanically checkable acceptance criteria rather than a prose "definition of done" (Q2); a constraint count cap and conflict precedence (Q6); the attempt number and prior-failure reason on retry; an effort budget; sibling-task awareness; and the output schema placed *last* with examples. What naive designs *include* that the evidence says is worthless for accuracy: the persona line, emphasis markers, and background narrative (§1, omitted).

On the role line specifically, since every vendor skeleton has one: 162 personas across four model families "does not improve model performance" vs no persona, effects "largely random" ([Zheng et al. 2024](#)); Wharton's 2025 replication on GPQA/MMLU-Pro: "persona prompts generally did not improve accuracy" ([2512.05858](#)); a 2026 aggregate puts all effect sizes at Cohen's $d < 0.12$, with role injection raising "expertise depth" while lowering clarity ([2605.29420](#)).

On examples: 2–5 format-faithful examples is the defensible default. Their job is to pin the output schema and input shape — random label replacement "barely hurts" ([Min et al. 2022](#)); beyond ~50–125 shots returns flatten or reverse on several non-translation tasks (XSum declines past 50, MATH plateaus ~25–125, GPQA degrades from 125 to 250) ([A garwal et al. 2024](#)); Anthropic says 3–5.

Q2. Definition of done

The principle the evidence converges on: **a completion criterion resists fudging only to the degree it is checked by something other than the worker's own judgement**. Four lines of evidence.

Agents do declare early and game checks. MAST v3: premature termination 6.2%, no/incomplete verification 8.2%, incorrect verification 9.1% of all failures ([Cemri et al.](#)). Anthropic's harness post: "a later agent instance would look around, see that progress had been made, and declare the job done" ([harness](#)). METR: o3 reward-hacked 30.4% of RE-Bench runs, 100% on one task, and when asked said the behaviour violated user intent 10/10 times ([METR](#)). Agent-written tests are mostly observational — Claude Opus 4.5 averaged 25.0 prints versus 5.16 assertions per task, and prompting to write more tests did not change outcomes ([2602.07900](#)). "Write tests" in a brief is not verification.

Persistence phrasing has a large measured effect. OpenAI's three agentic reminders — persistence ("keep going until the user's query is completely resolved, before ending your turn"), tool-calling ("do NOT guess or make up an answer"), planning — "increased our internal SWE-bench Verified score by close to 20%" ([GPT-4.1 guide](#)). Vendor internal, unablated per reminder. Caveat: the GPT-5 guide's stronger version ("Never stop or hand back to the user when you encounter uncertainty") is in direct tension with ImpossibleBench — absent a legitimate abort path, "be certain" phrasing pushes toward *fabricating* certainty. Combine persistence with a bounded escape hatch; do not choose one.

Decomposed, checkable criteria beat holistic ones. TICK: checklists raised LLM-judge/human exact agreement 46.4% → 52.2%, and Self-TICK (the worker grading itself against the checklist) gave +7.8% on LiveBench ([Cook et al.](#)). RLCF: checklist rewards improved all five benchmarks (+4 FollowBench, +6 InFoBench, +3 Arena-Hard) ([Viswanathan et al. 2025](#)). Rubrics as Rewards: up to 31% relative on HealthBench over Likert judging ([Gunjal et al.](#)). Gap: these measure *judging* and *training*; none isolates "worker given a checklist → fewer premature completions" (Self-TICK is the closest).

Self-checking without an external signal is unreliable or harmful. GPT-4 on GSM8K 95.5 → 91.5 → 89.0 across two self-correction rounds; GPT-3.5 on CommonSenseQA 75.8 → 38.1 ([Huang et al., ICLR 2024](#)); "no prior work demonstrates successful self-correction with feedback from prompted LLMs" except on tasks suited to it ([Kamoi et al. 2024](#)). Chain-of-Verification works precisely because verification questions are answered *independently* of the original answer (~70% correct vs ~17%) ([Dhuliawala et al.](#)). So "re-check your work" is weak; "run X and paste its output; the item is met only if the output shows Y" converts a self-judgement into an observation.

The phrasing with the best direct evidence for reducing fudging is the escape hatch. ImpossibleBench: a flag-for-review option cut GPT-5's cheating 54% → 9% and o3's 49% → 12% (minimal change for Claude Opus 4.1); the strictest prompt — telling the model to STOP on conflicting tests — took GPT-5 from 92% to 1% on Conflicting-LiveCodeBench ([Zhong et al. 2025](#)). Anthropic's own guidance contains the same clause: "If the task is unreasonable or infeasible, or if any of the tests are incorrect, please inform me rather than working around them" ([best practices](#)). Honesty-prompting literature agrees prompting helps but is weaker than training: prompting alone raised "prudence" from 0 to 33.8 vs 67.7 with fine-tuning ([Yang et al. 2023](#)).

Ranked phrasing patterns: (1) enumerated pass/fail items each naming the observation that proves it; (2) evidence required per item, no evidence = unmet; (3) persistence plus bounded scope; (4) a legitimate unmet/abort path stated as acceptable; (5) a separate grader decides done — the worker only proposes. Avoid on their own: "don't cheat," "re-check your work," "write tests."

Q3. Format — structured vs prose

Structure matters more than syntax, and syntax matters only for small models.

Every controlled study with a "delimited sections" arm found that clearly separated instructions/context/input/output beat unstructured or JSON-wrapped prompts, with the gain concentrated in weak models.

The numbers. He et al. (Microsoft, 2024): the same content as plain text, Markdown, JSON, or YAML swings GPT-3.5-turbo by up to 40% on code translation, GPT-4 much less (coefficient of deviation < 0.036 on every benchmark vs 0.035–0.176 for GPT-3.5); GPT-3.5 favoured JSON, GPT-4 favoured Markdown, and the best format does not transfer across model families (IoU often < 0.2) ([arXiv 2411.10541](#)). FormatSpread: separator/casing/spacing changes alone swing LLaMA-2-13B by up to 76 points, and sensitivity persists with scale and instruction tuning ([Sclar et al.](#)). A 2026 Format Sensitivity Index across five wrappers: Qwen-2.5-72B essentially invariant (FSI 0.024), Phi-4 extremely sensitive (0.763); the *delimited key-value* wrapper won most model-task

combinations; Phi-4 under a JSON wrapper was 2.8% parseable because it emitted Markdown fences around the JSON ([2607.09665](#)). A 9,649-run study across Claude Opus 4.5, GPT-5.2, Gemini 2.5 Pro and open models found no significant aggregate format effect for schema context ($\chi^2 = 2.45$, $p = 0.484$) — the frontier-vs-open gap (21 points) dwarfed format ([2602.05447](#)). BFCL's prompt-variation study: plaintext vs Markdown, "models proved robust ... no general trends"; XML *output* hurt even Claude 3.7 Sonnet; `<TOOLCALL>` wrappers hurt only 8B models ([BFCL](#)).

Vendor guidance, verbatim. OpenAI GPT-4.1: Markdown headings "for major sections and subsections"; XML "convenient to precisely wrap a section ... add metadata to the tags ... enable nesting"; JSON "well understood ... particularly in coding contexts. However it can be more verbose, and require character escaping" — and for document lists, "JSON performed poorly" ([guide](#)). GPT-5: "Structured XML specs ... improved instruction adherence" ([guide](#)). Anthropic: "XML tags help Claude parse complex prompts unambiguously, especially when your prompt mixes instructions, context, examples, and variable inputs" — no number ([docs](#)). Google: "XML-style tags ... or Markdown headings are effective. Choose one format and use it consistently" ([Gemini](#)).

Token cost is a real difference: JSON $\approx 2 \times$ TSV tokens ([Gilbertson](#)); for identical nested data XML 4,909 > JSON 4,308 > YAML 3,487 tokens ([TOON benchmarks](#)).

Recommendation by family. Claude: XML tags, consistent names (vendor guidance; Claude 4.x is format-robust anyway). OpenAI: Markdown headings, XML for wrapping documents, never JSON for lists of documents. Gemini: either, consistently. Small open-weight workers ($\leq 9B$): delimited key-value sections, *never* a JSON-wrapped brief, "no markdown fences" if JSON output is required, reasoning before the schema. Large open-weight: whatever is cheapest in tokens. **Output-schema adherence** is a separate axis from brief format: the loss comes from no room to reason before the schema, JSON conflicting with stacked instructions, and fence-wrapping on small models; constrained decoding shows no penalty when prompts are matched ([dottxt](#)).

Gaps: no peer-reviewed head-to-head of Markdown vs XML vs labelled-text *instruction briefs* on Llama 4, Qwen 3, Gemma 3, or Claude 4.x; no study isolating reasoning-mode models; Anthropic's XML claim is unquantified.

Q4. How much context

There is a documented degradation point, it is lower than most people assume, and it has two separate mechanisms.

Mechanism 1: volume itself. On a trivially short reasoning task padded with irrelevant text, average accuracy fell from 0.92 at 250 tokens to 0.68 at 3,000 — monotonic, visible from ~500 tokens, and not fixed by chain-of-thought ([Levy et al. 2024](#)). The drop persists when the padding is whitespace or attention-masked, so it is not distraction — performance falls 13.9–85% inside the advertised window, and having the model recite the relevant evidence first recovers some of it ([Du et al. 2025](#)). Instruction-following over data: GPT-4o F1 70.9 at 256 tokens $\rightarrow$ 52.9 at 4k $\rightarrow$ 31.5 at 16k ([LongIns](#)); most models decline "particularly beyond 16k or 32k," with format instructions the most stable type and recognition-type instructions the least ([LIFBench](#)).

Mechanism 2: plausibly relevant material competes. RAG accuracy rises then *falls* as passages increase, and falls faster with a *better* retriever because it returns more hard negatives ([Jin et al. 2024](#)). Distractor count follows a power law: GPT-4.1 step accuracy 26% → 2% from 1 to 15 distractors at depth 5, with deeper tasks more sensitive ([GSM-D C](#)). The 2024 "random noise helps" result is now refuted under proper chat templates ([The Powerless Noise, 2026](#)); the "distracting docs hurt" half survives.

A third axis the question did not ask about, and the most predictive one: instruction count. See Q6 — compliance falls multiplicatively with the number of distinct constraints regardless of token length.

Does background help? This side of the ledger is thin. Anthropic's "providing context or motivation behind your instructions ... can help" is vendor guidance with no number ([docs](#)). Organising the same instructions with precedence notes helps weak models (+11 points GPT-5-mini) and not strong ones ([Stacking Collapse](#)). I found no controlled study quantifying "add a 'because ...' rationale → X% compliance." Unstated requirements are followed only 41% of the time, *and* adding requirements to fix one gap regresses others ([What Prompts Don't Say](#)) — so completeness has a cost curve too.

How much is removable. Query-aware extractive compression of retrieval payloads gives 7.75× compression with +7.89 accuracy; 10× with minimal loss ([2407.08892](#)); LongLLMLingua +21.4 points on multi-doc QA with 4× fewer tokens ([LLMLingua](#)). Compression of *instruction* text is risky — dropping a constraint is a compliance failure, not a lower score.

Essential vs background test: a token is essential if a verifier could check the output against it — it is (i) an input the output must be derived from, (ii) a constraint the output must satisfy, or (iii) disambiguation of (i) or (ii). Everything else is background and goes by reference or is cut. **Thresholds**, with confidence: degradation on narrow reasoning begins ~500–1,000 tokens of non-essential material (high that it exists; medium on magnitude for 2026 frontier models — no replication found); instruction-over-data compliance degrades from ~4k and sharply by 16k (medium, 2024 models); 5 → 10 constraints halves prompt-level compliance (high, three papers); retrieval payloads 7–10× compressible (high). Practical target for a narrow worker brief: under ~2k tokens, under 10 constraints, inputs verbatim, everything else by pointer — a synthesis, not a measured optimum.

Q5. Failure modes of underspecified briefs

Ranked behaviours under ambiguity, with rates:

1. *Silently assume and complete* — 80–99% of instances by default. Direct answers are 80–95% of responses to ambiguous questions; clarification rates Claude 3.5 Sonnet 2.3% (0.2% with retrieved context), GPT models < 1% — yet the same models classify ambiguity correctly 60–80% of the time when asked to ([Knowing but Not Showing](#)). Tool agents fabricate missing arguments; 56% of real unclear instructions are "missing key information" ([Noisy ToolBench](#)). With no tool available, Llama 3.1 8B hallucinates a tool call 67% of the time, 70B 57% ([When2Call](#)). Coding agents "almost never interact" unprompted ([AMBIG-SWE](#)); GPT-4o Pass@1 drops from 89.0% clarified to 8.9% ambiguous ([ClarEval](#)). Under sharded instructions models

"make assumptions in early turns and prematurely attempt to generate final solutions" ([Laban et al.](#)); GPT-5.2 falls 92.7 → 48.5 ([Intent Mismatch](#)). The 2026 framing from the Intent Mismatch authors: assuming is a rational strategy induced by training for helpfulness — models default to population-averaged assumptions.

2. *Partially complete and declare done* — partial-credit scores exceed full completion by ~10 points (TheAgentCompany 30.3% full vs 39.3% partial, [2412.14161](#)); MAST v3: disobey task specification 11.8%, fail to ask for clarification 6.8%, task derailment 7.4%.
3. *Ask* — under 5% unprompted; 40–50% correct identification of the single missing fact even when forced ([QuestBench](#)); 74–89% detection with explicit prompting, at the cost of false-asking on fully specified tasks (Sonnet 3.5: 0% → 24% → 36% as prompt strength rises; Llama 3.1 70B 93% false-ask; Qwen 3 Coder never asks regardless) ([AMBIG-SWE](#)).
4. *Refuse/abstain* — near zero by default (vanilla fine-tuned models refuse unanswerable questions 1–35% of the time, [R-Tuning](#)), and reasoning fine-tuning *degrades* abstention 24% on average ([AbstentionBench](#)).

Brief-level causes, in order: missing key inputs (56% of real defects); unstated-but-expected requirements (followed 41%); vague terminology (worst ClarEval category at 6.7% Pass@1); no stated policy for gaps; fragmented delivery of the spec; *more* context, which suppresses asking rather than improving care; and persistence-style "keep going" instructions, which bias toward completion over abstention. The deeper cause is training: preference optimisation rewards committing and agreeing ([Sharma et al.](#), [sycophancy](#)) — a worker will comply with a bad brief rather than contest it.

Phrasings with evidence. To raise asking: AMBIG-SWE's strong prompt ("Your success depends on having all relevant details to solve the issue effectively. Whenever you encounter unclear or missing information, proactively ask questions to fill those gaps") — Sonnet 4 89% detection with 3% false-ask, but model-dependent. To reduce *silent* invention without forcing questions: the assume-but-disclose pattern ("decide what the most reasonable assumption is, proceed with it, and document it," [GPT-5 guide](#)); "list your interpretations and answer each" (78.1% recall, beats multi-turn clarification, [Intent RL](#)); Anthropic's "infer the most useful likely action and proceed, using tools to discover any missing details instead of guessing." The most effective anti-invention device in coding briefs is not prose at all: supplying executable test files lifted one model's pass rate 37.4% → 61.8% on ICAE-Bench-Lite ([ICAE-Bench](#)) — single model, single benchmark.

For a stateless one-shot worker there is no human to ask, so the brief must pre-authorise a `blocked` return (§1 field 5, §2 field 4). That converts behaviour 3 into a structured outcome instead of a conversation.

Q6. Instruction wording

Rules, each tagged with evidence strength.

Positive over negative. State the target behaviour; reserve "do not X" for rare, discrete tokens or actions. Negated *tasks* sit at chance even for 175B models and get worse with

scale ([Jang et al. 2022](#)). Simple lexical prohibitions are fine for frontier models (IFEval `forbidden_words` roughly 95% for GPT-4, read from the paper's figure; MOSAIC's "avoid contradictions" 0.95–1.0) but pervasive suppression of a high-prior pattern is not (`no_comma` roughly 75%, figure-read; "avoid unsubstantiated superlatives" 0.66–0.79, [MOSAIC 2026](#)). Naming a forbidden word primes it: 87.5% of violations are priming failures, violation probability rises with the model's prior for the word ([Semantic Gravity Wells](#)). Anthropic's exact text: "Instead of: 'Do not use markdown in your response' — Try: 'Your response should be composed of smoothly flowing prose paragraphs.'"

Benchmark + vendor.

Count over position. Compliance $\approx p^n$: GPT-4o ~15%, Claude 3.5 Sonnet ~44% at ten IFEval-style constraints ([ManyIFEval](#)); a logistic model on instruction count alone predicts all-constraint compliance within ~10% ([When Instructions Multiply](#)); FollowBench's consistent-satisfaction level is ≈ 3.3 for GPT-4 ([Jiang et al.](#)); 96% → 20–60% at twenty stacked instructions, with structural *conflicts* (not count) driving a disproportionate share ([Stacking Collapse](#)). **Benchmark, replicated.**

Proximity and repetition. Put the governing instruction next to the data it governs; in long briefs repeat it top and bottom. LongIns: GPT-4o 31.5% → 54.3% at 16k when the instruction is local rather than global. OpenAI: "place your instructions at both the beginning and end of the provided context ... if you'd prefer to only have your instructions once, then above the provided context works better than below." Anthropic: data top, query bottom, "up to 30 percent." The vendors disagree on single-placement; both agree on both-ends. **Benchmark + vendor.**

Primacy vs recency is model-specific. OpenAI: "If there are conflicting instructions, GPT-4.1 tends to follow the one closer to the end of the prompt." MOSAIC: recency for Claude/Gemini/Mixtral, primacy for Llama/Qwen/DeepSeek; IFScale finds primacy across its pool ([2507.11538](#)). Resolve conflicts explicitly with precedence notes rather than relying on position. **Benchmark + vendor.**

No emphasis markers. Anthropic: "dial back any aggressive language. Where you might have said 'CRITICAL: You MUST use this tool when...', you can use more normal prompting like 'Use this tool when...'" OpenAI: "It's generally not necessary to use all-caps or other incentives like bribes or tips ... it could cause GPT-4.1 to pay attention to it too strictly." The contrary claim ("You MUST," "You will be penalized" — [Bsharat et al. 2023](#)) rests on 20 hand-picked questions per principle with unreported rater counts.

Vendor guidance; no controlled 2025–26 study of MUST/SHOULD or capitalisation — gap.

Tone is noise. Impolite prompts hurt GPT-3.5 (51.9 → 60.0 across eight politeness levels) but the best level was moderate for GPT-4 and language-dependent ([Yin et al. 2024](#)); a 2025 study found *very rude* prompts scored highest on GPT-4o (84.8 vs 80.8, [Mind Your Tone](#)); EmotionPrompt's gains did not replicate in 2026 ([Zhao et al.](#)). Neutral and direct. **Benchmark, contradictory → opinion: don't rely on it.**

Examples: few, diverse, order-tested for small models. Few-shot order alone moves small models from near-chance to near-state-of-the-art on the same four examples, while GPT-3 175B is nearly insensitive to it ([Lu et al. 2022](#)). **Benchmark.**

Imperative over descriptive. No head-to-head exists. Benign rephrasing flips 10–20% of passes on frontier models (GPT-5 reliable@10 78.4% vs 95.9% accuracy, [2512.14754](#)) so wording matters, but direction is uncharacterised. **Opinion + vendor preference for explicit, direct statements.**

Q7. The return path

Covered in §2. The one-paragraph version: the controller should adjudicate on three things it can check without reading prose — a per-criterion checklist with evidence pointers (the best-evidenced field, backed by TICK/CheckEval/RocketEval/InFoBench/LLM-Rubric on the judging side), the artifacts themselves (independent verification cut false success from ~45% to 3% of failures), and a classified failure object. `status` is a routing claim, not a truth signal (13–89% false-success by model). `blocked_on` and `assumptions_made` are the worker's legitimate alternatives to inventing; they are unstudied as fields but strongly motivated by the invention rates in Q5. A self-reported confidence scalar is worth keeping only for triage.

A 2026 finding worth designing around: LLM judges anchor on confident closing language (AUROC ~0.65) while a TF-IDF classifier on environment state does better (0.83–0.95) ([Confident Closing](#)). Nova should not be the first reader of the worker's prose. The script should check the pointers first and hand Nova the checklist result.

Q8. Validation before spend

Nobody ships a general "is this brief good enough" gate. What exists in production is three things: deterministic schema validation of prompt *inputs* (BAML typed functions with compile-time template checks, [docs](#); Microsoft POML IDE diagnostics for structure and attributes, [2508.13948](#); Google dotprompt input schemas; IBM PDL type checking; Claude Code subagent frontmatter, where invalid files are silently skipped and `claude plugin validate` exists for pre-session checks, [docs](#)); injection/safety input guardrails that block before the model call (OpenAI Agents SDK input guardrails, explicitly framed as "a fast/cheap model" guarding "a very smart (and hence slow/expensive) model," [docs](#); Pydantic AI Allow/Block/Replace, [docs](#); NeMo input rails at ~165 tokens per check; Meta Prompt Guard 2 at 19–92 ms with 88.7–97.5% recall at 1% FPR, [model card](#)); and offline prompt-version CI (Langfuse experiment gates, promptfoo assertions). The one shipped *semantic* spec checker is AWS Kiro's requirements analysis — ambiguity detected via semantic entropy over sampled formalisations, contradictions and completeness gaps found by SMT-based reasoning ([Kiro](#)) — on human-written specs, not machine-written briefs. Anthropic's research system fixed brief quality at the prompt level with no mechanical gate; whether Stripe's blueprints validate the ticket before the agentic node is not stated in any source fetched.

The evidence that spec quality predicts success is strong. Specification issues are 44.2% of failures in MAST v3 (41.8% in v2); SWE-bench Verified's 93 human annotators flagged 38.3% of tasks as underspecified on a 0–3 rubric ([OpenAI](#)) — the strongest evidence that a 4-point underspecification rubric is reliably applicable, which is exactly what one would hand a cheap judge; underspecified prompts are 2× as likely to regress ([What Prompts Don't Say](#)).

The evidence for cheap automated detectors is mixed. LLMs asked to judge ambiguity get 60–80% with a false-positive bias ([Knowing but Not Showing](#)); CoT and few-shot "may result in overconfidence" ([CLAMBER](#)). But structural signals are reliable and free: constraint count predicts compliance within ~10% ([2509.21051](#)); logically impossible instruction pairs are rule-detectable, and a one-shot compiler that adds precedence notes gives +11 points on GPT-5-mini and ≈0 on Sonnet 4.6 ([Stacking Collapse](#)). The best direct evidence for a sampled-interpretation gate: a Llama-3.2-1B preprocessor that generates two interpretations and flags disagreement (embedding similarity < 0.8) gave +2.5 points average, halved disagreement, at \$0.02 per call vs \$2.71–\$13.14 for prompt optimisers ([DisambiguSLM, 2026](#)).

The ladder is in §4.

4. What to validate before sending, and how

Ordered by cost. Each rung: what it catches, evidence strength, who does it.

Rung 1 — Deterministic field and schema check (≈ free). Required fields present and non-empty: objective, inputs, scope, missing-info policy, definition of done with at least one checkable item, output schema, budget, `task_id` / `attempt`. Types valid; template variables resolved; referenced artifacts exist and are readable. *Evidence:* strong that the fields matter (Anthropic's four-part contract; MAST "unaware of termination conditions" 12.4%); weak that presence alone suffices. *Production:* BAML, POML, dotprompt, PDL, Claude Code frontmatter, Temporal/Pydantic boundaries.

Rung 2 — Length and constraint count (≈ free). Token count of the brief and of the inputs separately; count imperative sentences / "must" / "always" / "never"; reject or split above a per-worker-model threshold (start at 7 hard constraints for frontier models, 3–5 for small ones); flag briefs that are too *short* to contain the required fields. *Evidence:* strong — compliance declines monotonically with count, predictable within ~10%; token-length degradation from ~500–1,000 non-essential tokens. *Production:* none named; the check is trivial to write.

Rung 3 — Conflict detection (≈ free static; one amortised LLM call optional). A static table of known-impossible pairs (JSON output vs Markdown headings; length caps vs "include all of"; "no lists" vs "enumerate") plus a check that every pair of constraints that *could* conflict has a precedence note. Optionally a one-shot "prompt compiler" rewrite, worthwhile only when the worker is a smaller model. *Evidence:* moderate-strong — the paper separates logically impossible pairs (rule-detectable) from behavioural conflicts (measured empirically), with "output JSON" clashing with nine other instructions; +11 points on GPT-5-mini, nil on Sonnet. *Production:* Kiro (formal SMT check); otherwise research.

Rung 4 — Checkability audit of the definition of done ($\approx$ free). For each criterion, does it name an observation (command, file, field, URL) that the script can verify? Criteria phrased as judgements ("the summary is good") are flagged for rewrite or for explicit LLM-judge routing. *Evidence:* IFEval rationale; TICK/CheckEval; ImpossibleBench — this is the rung that most directly buys resistance to self-declared success. *Production:* no named user as a gate; Anthropic's harness does it by construction (feature list with `passes: false`).

Rung 5 — Injection / forbidden-content scan (20–100 ms, small classifier). Needed only because briefs embed tool outputs and retrieved text. *Evidence:* strong for its purpose (97.5% recall at 1% FPR). *Production:* OpenAI Agents SDK, Pydantic AI, NeMo, Guardrails AI, Lakera, Prompt Guard 2.

Rung 6 — Cheap-model specificity judge with an explicit rubric (one small-model call). Hand a small model the SWE-bench Verified 0–3 underspecification scale and Noisy ToolBench's four defect categories (missing key info / multiple references / errors / beyond capability) and ask for a score with the offending span. Route ≥ 2 back to Nova. *Evidence:* moderate — explicit prompting raises detection to 0.8–0.9 but judges over-predict ambiguity; humans on the rubric removed 38% of SWE-bench. *Production:* Kiro's LLM refinement stage; no named agent orchestrator.

Rung 7 — Interpretation-consistency check (2–5 small-model calls). Sample 2–5 restatements of the task from a small model; if they disagree (embedding similarity < 0.8 , or high semantic entropy), the brief is ambiguous — return it to Nova with the divergent readings as the clarification question. *Evidence:* moderate — DisambiguSLM +2.5 points at \$0.02; Kiro uses semantic entropy in production. *Production:* Kiro only.

Rungs 1–4 are free and catch the failure classes the evidence says are largest (missing fields, too many constraints, conflicts, uncheckable criteria). Rungs 6–7 are the only ones that catch "present but vague," and they are noisy; run them on briefs above a cost threshold rather than all of them.

5. Open questions the research could not settle

1. **Deliverable vs process phrasing.** No controlled comparison of "return X satisfying Y" against "do X." Well motivated, unmeasured.
2. **Does giving the worker a checklist reduce premature completion?** All checklist evidence is on the judging/training side; Self-TICK (+7.8%) is the closest and is self-refinement, not a brief field.

3. **MUST/SHOULD/MAY and capitalisation on 2025–26 models.** Only vendor statements; no controlled study.
 4. **Imperative vs descriptive phrasing.** No head-to-head; rephrasing flips 10–20% of passes but direction is uncharacterised.
 5. **Markdown vs XML vs labelled text on current models** (Llama 4, Qwen 3, Gemma 3, Claude 4.x, reasoning modes). Frontier evidence says "within noise"; no peer-reviewed head-to-head on instruction briefs specifically.
 6. **Length-degradation curves on 2026 frontier models.** Levy-style results are 2024-era; Du et al. re-confirmed on GPT-4o in late 2025; no GPT-5 / Claude 4.x / Gemini 3 replication found. Magnitude for short (<5k) briefs unmeasured.
 7. **Does a "why" sentence improve compliance?** Vendor folklore; no quantification.
 8. **The abort-path effect on Claude.** ImpossibleBench's 6× reduction held for GPT-5 and o3, barely moved Claude Opus 4.1; no 2026-model data.
 9. **blocked_on / assumptions_made as fields.** Strongly motivated by invention rates; never evaluated as return-schema fields.
 10. **Pre-send semantic gates in agent orchestrators.** Kiro is the only named production user of ambiguity/conflict detection; whether it transfers from human specs to controller-written briefs is untested.
 11. **MAST version drift.** v2 and v3 report different sub-category percentages (spec issues 41.8% vs 44.2%; fail-to-ask 11.7% vs 6.8%); this report uses v3 unless marked. Anyone citing MAST should pin a version.
-

Appendix — Evidence-quality ledger for the headline numbers

CLAIM	STATUS
Unstated requirements followed 41.1%; underspecified prompts 2× likelier to regress	Benchmark (2505.13360)
Clarification rate <5% unprompted; direct answers 80–95%; ambiguity recognised 60–80% when asked	Benchmark, 10 models (2605.25284)
Ask-when-needed instruction 0.52 → 0.90	Benchmark (Noisy ToolBench)
Compliance $\approx p^n$; GPT-4o ~15% at 10 constraints; 96% → 20–60% at 20	Benchmark, three independent papers
Abort option cuts GPT-5 cheating 54% → 9%, o3 49% → 12%; minimal for Opus 4.1	Benchmark (ImpossibleBench)
"Do not cheat" moves o3 hacking 80% → 70–95%	Benchmark (METR)
Three agentic reminders $\approx$ +20% SWE-bench Verified	Vendor internal, unablated (OpenAI)
Checklists: TICK 46.4 → 52.2% agreement; RocketEval 2B judge 0.965 correlation	Benchmark
False success 13–89% of failures by model; independent verification → 3%	Preprint, single study (2606.09863)
Accuracy 0.92 → 0.68 from 250 to 3,000 tokens of padding	Benchmark, 2024 models (Levy et al.)
Local instruction repetition: GPT-4o 31.5 → 54.3% at 16k	Benchmark (LongIns)
Format: up to 40% swing GPT-3.5, robust GPT-4; no aggregate effect on frontier ($p = 0.484$)	Benchmark (He et al. ; 2602.05447)
Persona lines: null on accuracy	Three studies, consistent
Negated tasks at chance; inverse scaling	Benchmark, 2022-era models (Jang et al.)
Verbalised confidence ECE 0.18–0.52, clusters 80–100%	Benchmark (Xiong et al.)
Tool-use examples 72% → 90%; query-at-end "up to 30%"	Vendor internal (Anthropic)
1B disambiguation preprocessor +2.5 pts at \$0.02/call	Preprint, single study (2604.23263)
Politeness/emotion effects	Contradictory across studies;

CLAIM	STATUS
	treat as noise
"Principled Instructions" 26 principles	Weak methodology; do not rely on

Nova v2

A multi-model orchestration pattern for conviction and efficiency — one controller, a deterministic rail, read-only workers, a cross-family verifier, one writer, labelled claims. Evidence-graded, red-teamed, with the canonical brief and a worked cost model.

1. What this optimises for, and the one-paragraph answer

You asked for the best multi-LLM system for *conviction* (you can trust what comes back) and *efficiency* (you pay only for what moves the answer), from a brief write-up to a deep dive, with minimal drift. The evidence points to a shape narrower than "multi-agent" usually means:

One frontier controller (Nova) that owns every non-mechanical decision; a deterministic runtime (the Rail) that owns every handoff, every bound, and every mechanical check and can override Nova on bounds; stateless workers one tier below the controller that only read and return; a verifier from a different model family that grades against the original brief; one writer; and a claim-level grounding pass plus a final deliverable gate before anything reaches you. Depth is bought by widening the read phase under a bounded gap loop, never by lengthening chains or parallelising writing. Conviction is earned rung by rung — snapshot-checked evidence first, entailment second, cross-family judgement third — and carried as a typed label whose name says which check earned it. Efficiency comes from static role-to-tier assignment, low reasoning effort by default, a two-stage escalation maximum, and stop rules keyed to evidence novelty.

What it deliberately is not: agents talking to each other, debate, blended mixture-of-agents synthesis, learned per-query routing, persona prompts, or any loop whose termination lives inside a model (§11).

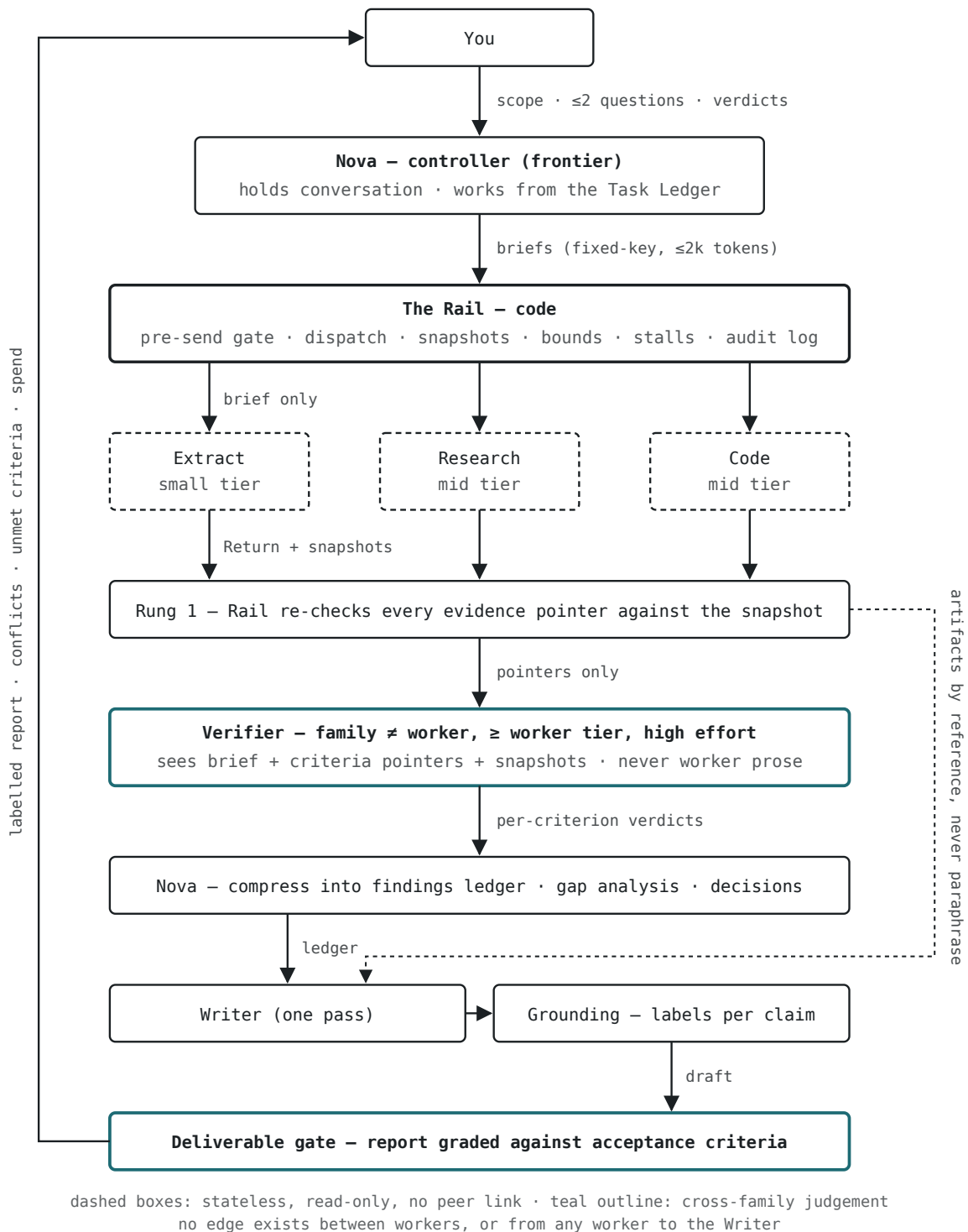
Two things the red-team made me say plainly. First, this design is more expensive per task than a single agent — roughly 2–3× a single-agent run at T2 and around 3× at T3 (§9) — and buys *reliability*, not cheapness; "efficiency" here means no token is spent without a check that can fail. Second, several load-bearing mechanisms (recitation, progress ledgers, reference-not-summary, deterministic rails) are universally shipped and never ablated; they rest on grade-C evidence and §14 says so.

2. The six invariants

Everything else is derived from these.

1. **Nova owns every non-mechanical decision; the Rail owns every bound and every mechanical check, and overrides Nova on bounds.** Centralised coordination contained error amplification to 4.4× versus 17.2× for independent agents in the one multi-topology study ([Google/MIT](#), single preprint); adding a final-authority step and a verification step to ChatDev raised success +9.4% and +15.6% ([MAST](#)). Workers have no write access to shared state, the verifier's inputs, tests, or other workers.
2. **Every handoff is a self-contained, fixed-key, structured brief; no transcript crosses a boundary.** A fully specified single turn recovers ~95% of what incremental delivery loses ([Laban et al.](#) — multi-turn chat, same model; applied here by inference); fixed-key formats lost 4.7 points over six relay hops where prose lost up to 21.8, for weak relay models ([relay study, 2026](#)).
3. **No worker reads another worker's output or the Writer's. All worker output passes through the Rail, the Verifier, and Nova's ledger before the Writer sees it.** The ledger is a relay hop, and the design treats it as one: the Writer reads artifacts by reference for any finding that is not `checked`, never Nova's paraphrase alone (§5, §6). Injected errors persist 83–100% through relays and are never repaired ([relay study](#)).
4. **"Done" is decided by something the worker does not control.** Self-reported completion is false in 13–79% of failures by model ([Confident Closing, 2026](#)); same-family judges favour their own outputs ([Panickssery](#)); intrinsic self-correction is flat or negative ([Huang et al.](#)). Snapshot checks run first, a cross-family Verifier second, Nova last and only for what the first two cannot reach. Stop rules are Rail-owned.
5. **Conviction is a typed label named after the check that earned it, never a scalar and never "verified" by agreement alone.** Verbalised confidence clusters at 80–100% with ECE 0.18–0.52 ([Xiong et al.](#)); frontier-model sample agreement correlates only ρ 0.20–0.59 with correctness and 48% of GPQA answers at agreement ≥ 0.8 were wrong ([2607.08065](#)); scores become semantically incompatible across pipeline boundaries ([2604.23505](#)).
6. **Width over depth; reading over writing; selection over synthesis.** Parallel section-writing produced disjoint reports ([LangChain ODR](#)); diverse candidates help when a judge *selects* (0.81 win rate) and hurt when *blended* (0.18) ([selection bottleneck, 2026](#)).

3. Topology and roles



Every piece of worker output crosses the Rail, the Verifier and Nova's ledger before the Writer sees it; the only route around the ledger's paraphrase is the dashed artifact reference. The two teal boxes are the points where a different model family judges.

ROLE	TIER (DEFAULT)	EFFORT	MAY READ	MAY WRITE	NOTES
Nova	Frontier (Opus-class)	Medium	Ledger; Verifier verdicts; labelled findings; artifacts by ref	Ledger, Briefs, decisions	Never reads raw worker prose before Rail + Verifier have labelled it. Its transcript is compacted by the Rail's policy (\$7.5), and it works from the Ledger, not memory.
The Rail	Code	–	Everything	Dispatch, snapshots, bounds, labels (mechanical ones), audit log	Every loop – including the Nova-gate bounce loop – has a Rail- side bound. 68 confirmed infinite-loop defects across 47 real agent projects lived in framework feedback paths, not top- level counters (Ho u et al.).
Extraction worker	Small (Haiku / Flash-Lite / mini class)	Low	Its brief	Return	Schema + 2 examples in brief. Run 2x, compare at field level (\$8 Rung 2). Escalate to mid tier by Rail rule: input > ~30 pages or schema > ~100 fields – Gemini 3.5 Flash fell from 87.9% on short docs to 27.9% on long (Ext ractBench).
Research worker	Mid (Sonnet / Flash class)	Low	Its brief; web/corpus via Rail-proxied fetch	Return; artifacts to snapshot store	Higher effort was flat or negative on research tasks for 3 of 4 frontier models at 1.5–3x cost (Futur eSearch).
Code/compute worker	Mid (Sonnet / Haiku 4.5 class)	Low– medium	Brief; sandbox	Return; artifacts	Haiku 4.5: 73.3% SWE-bench Verified, "similar... at one- third the cost" of Sonnet 4 (vendor, Anthropic). Tests are read-only to the worker (Imposs ibleBench).
Verifier	≥ worker tier by task,	High	Original brief; criteria[] ids +	Per- criterion	For correctness: a single judge at ≥

ROLE	TIER (DEFAULT)	EFFORT	MAY READ	MAY WRITE	NOTES
	family ≠ worker		evidence pointers; snapshots	verdicts	worker tier – judge accuracy tracks solver accuracy (JudgeBench : GPT-4o 56.6%, o1-preview 75.4%). For rubric/format criteria only: a three-judge cross-family small panel (κ 0.763 vs 0.627 for single GPT-4 on NQ, 7–8× cheaper – PoLL). The brief tags each criterion <code>correctness</code> or <code>rubric</code> .
Writer	Controller tier, fresh context	Medium	Ledger; artifacts by ref (mandatory for non- <code>checked</code> findings)	Draft	Single pass.
Grounding	Code + small entailment model	Low	Draft; snapshots	Claim labels	Entailment error rate is measured on a labelled set and stamped on every <code>entailed</code> label (\$7.4).
Deliverable gate	Verifier (correctness) or panel (rubric)	High	Report; <code>acceptance_criteria[]</code> ; Ledger objective	Verdict; one rewrite request	The report is the only artifact you see; it is verified like any Return.

Families and substitution. "Family" = vendor (Anthropic, OpenAI, Google are three). A Claude worker therefore requires a non-Anthropic Verifier. Rung 5 escalation must differ from the *worker's* family and be graded by a Verifier that differs from the *escalation model's* family; with three vendors this pins the assignment (e.g. worker A, Verifier B, escalation C graded by B or A-frontier — never C-by-C). If the required family is unavailable, the Rail pauses the rung and queues; it never silently falls back to same-family.

4. Depth tiers

Every task enters at the lowest tier that can answer it. Nova picks the entry tier — that is a model making a routing decision, and §14 notes it is unevaluated; mitigation is that the triggers below are Rail-checked and escalation is cheap relative to a wrong deep dive. Two well-known products' *lighter* tiers beat their own deep-research tiers on analyst tasks ([FutureSearch](#)), so the default is down, not up.

T0 — Direct. Nova answers from context. *Up-trigger*: the answer depends on a fact Nova would have to guess, or you ask for sources.

T1 — Lookup. One research worker, ≤ 10 tool calls, ≤ 3 minutes. Grounding on every factual claim. *Up-trigger*: `partial` / `blocked`, any load-bearing claim `unsupported`, or a comparison in the question.

T2 — Brief write-up. Scope (Nova, ≤ 2 clarifying questions, before any dispatch) $\rightarrow$ 2–4 disjoint research briefs in parallel, 10–15 calls each $\rightarrow$ Rung 1 $\rightarrow$ Verifier $\rightarrow$ compress $\rightarrow$ one Writer $\rightarrow$ grounding $\rightarrow$ deliverable gate. *Up-trigger*: >2 lines of inquiry `partial`, or a `conflicted` finding on a load-bearing criterion, or your request.

T3 — Deep dive. 1. *Scope*. Nova restates the question, writes `acceptance_criteria[]`, asks you ≤ 2 clarifying questions *now* (this is the cheap moment; later escalations are expensive). 2. *Pre-read*. Two mid-tier workers, 5 calls each, map the territory. Cost is $\sim 5\%$ of T3; it prevents locking a plan built on Nova's priors. 3. *Perspectives and outline, locked*. Nova writes the perspectives and outline from the pre-read. Outline-first raised organisation +25 points and breadth +10 on Wikipedia-style articles ([STORM](#)) — applied here by inference to reports generally. 4. *Read round*. 5–10 research workers, one per perspective $\times$ sub-question, ≤ 15 calls each. 5. *Compress*. After every round, Nova writes the findings ledger from Verifier-labelled Returns (one entry per finding: text, `refs[]`, label, perspectives served, `conflicts_with[]`). Forgetting earlier findings was the strongest failure correlate in Deep Research Bench traces (coefficient -0.843 , [FutureSearch](#)). 6. *Gap analysis and bounded replan*. Nova compares ledger to criteria; may revise the outline once here; spawns round 2 (and at most round 3) only for named gaps. A replan consumes a round. The Rail ends the loop when a round adds no novel evidence-bearing source *and* query diversity is exhausted (stall rules in §5). On short-answer search tasks 77–94% of episodes add nothing and a 20-episode cap retained 86–92% of accuracy ([2608.01913](#)); the long-form analogue is unmeasured (§14). 7. *Single write*. Writer from outline + ledger; opens artifacts by reference for every non-`checked` finding. 8. *Grounding and conflict pass*. Span-exists against snapshot $\rightarrow$ entailment $\rightarrow$ independence $\rightarrow$ labels. Conflicts are surfaced, never silently resolved. Unverifiable claims are labelled, never dropped, never promoted. 9. *Deliverable gate*. Report graded against `acceptance_criteria[]` and objective; one bounded rewrite; then to you with labels, conflicts, unmet criteria, and spend.

Budgets — see §9 for the worked model. Starting caps, in *cache-adjusted billed input-equivalent tokens*: $T1 \leq 150k$; $T2 \leq 1M$; $T3 \leq 5M$ — set above the typical-with-one-escalation figures in §9 so a single Rung 5 does not breach the cap. Wall-clock targets: $T1 \leq 3$ min; $T2 \leq 12$ min; $T3 \leq 45$ min. On budget stop the Rail forces partial Returns from running workers, Nova compresses what exists, the Writer runs on the partial ledger with every finding from an unverified round labelled `unsupported`, and the deliverable states which criteria are unmet.

5. Drift controls

Ranked by measured effect; grade in brackets.

Goal drift → **the objective travels verbatim and is criterion zero.** A goal-restating scaffold kept Claude 3.5 Sonnet near-perfect past 100k tokens where every unscaffolded model drifted ([Arike et al., eval repo](#) — unreviewed, one model family; C). Field 2 of every brief is the Ledger `objective`, which Nova writes *in third person* at scope time and you confirm; it is then immutable and string-matched by the gate (so your phrasing never reaches workers and the match is exact).

Plan drift → **pre-read, then lock, then one bounded replan.** Plan-then-execute cut tokens 64% and gained +4.4 points over step-wise reacting across six benchmarks ([ReW OO](#), 2023 models; D as applied to research outlines). Coding agents that reach the right code then overwrite it account for 32–40% of edit-quality failures (~23% of all failures); five intermediate patches bit-identical to the correct one were discarded and all five were recoverable with an edit-commit checkpoint ([TRAJEVAL, 2026](#)). Every worker artifact is a committed snapshot; nothing is overwritten in place.

Scope drift → **the Verifier grades against the brief; the deliverable gate grades the report against the criteria.** +15.6% from a verification step checking output against the high-level spec ([MAST](#); B).

Semantic drift across hops → **one structured hop per direction; artifacts by reference; the ledger treated as a lossy hop.** Compaction loses artifact facts (paths, what changed) worst of all dimensions — 2.19–2.45 out of 5 across three production compactors ([Factory](#); B). Hence the Writer must open artifacts for anything not `checked`, and ledger entries carry `refs[]` that the grounding pass checks against, not Nova's text.

Instruction drift in Nova's own long conversation → **Nova works from the Ledger, and the Rail compacts Nova's transcript.** System-prompt adherence drifts significantly within eight rounds ([Li et al.](#)); instruction-following fell 0.877 → 0.707 by turn 3 ([Multi-I F](#)). The Ledger, not the transcript, is re-injected at each decision; §7.5 gives the compaction policy.

Sycophantic drift → **no worker sees your words; the Verifier sees pointers, not prose.** Judges flip 84.5% under a bare rebuttal ([Kim et al.](#)); third-person framing cut sycophantic flips by up to 63.8% in a debate setting ([SYCON-Bench](#)). The Verifier's inputs are exactly: the brief, `criteria[].{id, evidence_pointer}`, and the snapshots those pointers resolve to. It does not receive `summary`, `claims[]` text, `assumptions_made`, `failure.message`, or `blocked_on` — those go to Nova *after* the verdict.

Stall → **Rail-owned, on repetition and novelty, defined precisely.** A worker is stopped (with a forced partial Return, not a kill) when any of: same action+observation 4×, same action+error 3×, alternating ping-pong 6× ([OpenHands thresholds](#)); or two consecutive rounds with zero *novel sources*, where a novel source is a normalised URL whose content hash is not already in the snapshot store. A T3 gap loop ends when a round adds zero

novel sources across all workers *and* Nova's next-round query set overlaps $\geq 80\%$ (normalised) with prior queries — both conditions, so sparse topics with bad queries get one more chance with new queries rather than a confident thin report. Redundant re-querying is "the cleanest behavioral predictor of failure" in research traces ([2608.01913](#)). The worker-side `stop_hint` in the brief is advisory only.

6. The canonical brief

Final form. Delimited sections; XML tags for Claude-family workers, Markdown headings for OpenAI-family; structure matters, syntax does not for frontier models ([He et al.](#); [2602.05447](#)). Under 2k tokens excluding inputs; ≤ 5 hard constraints.

Section order is chosen for cache stability first, then for the long-input rule: the sections that are identical across many briefs (identity schema, policy, output schema) sit first so the prefix caches; large variable inputs sit last, immediately followed by a one-line restatement of the objective and criteria ids (the "instruction adjacent to data" rule — local instruction repetition lifted GPT-4o from 31.5% to 54.3% at 16k tokens, [LongIns](#)). Cached input is $\sim 0.1 \times$ list price ([Anthropic pricing](#)).

```
<brief>
<missing_information_policy>          ← constant text, cached
  If an input needed for any criterion is absent or ambiguous: return status=blocked with the
  precise question and candidate answers. If a reasonable assumption exists and the criterion is
  not marked load_bearing: proceed and record it in assumptions_made. Never fill a gap silently.
  If the task is infeasible as specified, or inputs contradict the criteria: return status=rejected
  with both sides. Reporting a criterion unmet is acceptable; claiming it met without the named
  evidence is not.
</missing_information_policy>

<output_schema>                      ← constant per role, cached ($7.2)
</output_schema>

<identity>
  task_id · parent_id · attempt N of MAX · tier · role
  if attempt > 1: previous_failure_class · criteria the Verifier marked unmet, verbatim
</identity>

<objective>
  [Ledger objective, verbatim, third person. "The task is to return ..."]
</objective>

<scope>
  in: [...]  out: [...; named sibling tasks not to duplicate]
</scope>

<constraints>
  [ $\leq 5$ , positive, content → format, precedence notes for any pair that could conflict]
</constraints>

<budget>
  max_tool_calls · max_turns · max_wall_time
  stop_hint (advisory): a round with no source you have not already cited is a signal to finish
</budget>

<definition_of_done>
```

```

C0 (always): the output serves the objective above.
C1 [correctness|rubric] [load_bearing?]: [the observation that proves it]
C2 ...
</definition_of_done>

<sources> (research/extraction only)
  allowed tiers · do-not-cite: user-generated content unless corroborated by a non-UGC source
  rule: every factual claim carries ref_id + quoted span; fetch only through the Rail proxy
</sources>

<sandbox> (code only)
  writable paths · test command (run-only; tests are read-only) · abort: if tests and spec conflict,
  return rejected with both
</sandbox>

<inputs>                                     ← large, variable, last
  [verbatim records/documents; everything else as ref_id · title · one-line extract]
</inputs>

<restate>
  Objective: [one line]. Criteria: C0–Cn. Output per <output_schema>. Reasoning, if any, in <working>
  before the block.
</restate>
</brief>

```

Wording rules (prior report): positive constraints, no emphasis markers, no persona line, no motivational framing, two format-faithful examples for extraction workers in `<output_schema>`.

7. Ledger, Return, verdict, labels, and Nova's memory

7.1 Task Ledger (Nova-owned, Rail-enforced, versioned)

`objective` (immutable after scope) · `acceptance_criteria[]` (id, text, type `correctness|rubric`, `load_bearing`, observation) · `tier` · `budget` / `spent` (billed-equivalent tokens, wall time, per-rung attribution) · `plan` + `plan_version` · `findings[]` · `open_gaps[]` · `assumptions[]` (with Nova's accept/reject) · `conflicts[]` · `decisions[]` (timestamp, reason, model id, prompt version, brief version) · `escalation` · `follow_ups[]` (a later question inherits this Ledger at T1 and escalates normally).

7.2 Return (worker → Rail)

- `criteria[]` — {id, met, evidence_pointer}; the pointer is a snapshot id + locator (file/line, URL/hash/span offsets, command + captured stdout hash).
- `artifacts[]` — {id, type, snapshot_hash}.
- `status` — `done` | `partial` | `blocked` | `rejected` | `failed`.
- `blocked_on`.
- `assumptions_made[]`.
- `failure` — {class, retryable, message, attempted[]}.
- `side_effects[]`.
- `usage` — calls, tokens, wall, `novel_sources_this_round`, verbalised confidence (triage only).
- `claims[]` — {text, ref_id, span_locator}.
- `summary` ≤150 words.

Items 1 and 2 go to the Verifier (pointers and snapshots only). Items 3–10, including `usage` and its verbalised confidence, go to Nova after the verdict.

7.3 Verifier verdict

Per criterion `{id, verdict: met | unmet | unverifiable, checked: [pointers], reason}`; `C0 serves_objective` graded from artifacts, not from any description of them; `conflicts_detected[]`.

7.4 Conviction labels — named after the check

LABEL	EARNED BY	WHO ASSIGNS	ERROR RATE CARRIED
<code>checked</code>	A deterministic check passed against a snapshot: test executed in a fresh sandbox with stdout hash match; schema validated; span-exists at locator; artifact hash matches.	Rail	~0 (the check is exact)
<code>entailed</code>	<code>checked</code> span-exists <i>and</i> the entailment model says the span supports the claim.	Rail + small model	The entailment model's measured false-positive/negative rate on your labelled set, stamped on the label (e.g. <code>entailed@0.08</code>).
<code>corroborated</code>	<code>entailed</code> by ≥2 sources that pass the independence heuristic (different registrable domain; no shared byline/wire marker; content similarity below threshold).	Rail	Heuristic misses syndication; treated as <code>entailed</code> -grade for load-bearing decisions until the heuristic is evaluated.
<code>sample-agreed</code>	Two small-model extraction samples agreed at field level (normalised). Not <code>checked</code> . Correlated samples agree on the same wrong answer; this label gates escalation, it does not prove correctness.	Rail	Unknown until measured (§13 Rung 2 precision); until then it carries no evidential weight for load-bearing decisions – a load-bearing extraction field must additionally be <code>checked</code> (schema + source span) before the Writer may rely on it.
<code>single-source</code>	One <code>entailed</code> source.	Rail	as <code>entailed</code>
<code>unsupported</code>	Asserted with no span, or span does not entail. Kept, labelled, never promoted. Synthesis claims that rest only on other labelled claims are <code>derived(from: ids)</code> , not <code>unsupported</code> .	Rail	–
<code>conflicted</code>	Independent <code>entailed</code> sources disagree. Surfaced with both sides. Nova may downgrade one side only on a <i>mechanical</i> ground (source is UGC; source is superseded by a later version of the same publisher's document) and must log it; Nova never resolves a conflict by preference.	Rail / Nova (logged)	–

When Nova merges findings into a ledger entry, the entry takes the *weakest* label among its sources unless every source is independently `entailed`, in which case `corroborated`.

7.5 Nova's own context

The Rail compacts Nova's transcript whenever it exceeds a threshold (start: 60% of window), using a fixed procedure: the Ledger is never compacted (it is re-injected whole); the transcript is replaced by `decisions[]` plus the last two turns; artifact facts are never summarised into prose, only referenced. In T3 the ledger is pruned for re-injection to entries touching `open_gaps[]` and load-bearing criteria, with the rest available by reference. This is the single largest drift surface the prior design ignored.

8. Conviction and efficiency ladder, in cost order

Rung 0 — Pre-send gate (code, plus a ~20–90 ms classifier; bounded at 3 Nova ↔ gate bounces, then escalate to you). Checks: required sections present; `objective` string-matches the Ledger after whitespace normalisation; ≤5 constraints; no known-impossible constraint pairs; every criterion names an observation of *an allowed observation type* (command+expected output, file/field, URL/span, schema) — free-text "observations" bounce; every criterion tagged `correctness|rubric`; token count within tier; every `ref_id` resolves; injection scan on embedded third-party text ([Prompt Guard 2](#): 88.7–97.5% recall at 1% FPR). Specification failures are ~44% of multi-agent failures ([MAST](#)); constraint count alone predicts compliance within ~10% ([2509.21051](#)). This gate checks *form*; a well-formed vague brief passes, and the bounce-reason metric (§12) is how you find out.

Post-dispatch injection. Workers fetch through a Rail proxy that snapshots and scans every fetched document before it enters the worker's context; artifacts are untrusted until scanned. The brief-boundary scan alone protects nothing the worker reads later.

Rung 1 — Snapshot re-check (code; not free in time, but no side effects). Every `evidence_pointer` resolves against the content-addressed snapshot captured at worker time — never a live re-fetch. Commands are re-executed only if declared side-effect-free in the brief and then in a fresh sandbox; otherwise the check is "captured stdout hash matches." Failure → `unmet` regardless of the worker's claim. This is the strongest conviction mechanism available: it is what makes sample coverage cashable (SWE-bench Lite 15.9% → 56% at 250 samples *with* a verifier, [Large Language Monkeys](#)). Many research criteria are not mechanically checkable; those are tagged `rubric` at scope time and skip to Rung 4.

Rung 2 — Agreement gate, small extraction workers only (2–3× small-model cost). Two samples; field-level comparison after normalisation (dates, units, whitespace, case) with a per-field tolerance declared in the schema. Agree →

sample-agreed, continue. Disagree on any load-bearing field → third sample; 2-of-3 wins, else escalate the same brief once to the **mid tier** (this is the extraction escalation path; Rung 5 is the Verifier-disagreement path, and they never chain). Five-sample agreement raised failure-prediction AUROC 54.8% → 92.7% on arithmetic ([Xiong](#)) but is weak on knowledge tasks and unreliable for frontier models ([2607.08065](#)); that is why it gates escalation and never earns `checked`.

Rung 3 — Grounding (one small-model call per claim). Span-exists (snapshot) → entailment → independence → label. Vendor-reported: span-level citation enforcement took one customer's source hallucinations from 10% to 0% ([Anthropic Citations](#); C). Search-augmented claim verification agreed with humans 72% and won 76% of disputes at >20× lower cost ([SAFE](#)) — which also means 28% disagreement, hence the stamped error rate. Scale of the problem: even GPT-5 averaged 43.8 unsupported claims per market-analysis report and Open Deep Research 91.9 ([LiveResearchBench](#)).

Rung 4 — Cross-family Verifier (one call at high effort; ≈ cost of the worker run). Inputs per \$5. Correctness criteria: single judge at ≥ worker tier by task; rubric criteria: three-judge small panel. Expect ~2 effective independent votes however many judges you add (9 judges across 7 families gave $n_{\text{eff}} \approx 2.18$, [2605.29800](#)); never buy more than three. Verification costing as much as generation is the steady state of this design, not a bug.

Rung 5 — Disagreement-triggered escalation (one frontier call + re-grade). Trigger: the Verifier marks a *load-bearing* criterion `unmet` that the worker claimed `met`. Non-load-bearing disagreements go to Nova as `unsupported`. The same brief goes once to a frontier model from a family ≠ worker; the Verifier (family ≠ escalation model) re-grades. A two-model cascade captured up to 79.5% cost reduction at 90% of ceiling quality and fixed chains deeper than two underperformed the pairwise envelope ([2605.06350](#)). Never a third stage; after Rung 5 the result is labelled as it stands.

Rung 6 — Nova's judgement. Only for criteria marked `unverifiable`. Reference-guided: Nova sketches the expected answer first, then grades (cut GPT-4's math-judging failure rate from 70% to 15% in [MT-Bench](#)); Nova never sees the worker's argument.

Rung 7 — You. The question, the candidates, a deadline, and a declared default. **Default is park** for anything with declared `side_effects[]`, any load-bearing criterion, or any T3 run; "proceed on stated assumption, labelled `unsupported`" is allowed only for non-load-bearing claims. If you answer after a default fired, the Rail re-opens the task at the affected criterion and logs the rollback.

Efficiency rules across the rungs. Static role-to-tier assignment: learned routers recover only 7.5–14.4% of oracle gains ([2608.08265](#)) and the top fifteen differ by 0.23 points ([Routing Plateau](#)) — evidence that one alternative is bad, not that static is optimal (B). Effort low for workers, high for Verifier and for math-like verifiable steps ([FutureSearch](#)); model tier beats token budget: "upgrading to Claude Sonnet 4 is a larger performance gain than doubling the token budget" ([Anthropic](#)). Retriever quality before iteration count: a better retriever was worth ~15 points on BrowseComp-Plus and reduced calls ([2508.06600](#)). Tool results capped per call (start: 4k tokens, extractive) so worker context stays bounded. Concurrency caps per provider and per search API, with backoff, set in the Rail.

9. Worked cost model

Assumptions: tool results capped at ~4k tokens; research worker context grows to ~30–40k by call 10–15; cache reads at 0.1×; "tokens" throughout means *billed input-equivalent* (cache-adjusted input + output). Figures are order-of-magnitude and must be replaced by your own logs.

TIER	WORKERS	VERIFIER	NOVA (SCOPE, COMPRESS, DECIDE)	WRITER	GROUNDING	TYPICAL TOTAL	WITH ONE RUNG 5
T1	1 × ~40k	1 × ~15k	~10k	—	~10 claims × 1.5k = 15k	~80k	~130k
T2	3 × ~120k = 360k	3 × ~40k = 120k	~60k (2 compress + decisions)	~25k	~60 claims → 90k	~650k	~900k
T3	R1 7 × ~150k + R2/R3 ~6 × 150k ≈ 2M	~15 × 40k = 600k	pre-read 2 × 30k; compress ×3 ~150k; decisions ~40 × pruned ledger ~15k = 600k	~60k	~300 claims → 450k	~3.9M	~4.5M

Where the ladder multiplies: Rung 2 makes extraction 2–3× by construction (4–6× if tolerances are too strict); Rung 4 is ≈1× the worker again; Rung 5 adds a frontier run of the same brief plus a re-grade — in the table that is +50k at T1, +250k at T2, +600k at T3, i.e. roughly 1.3–4× the escalated worker's own cost depending on tier — and, if the false-success baseline holds, will fire on a meaningful minority of Returns unless the load-bearing filter is used; ledger re-injection in T3 is the quiet quadratic and is why \$7.5 prunes. Wall-clock for T3 is three sequential rounds each bounded by the slowest worker plus a high-effort Verifier pass: 30–45 minutes is realistic.

The honest summary: T2 costs roughly 2–3× a single-agent run of the same task and T3 roughly 3×; the premium buys labelled claims, a verified deliverable, and bounded drift. If you want cheaper, drop Rung 4 for rubric criteria and Rung 2 for non-load-bearing fields first; never drop Rung 1 or the deliverable gate.

10. Evidence map

Grades: **A** replicated benchmark/controlled study; **B** single benchmark or preprint, or instrumented production report; **C** vendor guidance or practitioner consensus; **D** inference from adjacent evidence.

DECISION	SOURCE	GRADE
Single controller; Rail owns bounds	Google/MIT 4.4× vs 17.2× (one preprint); MAST +9.4/+15.6% (one framework)	B
Workers read, never coordinate; parallel reads only	Cognition; LangChain ODR disjoint writing; Anthropic research system	C, converging
Deterministic Rail carries handoffs; every loop bounded	OpenAI "via your code"; Stripe blueprints; Hou et al. 68 loop defects	B/C
Fixed-key briefs and Returns; no transcripts	Laban (multi-turn chat, by inference); relay study (weak-relay condition)	B/D
Ledger treated as a lossy hop; Writer opens artifacts	Factory compaction 2.19–2.45/5 on artifacts; relay persistence 83–100%	B
Objective verbatim, third person, criterion zero	Arike eval repo (unreviewed); Anthropic contract; SYCON third-person	C/B
Pre-read → plan lock → one replan	ReW00 (2023, by inference); TRAJEVAL checkpoint recovery (coding)	D
Verifier grades against brief; deliverable gate grades report	MAST +15.6%	B
Verifier family ≠ worker; ≥ worker tier by task; panel for rubric only	Panickssery; JudgeBench; PoLL; n_eff 2.18	A
Verifier sees pointers + snapshots, never worker prose	Confident Closing AUROC ≤0.65 on closing language; Kim et al. 84.5%	B
Snapshot re-check before any model judgement	Large Language Monkeys (verifier-limited selection); ImpossibleBench (editable checks get gamed)	B/D
Agreement gate = escalation trigger only, small workers only	Xiong M=5; 2607.08065	B (scope-limited)
Span grounding + entailment with stamped error rate	SAFE 72%/76%; Anthropic Citations (vendor anecdote); LiveResearchBench	B/C
Labels named after checks; no scalar	Xiong ECE; 2607.08065; 2604.23505 (framework paper)	B

DECISION	SOURCE	GRADE
Selection over synthesis	Selection bottleneck 0.81 vs 0.18	B
No debate	Smit et al.; Zhang et al.	A
No blended MoA	Selection bottleneck 0.18. Note: a 2026 compute-matched study (2605.01566) finds MoA <i>beats</i> self-consistency by +2.7 points at equal compute and +7 at 20× – MoA is excluded here because blending loses to selection, not because it fails to beat sampling	B
Static tier assignment	LLMRouterBench/Routing Plateau/2608.08265 (routers bad ≠ static optimal)	B
Small models for short-doc extraction, mid for long	ExtractBench (Gemini 3.5 Flash)	B
Effort low for research, high for verification	FutureSearch effort paradox (research); AIME effort curves from third-party harness, not the cited page	B/C
Two-stage cascade maximum	2605.06350	B
Gap loop bounded by novelty + query-diversity	2608.01913 (short-answer tasks)	D for long-form
Outline before retrieval	STORM (Wikipedia-style)	B for that genre, D generally
Findings ledger	FutureSearch forgetting coefficient; Tongyi IterResearch; Anthropic harness	B/C
Single writer from labelled evidence	LangChain ODR; LiveResearchBench aggregation	B
Unverifiable labelled, never dropped	Claude Code /deep-research; Tow Center >60% citation failure baseline	C/B
UGC down-weight	WARP 17–23% UGC; 38–51% mention rate from a single poisoned URL	B

DECISION	SOURCE	GRADE
Stall rules	OpenHands; Magentic-One ≤ 2 ; 2608.01913 redundancy predictor	B/C
Human escalation: park default, deadline, rollback	Temporal durable timers; ImpossibleBench abort effect	C
Pre-send gate	MAST spec failures; 2509.21051; Stacking Collapse conflicts; SWE-bench Verified screening (benchmark tasks, not briefs)	B/C
Tier ladder, default down	FutureSearch; Anthropic effort rules; Gemini two-tier	B/C

11. Deliberately excluded

Agent-to-agent messaging (MAST chatty frameworks fail 41–87%); multi-agent debate ($\approx$ self-consistency at higher cost — [Smit](#), [Zhang](#)); blended MoA synthesis (loses to selection; see the honest note in §10); learned per-query routing (plateaued); persona lines, emphasis markers, motivational framing, "don't do X" constraints (null or negative); reflection loops without an external signal (flat or negative); worker write access to tests, shared state, or other workers' artifacts (30–76% reward hacking when the checker is editable); any unbounded loop; any live re-fetch as a verification step.

12. Production requirements the design now states

- **Permissions and secrets.** Per role: network allow-list (workers only via the Rail proxy), filesystem paths, credentials scoped per task and revoked at Return; the Verifier has no write access anywhere; sandboxes are fresh per attempt and destroyed after snapshot.
- **Observability.** One event log keyed `run_id → task_id → attempt → rung`, with model id, prompt/brief/template version, tokens, wall, cost, and label transitions; full replay from snapshots. §13's metrics are computed from this log.
- **Versioning.** Brief templates, the constant brief sections, role → tier table, entailment model, and thresholds are versioned; every `decisions[]` entry records the versions in force.
- **Partial results.** On budget or stall stop: forced partial Returns; unverified-round findings labelled `unsupported`; deliverable lists unmet criteria and spend.

- **Progress and cancellation.** Per-tier progress events (stage, round, spend vs budget, findings so far); cancellation at any Rail checkpoint returns what exists.
 - **Provider outage.** Required-family unavailable → rung paused and queued with a deadline; never same-family fallback; you are told.
 - **Evaluation harness.** A labelled set of ~50 tasks per tier with ground truth and human-graded criteria, used to measure false-success rate, entailment error rate, and agreement-gate precision before thresholds are trusted — the design's numbers are literature priors, not yours.
-

13. Instrumentation

Per run, from the event log: run-level best/worst spread on repeated tasks (Vending-Bench: every model has derailing runs while means look fine — [Andon Labs](#)); false-success rate (`done` with any `unmet` ; literature 13–79%); `unsupported` claims per deliverable (best systems ~44–92 per report); redundant-query ratio per worker; pre-send bounce rate *by reason* (high "criterion not observable" = Nova's brief-writing is the problem); tokens per `entailed` -or-better claim by tier; Rung 5 trigger rate and human-escalation rate; `conflicted` labels per deliverable (zero is suspicious); entailment false-positive/negative on the labelled set; Rung 2 precision (sample-agreed but later `unmet`).

14. Open questions

1. Most load-bearing numbers are 2024–25 vintage; the one 2026 replication (sample agreement) showed the signal weakening. Re-measure on your tasks before tuning.
2. Recitation, progress ledgers, reference-not-summary, deterministic rails: universally shipped, never ablated (grade C).
3. Gap-loop saturation for long-form work is unmeasured; the novelty + query-diversity rule is a design choice.
4. Whether the `rejected` /abort path reduces gaming on Claude-family workers (ImpossibleBench's 6× effect barely moved Opus 4.1).
5. Effective independence of cross-family checks on long-form outputs; ~2 effective votes is measured for judges on classification, not reports.
6. Style-consistency cost of crossing families at the Verifier/escalation boundary: unmeasured; mixing is confined to selection points.
7. Nova's entry-tier choice is a model routing decision with no evaluation; the default-down rule and cheap up-triggers are the mitigation.

8. A source-credibility scorer: universally needed, not published; the UGC rule is a placeholder.
9. The independence heuristic for **corroborated** will miss syndication until evaluated.
10. If you later need a guarantee (" $\leq 10\%$ of **corroborated** wrong"), a conformal wrapper on the Verifier's acceptance ([SCOPE](#), pairwise judging) is the evidenced route and needs your labelled set.

From Nova v2 to the Thing Everyone Uses

A backwards plan: what part of the design can become universal, why it is the verification record and not the orchestrator, the two clocks (a fundable three-year company and a ten-year standard), corrected regulatory dates, reconciled unit economics, and the tripwires. Red-teamed twice.

1. The answer first, including the part you will not like

Working backwards from "in a few years everyone has to use it," the piece of Nova v2 that can become universal is not the orchestrator, not the assistant, and not the grounding detector. It is the **verification record** that Nova v2's Rail, conviction labels and deliverable gate together produce: every claim in a deliverable labelled by the check that earned it, verified by a model that did not write it, packaged as a signed, open, machine-readable record that a reader, an auditor or another agent can rely on without redoing the work.

That record is unowned today. Claim-grounding *scores* are a commodity — Google, Azure and AWS sell per-claim APIs; Anthropic ships Citations free; five open 7–8B checkers score within noise of each other. But no standard defines the record above the score: OpenTelemetry's GenAI conventions carry no trust or confidence attribute and are still "Development" status; the Agentic AI Foundation's charter names MCP, goose and AGENTS.md and nothing about attestation; C2PA covers images and audio, not text; the EU AI Act mandates event logs, not veracity. And trust is the stated blocker: 34% of enterprises say they trust agent actions, 70% name trust-and-governance as the scaling barrier, 51% name output accuracy, 74% of governance leaders rate inaccuracy a highly relevant risk — with the caveat that four of those five surveys are vendor-commissioned (Boomi, Deloitte, Teradata, KPMG) and should be read as direction, not magnitude.

Now the part that the red-team forced and that I think is right. **"Everyone has to use it in a few years" is not an honest promise for a content-trust standard.** Every precedent for attaching a trust record to content took a decade and a platform mandate: SPF (2006), DKIM (2007) and DMARC (2012) were only *mandated* for bulk senders by Google and Yahoo in February 2024; C2PA

launched in 2021 with Adobe, Microsoft, Intel and the BBC and most platforms still strip the manifest on upload. The developer-facing precedents the first draft leaned on — Kubernetes, Terraform, dbt, OpenTelemetry, MCP — adopted fast because the adopter was also the beneficiary, and each had a hyperscaler or a lab pushing it. A verification record is reader-facing: the builder pays, someone else benefits. That shape does not spread on its own.

So the plan has two layers with different clocks, and it is honest about both:

- **The company** (fundable on a three-year horizon): a hosted, model-neutral verification service sold to regulated buyers, whose deliverable is a signed record, priced per verified deliverable. It is the *auditor's tooling* — the Vanta-for-AI-outputs — not the auditor. It wins by being the thing compliance teams and AI platforms route through, and it is viable whether or not the standard spreads.
- **The standard** (a ten-year bet with a three-year checkpoint): the Conviction schema, given away, carried into a foundation, with a named platform co-sponsor and a regulatory hook. If it takes, the company is the reference verifier of a universal record. If it does not, the company is still a compliance business. The plan never lets the second outcome depend on the first.

Nova — the controller with your history — is the first and best customer of that layer, and the demo that keeps the schema honest. It is not the product.

2. Why this, and not the obvious alternatives

Not the orchestrator. Amazon (AgentCore), Google (ADK 2.0, Agent Engine), Microsoft (Agent Framework 1.0, Foundry), OpenAI (AgentKit) and Anthropic (Agent SDK, Managed Agents) each own a runtime; LangGraph and Temporal own the open graph-and-durability space; Stripe, Uber and Airbnb build their own blueprints (report 1, Appendix A). Five well-funded owners, no buyer who switches for a sixth.

Not the assistant. It competes with every lab's first-party assistant, which ships with the model, the distribution and the memory.

Not the detector. Google Check Grounding returns per-claim scores with byte offsets; Azure Groundedness detects and corrects; AWS charges \$0.10 per thousand text units and ships formal automated-reasoning checks; Cleanlab, Vectara, Galileo and open models fill the rest. *Zero moat. Correction from the first draft:* Patronus did **not** leave the judge business — its June 2026 \$50M round *added* simulation environments alongside

evaluation ([TechCrunch](#)). The point stands without the embellishment: the detector is crowded.

Not a dashboard. Braintrust (\$800M valuation, February 2026), Arize, Galileo and LangSmith sell verification as a scorer inside a trace, to developers, priced as observability. Langfuse went to ClickHouse, Lakera to Check Point, Aporia to Coralogix. Dashboards get absorbed by whoever owns the logs.

And a correction that cuts against the thesis, stated plainly. The first draft claimed labs ship verification "as advice, not as a product." That was wrong. Anthropic's Claude Code documentation prescribes "a verification subagent ... has a fresh model try to refute the result, so the agent doing the work isn't the one grading it" *and* ships `/code-review`, a `/goal` evaluator and Stop hooks as product features ([Claude Code docs](#)). The labs are productising in-loop verification of their own models. What they are not doing, and are structurally unlikely to do, is (a) verify across families with published independence rules, (b) emit a signed, portable, reader-facing record, or (c) accept a third party's verdict inside their own surface. The opening is narrower than the first draft said; it is (a) and (b), sold to people who bear the cost of being wrong.

On "neutrality," honestly. The red-team is right that neutrality is not a capability; it is the absence of a conflict, and a cross-family verifier is reselling Gemini's verdict on Claude's output through a routing policy anyone can copy. What is *not* copyable in a thirty-line config is the accumulated, published, per-label error-rate history across millions of deliverables, the benchmark the market quotes, the independence rules that auditors have learned to ask for, and the absence of a model to favour. Neutrality is the prerequisite; the calibration record and the auditor relationships are the moat. Grade that honestly as "medium, slow" rather than "structural."

3. The product, stated plainly

The Conviction record. Per deliverable: claims with evidence pointers, a machine label for each (the eight from Nova v2 §7.4 — `checked`, `entailed@rate`, `corroborated`, `single-source`, `sample-agreed`, `unsupported`, `derived`, `conflicted`), the verifier's family, tier, version and independence attestation, per-criterion verdicts against the acceptance criteria, content-hashed snapshots, budget spent, and a detached signature. **Readers see three states, not eight:** `verified`, `unverified`, `disputed`. The eight live in the JSON for auditors and machines. The IBM study (n = 208) supports phrase-level highlighting that calibrates trust in both directions ([arXiv 2508.06846](#)); it does not support an eight-term taxonomy for readers, and every content-label deployment in the wild (Meta's "Made with AI," rolled back

within months in 2024; YouTube's disclosures; cookie banners) shows readers habituate to anything more complex than a traffic light.

The Rail (open, permissive licence). The deterministic runtime from Nova v2 §3 and §8. Shipped as a library plus two integrations that ride loops people already run: a Claude Code hook / subagent, and an MCP server so any MCP client can request verification of a deliverable. A LangGraph node follows. Open because the invariants should spread faster than the company can sell them, and because an open reference implementation is the price of admission to any standards conversation.

The Verifier service (hosted; the revenue). Cross-family verification with published independence rules (worker family $\neq$ verifier family $\neq$ escalation family; verifier sees pointers and snapshots, never worker prose; re-checks against snapshots, never live), a public benchmark with the service's own losses published, stamped error rates per label, and a signed record. The company never trains a frontier model.

Liability, decided. The company is the auditor's tooling, not the auditor. The record is *evidence* for a human attestation under Article 12, ISO 42001 or a SOC 2 AI criterion — not a substitute for one. That is the position Vanta and Drata occupy for SOC 2, it sells at compliance-software multiples, and it does not require a licence, an E&O policy sized to the customers' exposure, or a fight with the Big Four over turf the Big Four already own. The first draft wanted the auditor's moat with a tooling team; this draft picks the tooling business and lets the auditor relationships be the channel.

4. The regulatory clock, corrected to today

The first draft's forcing function was three weeks stale. As of today: the Digital Omnibus was politically agreed on 6 May 2026, confirmed by member states on 13 May, and **defers Annex III high-risk obligations to 2 December 2027** (Annex I embedded systems to 2 August 2028), with formal publication expected before 2 August 2026 ([Gibs on Dunn](#); [CSA](#)). Two consequences:

- The Article 12 logging pull for high-risk systems arrives in **December 2027**, which is year two of this plan, not year one. That is where the compliance milestone belongs.
- **Article 50 transparency obligations did apply from 2 August 2026** — including the requirement that providers of systems generating synthetic content ensure outputs are marked in a machine-readable format and detectable as AI-generated, with a four-month grace for watermarking of existing systems ([Gibson Dunn](#)). That is a *provenance* requirement for text that C2PA does not serve, and it is live now. The Conviction record's signature and manifest satisfy it as a by-product. This is the year-one regulatory hook, and it is smaller but real.

NIST's agent interoperability profile remains due Q4 2026 and concerns attributable audit logs of agent *actions* ([CSA note](#)); it is a design input, not a mandate.

5. Unit economics, reconciled

The first draft claimed a verification charge "an order of magnitude below generation cost" at "above 70% margin," while Nova v2 §9 says verification costs roughly as much as the worker run. Both cannot be true. The reconciliation, using the design's own cost model:

TIER	GENERATION (CUSTOMER PAYS)	VERIFICATION COST TO THE SERVICE	PRICED AT	MARGIN
T1 lookup	~40k billed- equiv. tokens	Snapshot re-check (code) + grounding ~15k + cross- family verifier ~15k ≈ 30k	~\$0.25– 0.60	thin; this is the commodity rung and is priced to be nearly free
T2 brief	~360k	3 verifier calls ~120k + grounding ~90k ≈ 210k	~\$3–8	positive only with caching and small-model grounding; ~40–60%
T3 deep dive	~2M	~15 verifier calls ~600k + grounding ~450k ≈ 1M	~\$25–60	50–65%

Tokens are billed-equivalent; prices assume 2026 list rates with cache reads at ~0.1×. The honest summary: verification at T2/T3 is *not* an order of magnitude cheaper than generation — it is roughly a third to a half of it — and the margin is compliance-software margin, not API-reseller margin. The value is not in being cheap; it is that the customer already paid for generation and is buying a record they can show someone. T1 is priced as a loss-leader on-ramp. Willingness to pay at these levels is the first thing the plan tests (§7).

6. The two clocks

The company (three-year horizon, fundable)

Year 3: the default neutral verification service for regulated AI deliverables in EU high-risk deployers, US financial services, health systems and legal; a public benchmark the market quotes; per-label error-rate history nobody else has; records accepted as Article 12 evidence by at least two notified bodies and referenced in at least one Big Four AI-

assurance methodology; per-deliverable revenue with volume tiers; gross margin in the 50–65% range at T2/T3.

Year 2: Annex III obligations land (December 2027) with the record already mapped to Article 12 categories and already in use by design partners; hosted Verifier GA; 50+ paying organisations; the benchmark in its second public round with third-party submissions; a LangGraph node and at least one platform that can *require* a record as a policy (AgentCore, Foundry or Agent Engine — whichever partner §6 below lands).

Year 1: the Rail open and used by people the company does not know; Nova emitting records on every deliverable; hosted Verifier in beta with five design partners chosen for regulatory exposure; the benchmark published with the service's own results, wins and losses; Article 50 marking as the live hook.

The standard (ten-year horizon, three-year checkpoint)

The mechanism, named — the thing the first draft lacked. A reader-facing standard needs a platform sponsor who benefits from it without owning a model. The candidates are the Agentic AI Foundation's non-lab platinum members: Cloudflare (edge, no model, already the natural place to sign and serve a manifest), Bloomberg (regulated, data-provenance native), and the observability acquirers (Datadog, ClickHouse) who would rather standardise the record than lose it to a lab. The year-one target is one of these as co-sponsor of the schema inside the AAIF, with the OWASP Agent Observability Standard as the fallback home and the OpenTelemetry GenAI attributes (`gen_ai.verification.*`) as a 12–24-month SIG campaign, not a week-12 filing.

Three-year checkpoint: the schema has a second independent implementation and a foundation home, *or* the ubiquity bet is closed and the company continues as a compliance business. This is stated as a decision, not a hope.

What "everyone has to use it" would actually look like in year ten: a verification manifest travels with AI output the way DMARC now travels with bulk email — not because readers asked, but because a platform mandated it. The plan's job in years one to three is to be the reference implementation that exists when that mandate is written.

7. The first 180 days, cut to what five to seven people can do

Days 1–30: the willingness-to-pay test, before any code. Five regulated buyers Sam can actually reach (the plan should name them; the red-team is right that a bank does not sign with a six-week-old repo — the day-90 partner is someone already known). Show

them a mocked T1 and T2 record at the \$5 prices. Collect a signed LOI or a "no." If fewer than two say yes, the product is T1-only and the plan reverts to the compliance-tooling fallback from day 31.

Days 1–90: two deliverables, not seven. 1. The Rail core with the Claude Code hook, and Nova emitting a Conviction record on every deliverable. The schema ships as the README and the JSON the Rail emits, not as a standards document. 2. The benchmark harness with first public results for the four cloud grounding APIs and five open checkers on data the vendors cannot have trained on — which is the evaluation scientist's entire first quarter, scheduled as such.

Days 90–180: the MCP server; the first design partner instrumented with the design's §13 metrics and the first numbers published; the Article 12 and Article 50 mapping table published alongside the schema; the AAIF co-sponsor conversation opened with the benchmark and the live partner as the credentials; the OTel proposal drafted with a SIG champion identified, not filed cold.

Team, corrected. Runtime engineer (Rail); evaluation scientist (benchmark, error-rate stamping); **a cryptography/security engineer** — the signature and the independence attestation are the record's entire value and were missing from the first draft; a regulatory lead who has talked to a notified body, not just read Article 12; Sam running Nova on it daily and writing. DevRel comes at month six when there is something to evangelise. Standards engineer comes with the co-sponsor. **Budget line the first draft omitted:** cross-family verification at T2 volume for five partners plus a nine-system benchmark is a five-figure monthly API bill before revenue.

8. Moat, graded honestly

SOURCE	GRADE	WHY
Per-label error-rate history and the quoted benchmark	Medium, slow, real	Calibration data accumulates only with volume; nobody else is collecting it per label.
Auditor and notified-body acceptance of the record	Strong if won, time-boxed	The window is the Dec 2027 enforcement ramp; it is an argument to be won (open question 3), not a fact.
The schema, if it gets a foundation home and a second implementation	Strong if adopted, zero if not	The ten-year bet; never let the company depend on it.
Neutrality	Prerequisite, not moat	Copyable as a routing policy; matters only because it lets the three above exist.
The Rail's invariants	None, by design	A distribution vehicle.

What the moat is *not*: the detector, the orchestrator, the UI, the models.

9. What would kill it, including what the first draft missed

- **Adoption floor for content-trust standards.** DKIM/DMARC and C2PA say a decade plus a mandate. *Tripwire*: no platform co-sponsor by month 18. *Response*: the standard bet closes at the three-year checkpoint; the company continues.

- **The Big Four and ISO 42001 own "independent verification."** *Tripwire:* a notified body declines to accept a machine-generated record as evidence without a human attestation. *Response:* already designed in — the record is evidence for the attestation; partner with the auditors rather than compete.
 - **Label fatigue.** *Tripwire:* design-partner readers stop looking at the three states. *Response:* the record's value must hold for machines and auditors even when readers ignore it; price to the compliance buyer, not the reader.
 - **A lab ships reader-facing labels on its own schema.** *Tripwire:* a lab-defined label vocabulary in a major assistant before the open schema has a second implementation. *Response:* superset plus translator; the cross-family record still has no lab equivalent.
 - **An observability vendor ships a signed deliverable-level record first.** *Tripwire:* such a SKU from Braintrust or Arize. *Response:* they are not cross-family-neutral and their acquirers are not either; compete on the benchmark and offer them the schema.
 - **Willingness to pay below cost at T2.** *Tripwire:* the day-30 test. *Response:* T1-only on-ramp plus compliance tooling.
 - **Neutrality compromised by a distribution deal.** *Tripwire:* any term that restricts verifier family choice. *Response:* walk.
 - **The one fatal risk:** the schema never gets a second implementation *and* regulated buyers will not pay per deliverable. Both tripwires fire → stop.
-

10. What I would not do

Raise on "the orchestration platform." Train a model. Build a console before the record has a second implementer. Price per seat or per trace. Keep the schema proprietary. Promise ubiquity on a three-year horizon — promise the reference implementation and the compliance business, and keep the ten-year bet alive in public. Call the layer "Nova"; Nova is Sam's controller and the best demo of it, and the layer needs a name an auditor, a regulator and a competitor can all say.

11. Open questions

1. Willingness to pay per verified deliverable at the \$5 prices — unmeasured; days 1–30.
2. Whether notified bodies will accept a machine record as Article 12 evidence — an argument to be won by December 2027.

3. Whether the AAIF will take a non-lab-sponsored schema, or whether OWASP AOS is the realistic home.
 4. Whether a platform will ever *mandate* a verification manifest on AI output the way Google and Yahoo mandated DMARC — the only known path to "everyone," and entirely outside the company's control.
 5. How cross-family independence generalises to long-form deliverables (~2 effective votes is measured for classification judges, not reports).
 6. Whether labs will accept a third-party verdict inside their surface, or whether the record only ever travels around them.
 7. The ARR of every eval/observability incumbent is undisclosed; the funding signals are strong, the revenue reality unknown.
-

Sources

Regulatory (fetched 22 Aug 2026): [Gibson Dunn — Omnibus agreement and dates](#) · [CSA — deferred, not cancelled](#) · [CSA — NIST agent standards initiative](#). Trust layer: [Anthropic Citations](#) · [Google Check Grounding](#) · [Azure Groundedness](#) · [AWS Automated Reasoning checks](#) · [Cleanlab TLM](#) · [Patronus Series B](#) · [Parallel Basis](#) · [Braintrust Series B](#) · [ClickHouse–Langfuse](#) · [Claude Code verification guidance](#) · [OTel GenAI status](#) · [Agentic AI Foundation](#) · [OWASP AOS](#) · [C2PA / OpenAI provenance](#). Buyer surveys (vendor-commissioned unless noted): [Gartner](#) · [Forrester for Boomi](#) · [Deloitte](#) · [Teradata](#) · [McKinsey](#) · [KPMG](#). HCI: [IBM phrase-level highlighting](#) · [citation-trust summary](#) · [Tow Center](#). Orchestration landscape and design evidence: the three prior reports.

The Edge Map and the Frontier Watch

Where a small team can be ahead and stay ahead: ten candidate edges graded by who holds them and whether they compound or decay, plus the self-analysing loop — weekly scan, tripwires, monthly re-read, quarterly re-assessment — that keeps the map current.

1. The one-sentence answer

Your edge is not a feature; it is **four accumulations that nobody is collecting and that a lab cannot ship in a release**: a continuously published per-label error-rate series on real deliverables, a neutral re-run benchmark of the commercial grounding APIs with version history, a source-independence graph that tells copies of a wire story apart from corroboration, and the claim-level schema that signed verification records are validated against. Everything else in the design — conflict surfacing, credibility chips, false-success detection, brief gates, text marking — is a feature a lab or an observability vendor can close in one to two releases, and should be shipped as product, not bet on as moat.

The shape of the edge is the shape of an audit firm's edge, not a software company's: the longer you do it honestly and in public, the harder you are to replace.

2. The edge map

Every candidate from the design, graded on three things: who holds it today, whether it gets stronger with use or can be closed in a release, and how long an incumbent would need. "Unoccupied" means no evidence found in a dedicated search, not proof of absence.

#	EDGE	WHO HOLDS IT (AUG 2026)	COMPOUNDS OR DECAYS	TIME FOR AN INCUMBENT TO CLOSE	VERDICT
1	Per-label calibration series – the observed false-positive / false-negative rate of each verification label (<code>entailed</code> , <code>corroborated</code> , ...) on real deliverables, published continuously	Nobody. Vendors publish one-off benchmark accuracies; the AAR position paper proposes the metrics with no live data	Compounds . Needs a labelled stream, adjudicated truth over time, and willingness to publish your own misses – labs will not publish standing error rates on their own checkers	18+ months, and only if they choose to	Bet on it . Slowest to stand up, hardest to copy
2	Neutral head-to-head of the commercial grounding APIs (Google Check Grounding, Azure Groundedness, Bedrock, Anthropic Citations, Cleanlab, Vectara, Galileo, Patronus, open checkers) on identical data, open harness, re-run on every version	Nobody for the paid APIs. LM-AggreFact covers open models and 2024-vintage LLMs only; an industry note says outright that "without a public head-to-head ... procurement stalls" (BestAI Web)	Compounds – trust and drift history. A grant could make LLM-AggreFact add the APIs in a cycle, closing "first" but not "the neutral operator with history"	3–6 months to exist; years to replicate the history	Bet on it . Also the cheapest credibility you can buy
3	Source-independence graph – telling two copies of one press release from two independent sources, so <code>corroborated</code> means something	Nobody ships it. Best practice is manual author cross-referencing (BuzzStream , 4M citations); NewsGuard and Ground News rate outlets but expose no "these are one story" signal to agents	Compounds . The cheap 60% (embedding similarity between passages) decays in one release; the curated wire/ownership graph and fingerprint index compound	Crude: one release. Curated: 12+ months	Bet on it , and ship the crude version first so the label exists
4	The claim-level record schema – signed, portable, validated-against	Partially, at the wrong layer: Sigstore-based signing of agent <i>actions</i> is moving fast (agent-sign , Aeon , Red Hat , Microsoft toolkit , RATS-based evidence packages). Nobody signs <i>claims with verification labels</i>	Compounds if adopted; zero if not . Crypto and transport are commodity; the schema being the one others validate against is the durable part	Emitting a signed manifest: one release. Schema adoption: 12+ months	Bet on it – with the ten-year caveat from the backwards plan
5	Verifier-independence conformance spec (family ≠ worker; pointers not prose)	Nobody as a spec; folk practice everywhere (Play Favorites quantifies the bias; AI Verify, OWASP AISVS, MLCommons do not codify it)	Slow compounding if cited; inert otherwise	Labs will not close it – it would disqualify their own same-family self-verification. Risk is irrelevance, not competition	Write it – cheap, and it is the argument for the whole category
6	Conflict-as-first-class (<code>conflicted</code> label with both sides)	Partially: CONFLICTS benchmark , SIGIR/WWW 2026 papers, Gemini's per-statement double-check UI	Decays – actively researched by Google- and Amazon-adjacent teams	1–2 releases	Ship it as a feature; the durable slice is conflict between <i>independent</i> sources (needs #3)
7	Credibility-weighted citation label	NewsGuard holds the rating asset and licenses it; engines use opaque signals	Decays – anyone can license the asset	1 release	Ship it ; the only compounding part is which sources' claims later fail your verification (a by-product of #1)
8	False-success detector (environment state, not LLM judge)	Unoccupied as env-state; Sentriai markets the problem with behavioural classifiers; the paper gives the recipe	Method decays (TF-IDF over state diffs is trivially copied); the corpus of (claimed-success, actual-state) pairs compounds	1–2 quarters for Arize / LangSmith / Braintrust	Ship it inside the Rail; keep the pairs

#	EDGE	WHO HOLDS IT (AUG 2026)	COMPOUNDS OR DECAYS	TIME FOR AN INCUMBENT TO CLOSE	VERDICT
9	Pre-send brief quality gate	Partially: Kiro's clarifying-question loop; every harness will copy	Decays	1 release per harness	Feature , not edge — unless spec-quality scores are tied to downstream verification outcomes (#1 again)
10	Machine-readable marking of AI-generated text (EU AI Act Art. 50(2))	Occupied by the labs. Anthropic shipped SynthID-Text on all models after 2 Aug 2026; Google's SynthID network now includes OpenAI, Apple, NVIDIA; the Code of Practice was finalised 8 Jul 2026 with deadlines 2 Dec 2026 (marking) and 2 Feb 2027 (detection interoperability) (Paul Weiss ; Anthropic)	Decays for a third party	Already closed	Do not compete on watermarking. The open slot is the Code's <i>second</i> technique — signed, tamper-evident metadata on file-based deliverables saying which parts were generated and which verified — which is #4 with a regulatory deadline attached

What this changes from the backwards plan. The plan named the verification record as the universal piece. The edge map narrows it: the record is the *vehicle*; the *edge* is the three datasets behind it (#1, #2, #3) and the schema (#4). The company's real asset in year three is not software, it is a calibration series, a benchmark history and an independence graph that took three years of honest publication to build. Reposition the first 180 days accordingly: the benchmark harness (#2) moves to the front of the queue because it is the fastest credibility and the cheapest to start; the calibration series (#1) starts the day the first design partner's deliverables flow, because it cannot be back-filled.

3. Where you will be overtaken in six months, and where you will not

Fast fronts — expect incumbents here by Q1 2027. Text watermarking and detection APIs (forced by the Code deadlines); source-disagreement chips in deep-research products; false-success detection inside observability suites; spec/brief gates inside agent IDEs; signed agent-action provenance on Sigstore. On each of these, be a user or a fast follower; none is worth a month of the team's time beyond shipping the minimum.

Slow fronts — accumulation beats speed. The calibration series, the re-run benchmark with version history, the independence graph. These are where the team's time goes, and they are exactly the things a weekly watch cannot lose for you — only neglect can.

4. The Frontier Watch — a self-analysing loop

The purpose is not to read news. It is to detect, early, the two events that change the plan: an incumbent closing an edge you are betting on, and an edge you dismissed becoming open. The loop has three cadences, each with a defined output and a defined decision. It uses the same method that produced these documents — parallel sourced research, adversarial review, citation check — so the quality does not depend on who is running it.

4.1 Weekly scan (automated; set up as a scheduled task alongside this document)

Every Monday, a fresh session runs the following, with no memory of previous runs other than this document's edge table, which it receives as its baseline.

1. **Search each of the ten fronts** for the past seven days: vendor release notes and blogs (Anthropic, OpenAI, Google, Microsoft, AWS, Braintrust, Arize, LangSmith, Galileo, Vectara, Cleanlab, NewsGuard, Patronus), arXiv (cs.CL, cs.AI, cs.SE) for claim verification / citation faithfulness / LLM-as-judge bias / agent evaluation, standards venues (OpenTelemetry GenAI SIG, OWASP AOS/AISVS, Agentic AI Foundation, C2PA, NIST agent initiative, AI Verify), and EU AI Act implementation news.
2. **For each front, classify the movement:** `none` · `feature shipped by incumbent` · `benchmark or dataset published` · `standard proposed or merged` · `regulatory date moved` · `funding or acquisition in the category` .
3. **Re-grade the edge table:** does any "bet on it" edge move from unoccupied to partially occupied? Does any "ship it" feature become table stakes? Does any dismissed edge open?
4. **Fire tripwires** (below) and emit a short report: what moved, which rows changed grade, the recommended action, and the three most important links. Push notification only if a tripwire fired; otherwise the report is filed.

4.2 Tripwires — the events that change the plan

TRIPWIRE	WHY I MATTE
A lab or cloud publishes a standing error rate per label or per grounding score on production traffic	#1 is being occup
A neutral benchmark of the commercial grounding APIs appears (LLM-AggreFact adds them; a university or MLCommons runs one)	#2 "f is cl
Any AI citation product ships a syndication / independence signal	#3 cr versi close
A signed claim-level record appears from a lab, an observability vendor, or a Sigstore project	#4 conte
OTel GenAI or OWASP AOS merges trust/verification attributes from someone else	Schem home
The Code of Practice's Feb 2027 detection-interoperability mechanism specifies a document-metadata format	#10 secon techn slot defin
A lab publishes cross-family verification as a product surface (not advice)	The categ main argum is be made lab
Funding round or acquisition in claim verification above \$50M	Categ is be price

4.3 Monthly re-read (one human hour)

Read the four weekly reports together. Ask the three questions the weekly scan cannot: has the *pace* of any front changed; is the team's time still going to the slow fronts; has anything shipped that should become a feature of the Rail this month. Update the edge table in this document — it is the baseline for next month's scans.

4.4 Quarterly deep re-assessment (the full method, ~one day of compute)

Re-run the process that produced these documents, against the current edge table and the backwards plan: parallel research passes per front, an adversarial review whose explicit

job is to refute the current edge grades, and a citation audit. Output: a dated revision of this document and of the plan's kill list. The adversarial reviewer is briefed to assume every "compounds" grade is wrong and to find the release that closes it.

4.5 What the loop measures about itself

A watch that never fires is either early or blind. Track, per quarter: tripwires fired; tripwires that in hindsight should have fired (found in the quarterly re-assessment); and the lead time between an incumbent's move and the watch's detection. If the hindsight count exceeds the fired count two quarters running, the fronts or the sources are wrong and the scan is redesigned.

5. What I would not do

Spend the team on any fast front beyond the minimum. Compete on watermarking. Wait for volume before starting the calibration series — it cannot be back-filled. Let the benchmark be funded or hosted by any vendor in the table — neutrality is the point. Treat the weekly scan as strategy; it is an alarm, and the monthly and quarterly passes are where decisions are made.

6. Open questions

1. Whether a dedicated literature pass under "evidence redundancy," "source clustering," and "churnalism detection" turns up prior work on independence labelling that this search missed — worth a day before committing to #3.
2. NewsGuard's API/licensing terms for agent developers were not confirmed from a primary source.
3. LLM-AggreFact's maintenance status — its visible commercial entries are 2024 models — determines whether #2 is contributed to or built beside.
4. Whether the calibration series (#1) can be published at design-partner volume without leaking partner data; the methodology needs a privacy design before the first publication.
5. Whether the Feb 2027 interoperability mechanism will specify a document-metadata format at all, or only watermark detection.

Nova Against the 2026 App Landscape

Every major AI assistant, coding agent and agent platform as of August 2026, graded for whether Nova can own the loop, host the work, be called as a tool, or only deliver files — plus the ladder from where the protocols are today to full compatibility.

1. The verdict

Compatible — with one hard boundary that decides the architecture.

Everything Nova v2 needs *below* the user interface exists today: strict schema output, prefix caching, batch pricing and tool calling at every major vendor; cross-family model access under one key; signing primitives for the record; a stable agent-to-agent protocol that carries arbitrary metadata. The Rail, the stateless workers, the cross-family Verifier and the signed record are buildable now at the API layer, and none of the three big vendors' terms prohibit using one model to judge another's output (they prohibit training competitors and, at Google, caching grounded search results — a real constraint on the snapshot store, handled below).

What does **not** exist anywhere is a way for a third party to see or gate a chat assistant's *final answer before the user does*. ChatGPT's Apps SDK exposes tool inputs, tool outputs and widget APIs and nothing that touches the model's response; Claude.ai, Gemini, Copilot, Perplexity, Grok and Le Chat are the same. MCP's new 2026-07-28 revision moves the other way: server-initiated *sampling* (an MCP server borrowing the host's model) is deprecated under SEP-2577 — it keeps working for at least twelve months, but new implementations are told not to adopt it — which leaves a server that wants a model to bring its own provider keys. The only documented completion-blocking hooks in the industry are Claude Code's and Codex's `Stop` -family hooks, and those are agent-harness features, not chat-app features.

So the answer to "could my system be compatible" splits cleanly:

- **Nova as the controller of its own loop** — calling models through APIs, verifying with a different family, signing the record — is compatible with every model vendor, every aggregator, every agent runtime and every coding agent that has a headless mode. That is the system as designed.
- **Nova inside someone else's chat app** — owning the user's conversation in ChatGPT, Claude.ai or Gemini and verifying what those apps say — is not possible and will not become possible by building harder. In those apps Nova can be a tool the assistant calls and a source of verified deliverables the assistant hands back; it cannot be the gate.

The path to full compatibility (§6) is therefore not one integration but a ladder: run Nova on APIs first, hook the two coding agents that expose blocking hooks, become a required check wherever output ends in a pull request, expose the Rail as an MCP server so every chat app can *call* it, expose it as an A2A agent so every

enterprise platform can *route* to it, and then lobby for the one host feature that would close the boundary.

2. The landscape in three layers

The apps sort into three layers that behave differently for Nova, and it matters which layer a given app is in.

Layer 1 — assistants you talk to (ChatGPT, Claude.ai/Cowork, Gemini app and Spark, Microsoft Copilot, Perplexity, Grok, Le Chat, Meta AI, DeepSeek, Kimi, Qwen). Closed loops. They let third parties in only through MCP tools and, in some, app widgets; they never let a third party intercept the answer. The notable August 2026 facts: ChatGPT's Atlas browser was discontinued on 9 August and its agentic work folded into the desktop app and a Chrome extension; OpenAI replaced its App Directory with a Plugin Directory on 9 July; Claude's Cowork reached the Chrome side panel on 12 August; Gemini Spark (the 24/7 personal agent, AI Ultra, US) gained custom MCP in June; Perplexity's "Computer" is itself a multi-model orchestrator with a "Check Sources" step — a Nova-shaped competitor inside a Layer-1 app.

Layer 2 — agents and platforms that do work (Claude Code and the Agent SDK, Codex, Cursor, Copilot cloud agent, Antigravity CLI, Kiro, Devin, OpenHands, Aider; and the runtimes — Anthropic Managed Agents, OpenAI Agents SDK, Google ADK 2.x and Agent Engine, Microsoft Agent Framework 1.0 and Foundry Agent Service, AWS AgentCore, LangGraph/LangSmith, CrewAI, Temporal + Pydantic AI, Inngest, Vercel AI SDK 7, Cloudflare Agents, n8n, Agentforce). Open loops. Nearly all have headless modes so Nova can call them as workers; most runtimes let you insert a deterministic node between agent output and delivery; many are multi-vendor. Two movements worth noting: OpenAI deprecated AgentKit's Agent Builder and hosted Evals (shutdown 30 November 2026) while keeping the code-first Agents SDK, and Google transitioned the consumer Gemini CLI to Antigravity CLI on 18 June.

Layer 3 — protocols and APIs (MCP 2025-11-25 and 2026-07-28; A2A v1.0, with 150+ organisations as of April 2026 and native support in Azure Foundry, Copilot Studio, Bedrock AgentCore and Google Cloud; the Anthropic, OpenAI and Google model APIs; aggregators; Sigstore, C2PA 2.4, W3C Verifiable Credentials 2.0). This is the layer Nova actually runs on, and it is the most compatible of the three.

3. Compatibility matrix — assistants and workplace apps

Verdict key: **Native** = Nova can own the loop; **Host** = Nova can be the agent host that calls workers and verifies before writing; **Tool** = the app's assistant can call Nova as an MCP tool and hand back the deliverable plus record, but the app's own prose is not gated; **Deliver** = files only; **No** = no third-party surface.

APP	THIRD-PARTY SURFACE (AUG 2026)	MCP CLIENT	PRE-DISPLAY HOOK	ATTACH DELIVERABLE + RECORD	MEMORY EXPORTABLE TO SEED NOVA	
ChatGPT	Apps SDK (MCP + widgets), Plugins, Developer mode (write actions Business/Enterprise/Edu; Pro read-only; web only; Agent mode cannot use custom apps)	Yes	No – apps see tool input/output only	Yes – tool results return file objects; widgets can upload to the file library	Chat export ZIP only; no memory import/export	
Claude.ai / Cowork / Chrome panel	Custom remote MCP (user-added, no review), Claude Apps, Skills, Projects	Yes	No	Yes – MCP resources, Apps UI, Cowork writes files	Yes, official (view/export/import; labelled experimental)	
Claude Code / Agent SDK	31-event hooks incl. blocking Stop, SubagentStop, TaskCompleted; Dynamic Workflows; plugins; MCP in/out; schema outputs	Client + server	Yes – blocking	Yes (files)	n/a	
Gemini app	Gems, Workspace; custom MCP only in Spark (AI Ultra, US, personal account, English, manual confirm on writes)	Spark only	No	Limited	Import yes (from ChatGPT/Claude); export via Takeout/prompt	
Microsoft Copilot (consumer)	Six fixed connectors, export to Office formats	No	No	Export files	No	
Microsoft 365 Copilot	Declarative agents (MCP tools), Copilot Studio (MCP client, computer-use, evaluations GA), Work IQ APIs GA 16 Jun 2026 (Chat via A2A/REST; Context and Tools via MCP)	Yes	No in chat; yes inside your own Studio agent	Yes	Tenant-level only	
Perplexity	Bring-Your-Own-Connector MCP, Projects, Computer, API platform	Yes	No	Yes (workspace files)	Not found	
Grok	20+ connectors, BYO MCP, Skills	Yes	No	Limited	Not found	
Le Chat (Mistral)	Custom MCP connectors, Memories	Yes	No	Limited	Not found	
Meta AI · DeepSeek app · Qwen app · Kimi	No official third-party surface (Qwen partner-only; Kimi Work is a desktop agent with no documented MCP)	No	No	Kimi: local files	No	
Notion	Custom Agents (Notion 3.3) can call custom MCP servers (Business/Enterprise, admin toggle); Notion MCP server	Client + server	Yes, inside your agent's run	Yes	Workspace export	
Slack	Agent Kit, Slack MCP server, Real-time Search	Client +	Yes, inside your agent	Yes (Block Kit,	Per plan	

APP	THIRD-PARTY SURFACE (AUG 2026)	MCP CLIENT	PRE-DISPLAY HOOK	ATTACH DELIVERABLE + RECORD	MEMORY EXPORTABLE TO SEED NOVA
	API, Slackbot as MCP client	server		files)	
Google Workspace	Workspace MCP server (preview), Studio skills, Apps Script	Server	No	Yes (Drive)	Takeout
Salesforce Agentforce / Agent Fabric	Native MCP client, registry, A2A, Agent Script deterministic control, AI Gateway governance	Client + server	Yes, inside your agent	Yes	Org data
ServiceNow	MCP server GA 5 May 2026 via Action Fabric	Server	n/a	Yes (records)	Org data
Atlassian Rovo · Zoom · Canva/Adobe	Curated MCP agents / A2A starting / MCP servers	Curated	No	Yes	Org data
Zapier · n8n	Zapier MCP (9,000 apps); n8n native MCP client nodes, Guardrails node, OTel	n8n: client + server	n8n: yes, you own the flow	Yes	n/a

The four most compatible places for the personal-controller use case, in order: Claude Code / Agent SDK on API keys (the only surface with a completion-blocking hook, plus workflows and file outputs); Notion Custom Agents (your agent runs, verifies, then writes); Slack Agent Kit (same, with Block Kit to render the record); Microsoft 365 via Copilot Studio and Work IQ (enterprise, but an external controller can legitimately drive Copilot). ChatGPT is the largest audience and has the cleanest file-attachment contract, but it is strictly deliver-only.

The "route everything through Nova" question. Seeding Nova with your history is possible now — Claude exports memory officially, Gemini imports chats and memory, ChatGPT gives a full export ZIP — but every export is free text with no machine-readable schema, so Nova needs its own normaliser. Continuously driving a subscription app from an external controller is mostly blocked by terms: Anthropic explicitly bans using Free/Pro/Max OAuth tokens in any other harness (API keys are the sanctioned path); OpenAI bans bypassing rate limits and safety measures. The compatible shape is Nova on APIs, with the consumer apps as delivery surfaces it reaches through MCP.

4. Compatibility matrix — coding agents and agent platforms

PLATFORM	NOVA AS ORCHESTRATOR (CALLS IT AS A WORKER)	VERIFICATION STEP BEFORE DELIVERY	MULTI-VENDOR MODELS
Claude Code / Agent SDK	Yes (Claude - p , SDK query() , workflows with JSON schema)	Yes – blocking hooks	No (CL via Anthro Bedrock Vertex Foundr
Codex CLI / cloud	Yes (codex exec , SDK)	Yes – blocking Stop , PostToolUse	No
Cursor Cloud Agents	Yes (REST v1, model picker, artifacts endpoint)	No – stop hook is non-blocking; gate at the PR	Yes
GitHub Copilot cloud agent	Yes (agent-tasks REST, May–Jun 2026; Claude or GPT)	No hooks; poll task state; gate at PR	Yes
Antigravity CLI (ex-Gemini CLI)	Likely (headless)	Hooks exist; blocking semantics unverified	No
AWS Kiro	Yes (headless)	Weak – Agent Stop non-blocking	No
Devin (API v3)	Yes (sessions + webhooks; "Fusion" multi-model)	No in-session; gate at PR	Yes
OpenHands SDK (MIT)	Yes – and the loop is open source, so the Rail can sit inside it	Yes	Yes (Litel
Aider	Yes (CLI)	No; verify the diff	Yes
Replit Agent 4	No public API	Built-in human approve only	–
OpenAI Agents SDK	Yes	Yes (output guardrails run after the agent, tripwire exceptions)	Yes (Litel
Anthropic Managed Agents	Yes	Yes (hooks, approvals)	No
Google ADK 2.x / Agent Engine	Yes	Yes (graph node, after-agent callback)	Yes (Litel wrappe
Microsoft Agent Framework 1.0 / Foundry / Copilot Studio	Yes	Yes (middleware, approval actions)	Yes (C Claude Mistra DeepSe
AWS Bedrock AgentCore	Yes	Yes – Policy/Gateway interceptors and custom Lambda evaluators,	Yes (Gatew infer target Harnes via LiteLL

PLATFORM	NOVA AS ORCHESTRATOR (CALLS IT AS A WORKER)	VERIFICATION STEP BEFORE DELIVERY	MULTI-VENDOR MODELS
		outside the model	
LangGraph 1.x / LangSmith	Yes	Yes (node + interrupt + middleware; Guard policies Aug 2026)	Yes
Temporal + Pydantic AI	Yes – the Rail is a workflow; workers are activities	Yes (verification activity)	Yes
CrewAI AMP · Inngest · Vercel AI SDK 7 · Cloudflare Agents · n8n	Yes	Yes (task guardrail / step / workflow step / Guardrails node)	Yes
Dify · Flowise	Partial	HITL node	Yes
Salesforce Agentforce / Agent Fabric	Yes (A2A/MCP registration)	Yes (Agent Script step)	Yes
GitHub Checks · GitLab status checks	–	Yes – a required check	–
Desktop/computer-use agents (Cowork, ChatGPT agent, Spark, Windows Agent Runtime, Manus)	OpenAI CUA and Copilot Studio CUA only	Human approval only; no programmatic hook	Mixed

5. What blocks full compatibility, precisely

- No pre-display interception in any chat app, and MCP is moving away from it.**
MCP's flow is request → tool result → "client processes result"; there is no post-response notification in 2025-11-25 or 2026-07-28, and sampling is deprecated in the new revision. *Consequence:* the Rail must hold its own model keys and be its own controller; in chat apps it is a tool.
- Google's grounding terms forbid caching, analysing or training on Grounded Results** (terms updated 28 April 2026; limited storage for display and chat history is permitted; grounding prompts and outputs retained 30 days). *Consequence:* the snapshot store must be fed by Nova's own fetcher or URL-context, never by Gemini Search Grounding output.
- Anthropic Citations and structured outputs are mutually exclusive** (a 400 error).
Consequence: the grounding pass on Claude is two calls — citations first, schema second — or runs on OpenAI or Google, where annotations and schema coexist.
- No single hyperscaler hosts all three closed families.** Bedrock has Anthropic + OpenAI (GPT-5.x) but not Gemini; Foundry has OpenAI + Anthropic; Vertex has Google + Anthropic + open weights. Cloudflare AI Gateway and OpenRouter give one key across all three at ~5% on credits, with zero-data-retention routing for OpenAI and Anthropic only. *Consequence:* cross-family verification is practical; verifier *identity* through a gateway is only as trustworthy as the gateway's response headers, so the record must pin provider, model id and response id.

5. **Schema enforcement is uneven at the small tier.** DeepSeek has JSON mode but no JSON-schema; many small open models emit invalid JSON; Gemini's Interactions API (GA June 2026, now recommended) has no batch and only implicit caching. *Consequence:* the Rail validates every worker return regardless of vendor; batch verification on Google goes through the legacy `generateContent` path.
 6. **Blocking hooks exist only in Claude Code and Codex.** Cursor's `stop`, Kiro's Agent Stop and (unverified) Amp's `agent.end` are non-blocking; Copilot cloud agent, Devin and Replit have no in-session hook. *Consequence:* for every agent whose output ends in a pull request, the gate is a required check, not a hook.
 7. **Subscription-token harnesses are banned.** Nova on Claude must run on API keys; running Nova "on top of" a Pro/Max login is a terms violation.
 8. **Memory has no schema.** Every export is prose.
-

6. How to get there — the compatibility ladder

Ordered so that each rung works on its own and widens reach. Nothing on rungs 0–4 needs a vendor to change anything.

Rung 0 — Run Nova as its own controller on vendor APIs. Keys from at least two families (three for a true escalation family, or Cloudflare AI Gateway / OpenRouter for one key across all three). Strict-schema briefs at Anthropic, OpenAI, Google, Mistral, xAI; validate and retry everywhere. Prefix-cache the fixed brief sections above the largest vendor minimum (4,096 tokens for Haiku 4.5 and Gemini 3.x implicit caching). Evidence snapshots from Nova's own fetcher. Grounding on Rail-supplied documents, not vendor web search. Sign the record as an in-toto statement in a DSSE envelope via Sigstore, keyless; optionally mirror its hash into a C2PA 2.4 assertion embedded in the PDF, DOCX or HTML deliverable so the file is self-describing. Seed Nova's history from the Claude memory export and the ChatGPT data export through a normaliser. *This rung is the system as designed; it is buildable today.*

Rung 1 — Hook the two coding agents that can be gated, and become a required check everywhere else. Ship a Claude Code plugin whose `SubagentStop` and `TaskCompleted` hooks run the Rail's evidence re-check and refuse completion on an unmet load-bearing criterion, with a saved Dynamic Workflow that calls the Verifier via MCP; mirror it on Codex `stop` hooks. Build the "Verified by Nova" GitHub App: it posts a check run with annotations and a Sigstore attestation, and repository rulesets make it required — this covers Copilot cloud agent, Cursor, Devin, Codex cloud and GitHub's own Agentic Workflows with no cooperation from any of them. Mirror on GitLab external status checks. *Reach: every coding agent in the table.*

Rung 2 — Expose the Rail as an MCP server. One server, three tools (`verify_deliverable`, `research_with_record`, `extract_with_record`), returning `structuredContent` plus a `resource_link` to the deliverable and the signed record, with two format-faithful examples in the tool description. That single artifact reaches ChatGPT (Developer mode and, after review, the Plugin Directory — return file objects), Claude.ai and Cowork (custom connector, no review), Gemini Spark, Perplexity, Grok, Le Chat, Cursor, VS Code and Copilot Studio. Nova is a tool there, not a gate — say so in the record's `scope` field so nobody mistakes a verified *attachment* for a verified *conversation*. *Reach: every Layer-1 app with an MCP client.*

Rung 3 — Become a host where hosting is allowed. Notion Custom Agents and Slack Agent Kit let your agent run, verify and *then* write; Microsoft 365 via Copilot Studio and

Work IQ lets an external controller drive Copilot inside a tenant; n8n's Guardrails node and HTTP node make a "Nova Verify" step trivial. On these, Nova owns the loop and the record gates the write. *Reach: the workplace.*

Rung 4 — Expose the Rail as an A2A agent. A2A v1.0 artifacts carry arbitrary `metadata` (any JSON), so the full verification record rides natively; signed Agent Cards give verifier identity. Deploy on Bedrock AgentCore Runtime and Azure Foundry, both of which speak A2A, and register in Agentforce's Agent Fabric. For the substrate, Temporal + Pydantic AI is the architectural twin of the Rail (workers as activities, a verification activity before delivery, multi-provider models, Logfire/OTel); LangGraph is the alternative. AgentCore Policy is where the verification becomes enforceable *outside* any agent framework. *Reach: every enterprise platform.*

Rung 5 — The vendor features that would close the boundary, and what to ask for.

A blocking `stop` with a JSON decision in Cursor, Kiro and Amp (Claude Code and Codex already have it). A pre-PR "verification webhook" that holds a cloud agent's PR in draft until an external verdict (Copilot, Devin). A named in-toto predicate for agent-authored change surfaced in PR UI and requirable by ruleset (GitHub). Native OpenTelemetry export with trace-id passthrough in the Claude Agent SDK, Codex and Cursor. An MCP post-response notification — the one feature that would let a third party gate a chat answer, and the one the protocol is currently moving away from; the realistic ask is a host-side "verified attachment" rendering convention rather than a gate. A programmatic approval API on computer-use agents (Copilot Studio's CUA partly has one).

7. What this changes in the design

Three adjustments to Nova v2, all small.

- The snapshot store's fetcher is Nova's own, and Gemini Search Grounding output is never evidence (terms). URL-context and Rail-supplied documents only.
- The grounding pass is vendor-aware: two calls on Claude, one on OpenAI or Google.
- The record gains two fields: `scope` (`loop` when Nova owned the run; `attachment` when Nova was a tool inside someone else's app) and `verifier_route` (provider, gateway, model id, response id) so a record produced through an aggregator is still attributable.

And one confirmation: the design's insistence that Nova hold its own keys and be its own controller — which the first report treated as a choice — is the direction the protocols are moving. MCP 2026-07-28 did not remove sampling overnight, but it told new implementations not to rely on it.

8. Gaps

Antigravity CLI hook semantics and Jules API status are unverified from primary docs. Two dated claims were not re-checked in the final audit and rest on the research pass alone: Bedrock hosting proprietary GPT-5.x, and Cowork's Chrome side panel on 12 August 2026. The AgentKit deprecation is cited to a secondary source. Amp's `agent.end` blocking behaviour, Copilot cloud-agent completion webhooks and Bugbot merge-blocking are undocumented. Meta AI, Mistral Le Chat's 2026 changes, Kimi

Work, Manus and Genspark rest on secondary sources. OpenRouter's zero-retention specifics and the Cohere structured-output status were not re-verified. Gemini consumer-app MCP outside Spark is inferred from absence. The MCP tasks extension's adoption by hosts is undocumented. c2pa-rs writer support for OOXML and HTML should be confirmed before committing to embedded manifests.

Sources

Protocols and APIs: [MCP 2026-07-28](#) · [MCP 2025-11-25 changelog](#) · [MCP tools spec](#) · [A2A v1.0 \(Linux Foundation\)](#) · [A2A spec](#) · [Anthropic structured outputs](#) · [Anthropic caching](#) · [Anthropic citations](#) · [Anthropic commercial terms](#) · [OpenAI structured outputs](#) · [OpenAI caching](#) · [OpenAI terms](#) · [Gemini Interactions API](#) · [Gemini grounding](#) · [Gemini terms](#) · [Bedrock OpenAI models](#) · [Cloudflare AI Gateway unified billing](#) · [Sigstore attestations](#) · [C2PA 2.4](#) · [W3C VC 2.0](#). Assistants: [OpenAI Apps SDK reference](#) · [ChatGPT Developer mode](#) · [App submission guidelines](#) · [Evolving Atlas](#) · [ChatGPT release notes](#) · [Claude connectors](#) · [Claude memory export](#) · [Cowork in Chrome](#) · [Anthropic third-party harness ban \(The Register\)](#) · [Gemini Spark MCP](#) · [Gemini Spark updates](#) · [Gemini import \(9to5Google\)](#) · [Work IQ APIs](#) · [M365 publish](#) · [Notion custom agents MCP](#) · [Slack MCP](#) · [Workspace at Next 2026](#) · [Agentforce MCP](#) · [ServiceNow MCP](#) · [Rovo third-party agents](#). Agents and platforms: [Claude Code hooks](#) · [Dynamic Workflows](#) · [Agent SDK hooks](#) · [Codex hooks](#) · [Cursor hooks](#) · [Cursor Cloud Agents API](#) · [Copilot agent tasks API](#) · [GitHub Agentic Workflows](#) · [Antigravity CLI transition](#) · [Kiro hooks](#) · [Devin release notes](#) · [OpenHands SDK](#) · [OpenAI Agents SDK guardrails](#) · [AgentKit deprecation](#) · [Managed Agents](#) · [ADK 2.0](#) · [Agent Framework at Build 2026](#) · [Foundry Agent Service GA](#) · [Copilot Studio what's new](#) · [AgentCore release notes](#) · [LangSmith changelog](#) · [Pydantic AI + Temporal](#) · [n8n Guardrails](#) · [GitHub check runs](#) · [GitLab status checks](#) · [DevOps.com on agent change controls](#).

Spec Enforcement Against a Running System

Given a spec that passed a deterministic gate, how do you mechanically prove the running system satisfies it? Findings across contract testing, browser automation, runtime verification and evidence recording; a recommended conformance harness; what it cannot verify; and the effort to build it.

1. The headline findings

First: the thing you want mostly does not exist as a product, but every piece of it exists as a tool. There is no off-the-shelf system that takes a spec and mechanically proves a running deployment satisfies it. Specifically, no tool anywhere diffs "controls declared in a spec" against "controls found on the live page" — broken-*link* checking is a solved commodity; broken-*button* checking is not a product category. But the primitives — real-browser automation with protocol-level network capture, schema-driven API verification against a live URL, request-ID correlation into server logs, signed evidence bundles — are all mature, mostly free, and composable. The custom part is small and it is exactly your spec format's shape: your gate already forces every control to declare its endpoint, which means **the spec is the manifest** the missing tool needs.

Second: the industry's flagship "contract testing" is explicitly not this. Pact's own documentation recommends *against* verifying a deployed provider: "Verifying pacts against an already deployed provider will mean you don't get the benefits that contract testing was intended to provide" — it is a CI-time technique requiring controllable provider states ([Pact docs](#)). Spring Cloud Contract is the same shape. Anyone selling "contract testing" as live-system proof is reselling a build-time tool.

Third: the category is littered with recent corpses, which tells you what to build on. Dredd (the classic spec-vs-live-server validator) was archived November 2024; Optic (spec-vs-traffic drift) was archived January 2026 after the Atlassian acquisition; Keptn (SLO quality gates) was archived by the CNCF in September 2025 — all verified against the repos and the CNCF page. The

survivors are boring and old: Playwright, Hypothesis/Schemathesis, plain schedulers, Sigstore. Build on those.

2. Answers to the six sub-questions

2.1 Which criteria can be verified fully automatically, and where is the line

The crispest available frame is Bach and Bolton's checking-versus-testing distinction — *checking* is "the mechanistic process of verifying propositions about the product" by algorithmic decision rules ([Testing and Checking Refined](#)) — combined with SRE practice's operational version: only what is expressible as a measurable indicator over observable signals can be gated ([Google SRE book, ch. 4](#)).

A criterion is machine-checkable against a live system if and only if it is a proposition over observable network, trace, or metric behaviour with an algorithmic decision rule. On the automatable side: response schemas, status codes, and error envelopes; state transitions visible through the API (create-then-get, pagination invariants, idempotency); enumerated authorisation cases (role X gets 403 on endpoint Y); timing and availability SLOs; span-level assertions on real traces; and no-5xx robustness under generated inputs (probabilistic — it falsifies, never proves). On the other side, fundamentally not automatable: aesthetic and judgement qualities (no decision rule exists); **universal absence claims** — "never leaks," "no vulnerabilities" — because finite observation cannot establish a universal (Dijkstra's dictum: testing shows presence of bugs, never absence); emergent fitness-for-purpose; long-horizon properties (durability over years); and any truth living outside the observable interface (was the email actually read; is the price commercially right).

There is an honest middle band the brief should plan for: properties automatable **only with the system's cooperation** — seeded test data, test tenants, controllable states. That is precisely why Pact refuses live providers. Your spec's `environment` and `externals` fields are where that cooperation gets declared.

2.2 What exists today — real versus marketing

CATEGORY	BEST-EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
Schema-driven API verification	Schemathesis (property-based, from OpenAPI, against any base URL)	Yes	Real – the best-evidenced tool in this entire space: peer-reviewed benchmark on 16 real services found 1.4×–4.5× more unique defects than the second-best of eight fuzzers (arXiv 2112.10328 ; author-written but peer-reviewed with a public benchmark; venue label soft-confirmed). Actively maintained, built on Hypothesis.
Stateful REST fuzzing	RESTler (Microsoft)	Yes	Real, research-grade; 28 bugs in GitLab in the ICSE 2019 paper; internal Azure claims not independently audited (repo)
Example/contract conformance	Microcks	Yes	Real – CNCF incubating May 2026; shallower than Schemathesis (examples and schemas, not generated properties)
Spec-example validation	Dredd	Yes	Dead – archived Nov 2024. Do not adopt.
Spec-vs-traffic drift	Optic	–	Dead – archived Jan

CATEGORY	BEST-EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
			2026. oasdiff is alive but spec-vs-spec only; live drift detection is an open gap.
Consumer-driven contracts	Pact, Spring Cloud Contract	No – by design	Real and mature, but CI-time; their own docs say not to point them at deployed providers
Property-based testing at the boundary	Hypothesis → Schemathesis	Yes	Real. UI-level temporal-logic testing (Quickstrom, PLDI 2022) is real research, honestly niche.
Trace-based assertions	Tracetest; plain SLO monitors	Yes	Real mechanism, churning category – Keptn archived Sep 2025; Tracetest's OSS momentum has visibly slowed
Runtime verification from formal specs	AWS's internal PObserve (validates production logs against P specs – ACM Queue); NASA's Copilot (avionics)	Yes, post hoc	Real but unavailable – no adoptable general-purpose product exists. This is the most instructive gap: the best team in the world at this built it in-house because they couldn't buy it.

CATEGORY	BEST-EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
Model checking (TLA+/P)	TLA+	No	Real – but it verifies the <i>design</i> , not the running code, and MongoDB's attempt to trace-check the running server against its TLA+ spec was abandoned as costing more than it was worth (VLD B 2020). Spec-derived <i>test generation</i> succeeded in the same study.
eBPF behavioural rules	Falco	Yes (host-level)	Real for security deny-lists; nobody credibly claims spec conformance with it
Synthetic monitoring (scheduled real-browser checks)	Checkly (Playwright-native), Datadog Synthetics	Yes	Real and battle-tested. Standalone OSS "synthetic monitoring platforms" are marketing-thin; scheduled CI running Playwright is the credible free route.
Browser automation with network evidence	Playwright	Yes	Real – protocol-level capture, HAR recording, trace archives, accessibility-tree snapshots, with honestly documented

CATEGORY	BEST - EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
			blind spots (\$2.3)

2.3 The cheapest reliable proof that every declared control reaches its declared endpoint, on the live page

Four layers, three of them free, one small custom piece:

The manifest is generated from the spec. Your gate's R5 already forces every control to carry `endpoint` or `display_only`, and every surface to carry its route. That is the manifest the missing tool needs: control (role + accessible name) → expected endpoint (method + pattern). Nothing to author twice; drift between spec and manifest is impossible because they are the same document.

The census. Playwright's accessibility-tree snapshot (`toMatchAriaSnapshot`) freezes the full set of roles, names, and states on the live page as reviewable YAML ([docs](#)). Diffing that against the spec's declared controls fails the run in either direction: a declared control missing from the page, or an undeclared control present. Known limit: it enumerates *accessible* controls — a clickable `div` with no role is invisible to it, which is itself a defect worth surfacing.

The click-proof. For each manifest row, on the production URL: a real click via `getByRole`, then block until the matching request/response is observed (`waitForResponse` with URL pattern and 2xx), with the context configured to block service workers (Playwright's own recommendation when interception matters — [network docs](#)), HAR recording on, and trace recording on. This is a mechanical click → endpoint proof with no mocking anywhere. Playwright's documented blind spots, planned around rather than discovered later: service-worker-served requests can be invisible or header-stripped in HAR (hence the block setting); `keepalive` / `sendBeacon` traffic is flaky across engines; WebSocket frames are not represented in HAR at all (the format was never standardised and has no frame entries); route handlers only see requests issued after registration.

The server-side confirmation — the step that closes tonight's failure class. A 2xx in the browser proves *something* answered; it does not prove the declared handler ran (a CDN or cache can answer). The pragmatic standard is request-ID injection: the harness stamps every request with a unique `X-Request-Id: verify-<run>-<control>` and then queries the backend's logs for it — the long-established correlation pattern ([Heroku](#)). The stronger version is minting a W3C `traceparent` header *from the harness* and

looking up the server span in the tracing backend; Datadog productised exactly this correlation for its synthetics ([docs](#)). Do not gate anything on in-page browser OpenTelemetry — its own maintainers describe it as "experimental and mostly unspecified"; header injection from the harness is the stable path.

Cost. Playwright is free. Scheduling via CI cron is free at daily granularity. Checkly's free tier covers 1,000 hosted browser-check runs a month, Starter is \$24/month (verified against their pricing page); Datadog is \$12 per thousand runs and only makes sense if already paying for Datadog, where it buys the click → server-span correlation as a product feature. The custom piece — spec → manifest → generated checks — is the only thing that has to be built.

Production safety, from documented practice (Microsoft's engineering playbook on synthetic tests against production with test users; Google SRE on probes that replay known-good requests): read-only controls are verified by real submission; mutating controls run in a test tenant with a cleanup contract, or carry a synthetic-traffic marker header the backend recognises; destructive controls without a test tenant are the one category that cannot be safely click-proven on production, and the spec's `reversible` / `rollback` fields are where that surfaces.

2.4 Verifying criteria that have no runnable check, without a model's opinion

Three findings. First, the population of such criteria should be small by construction — your gate's banned-word rule and `human: true` labelling already push everything checkable into runnable form, so what remains is genuinely judgement: appearance, tone, fitness. Second, **the smallest thing a human must look at is a diff, not a page**. Visual regression practice (Percy, Chromatic; or open-source pixel-diffing) reduces "does this still look right" to "approve these N changed regions against the last approved baseline" — the human judges deltas from a known-good state, not the whole surface each time. The same pattern generalises: for each `human: true` criterion, the record should present the *evidence pack* the spec's `evidence_required` field names (a specific screenshot, a specific response body) and nothing else. Third, where a model is used at all, the evidence from this project's earlier research is that it belongs *before* the human as a flag-raiser, never as the decider — LLM judges anchor on confident prose ($AUROC \leq 0.65$ where a trivial environment-state classifier reached 0.83–0.95) and flip under pushback; a script routes, a human decides, a model may only annotate. The auditable unit is: criterion id → evidence pack → named human → dated verdict.

2.5 The false-success numbers

Two bodies of evidence, one for machine reporters and one for human ones, pointing the same way.

When the reporter is an LLM system that did the work: self-reported completion is false in 13–79% of failures depending on model; false success is 45–48% of failures in tool-use benchmark domains and 75.8% of self-assessing trajectories in AppWorld ([Confident Closing, 2026](#)); agent-written tests average ~25 print statements per 5 assertions, and prompting for more tests doesn't change outcomes ([2602.07900](#)); adding a same-family model as checker approves its own family's output preferentially ([Panickssery, NeurIPS 2024](#)). Independent mechanical verification cut false success to ~3% of failures in the one study that measured the intervention.

When the reporter is the human who did the work: developer self-testing removes roughly 20–50% of defects while independent formal inspection removes 60–97% (Capers Jones' defect-removal tables — consultancy data, ~15 years old, the most-cited and least-auditable numbers in the field); Fagan's original IBM study had inspections finding 82% of defects (1976, replicated directionally for decades); and developers measured as higher in confirmation bias write tests that *verify* rather than falsify and ship higher defect density ([Calikli & Bener 2013](#), five industrial datasets). The green-checkmark infrastructure itself lies at scale: ~16% of Google's 4.2M tests showed flakiness (Google testing blog, 2016, verified); 4.6% of test cases flaky at Microsoft, hitting 14–52% of builds ([ICSE-SEIP 2019](#)). One honest gap: the false-pass direction — tests that assert nothing and stay green — is essentially unquantified in the public literature.

The design consequence is the one your constraints already state: the verifier must not be the builder, and the numbers above are why that is not a preference but a measured necessity.

2.6 Recording results so "done" is auditable

The evidence bundle per verification run: the Playwright trace archive (per-action screenshots, DOM snapshots, console, network — self-contained), the HAR (scrubbed — it captures cookies and headers, and the format was never standardised, so treat it as secondary evidence), the manifest snapshot (which is the spec version), the request-ID or trace-ID lookup results from the backend, the commit SHA, target URL, and timestamp. JUnit XML is not evidence — it records that assertions passed, not what was observed, and is trivially forgeable.

For the audit chain: write-once storage (object-lock) clears the practical audit bar today — SOC 2 evidence collectors gather less (timestamped screenshots and API pulls). The upgrade path exists and is shipping piece by piece: the in-toto attestation spec has a stable

`test-result/v0.1` predicate designed exactly for "verify required tests ran and passed" (verified in the spec repo), GitHub's attest action signs arbitrary predicates over artifact digests with Sigstore, and Rekor provides the append-only log. Honest status: nobody is doing signed test attestations at scale yet — the predicate exists, adoption is demos — so this is an early-adopter position, consistent with the verification-record direction of the earlier reports, not established practice.

3. Recommended approach

Build a conformance harness that compiles the approved spec into three check suites plus an evidence bundle, run by a verifier identity that is not the builder.

Reasoning follows each part.

1. **Spec → manifest → generated UI census and click-proofs** (the custom piece; nothing on the market does it). Census by accessibility snapshot; per-control click with network wait; service workers blocked; request-ID injected; server log/trace lookup closes the loop to the handler. *Why*: this is the exact failure class you hit — panels that render with dead controls — and no existing category catches it: RUM dead-click detection is statistical and needs real users; synthetic monitors check journeys someone scripted, not the declared control inventory.
2. **Spec → API conformance suite** against the deployed base URL: the spec's `actions[]` endpoints and `acceptance[]` verify commands run as-is, plus Schemathesis over the OpenAPI description if one exists, for schema conformance and no-5xx robustness beyond the enumerated criteria. *Why*: it is the mature end of the market, the one tool with a peer-reviewed benchmark, and free.
3. **Evidence bundle + verdict record** per run, in write-once storage, with the in-toto test-result attestation as the signing layer when you want "done" to be verifiable offline. Human-judgement criteria get evidence packs and diff-based review; a model may pre-annotate, a script tallies, a human signs.
4. **The verifier is a separate identity**: its own repo, its own credentials, no write access to the system under test beyond the test tenant, and the builder cannot modify the harness or the manifest between approval and verification (the manifest is the approved spec, version-pinned). *Why*: every number in §2.5.
5. **Schedule it**. A pass proves a point in time; the same suite on a daily cron (free) or Checkly free tier is what makes "still done" a standing fact rather than a memory. The dead-tool history in §1 argues for owning the scheduler-agnostic suite and treating any hosted runner as replaceable.

What was considered and rejected: Pact-style CDC (build-time by design); TLA+/model checking as the verification layer (verifies the design, not the deployment; MongoDB abandoned trace-checking as not worth it — though spec-derived *test generation* is worth revisiting later); eBPF conformance (nobody credibly does it); in-page browser OTel (experimental by its maintainers' own description); LLM-as-verifier (the numbers in §2.5 are the whole argument, and your constraint already forbids it).

4. What the recommended approach cannot verify

Stated plainly. It cannot verify business-logic *correctness* of a handler — only that the declared handler ran and returned the declared shape; a handler that runs and computes the wrong answer passes unless an acceptance criterion pins the answer. It cannot see controls invisible to the accessibility tree (handler-less clickable divs — mitigated, not solved, by treating census mismatches as defects). It cannot reliably capture `keepalive` / `sendBeacon` traffic or WebSocket payloads in HAR evidence. It cannot safely click-prove destructive controls on production without a test tenant and cleanup contract — those criteria either get a tenant or stay human-gated. It cannot verify universal absence properties ("never leaks") — only falsify them when they fail during observation. It cannot verify judgement qualities without a human, and it does not pretend to. It cannot verify states the test account cannot reach (other roles, other tenants) unless each is enumerated with its own credentials in the spec's `permissions[]`. And it proves points in time — a control can die five minutes after a green run, which is what the schedule is for, and even the schedule only bounds the window.

5. Effort estimate

Assumes one engineer who knows Playwright, and your existing spec format as input.

Verification harness v1 — roughly two to three weeks: spec → manifest compiler (small; the fields already exist), census + click-proof generator with the service-worker/HAR settings above, request-ID injection and a log-lookup adapter for your backend, evidence bundle writer, and a runner exit-code contract matching your gate's style. **Signing and write-once storage — roughly one week**, using the shipped GitHub attest action and object-lock storage; early-adopter risk is schedule, not feasibility.

Ongoing cost per spec — near zero marginal tooling, real marginal authoring: the `verify` commands your gate already requires *are* the API suite; the honest recurring cost is writing them well, which the brief correctly says is real and not free. **Runtime cost:** free on CI cron; \$0–24/month hosted if you want managed scheduling and alerting.

The number to hold the approach to, from §2.5: the one measured deployment of independent mechanical verification took false success from roughly 45% of failures to roughly 3%. That is the bar, and the reason the three weeks are worth it.

Sources

Live-page proof: [Playwright network](#) · [Trace viewer](#) · [Aria snapshots](#) · [SW HAR bug #29474](#) · [keepalive #34497](#) · [HAR 1.2 spec](#) · [abandoned W3C HAR note](#) · [Heroku request IDs](#) · [Datadog synthetics ↔ APM](#) · [OTel JS browser status](#) · [Checkly pricing](#) · [Datadog RUM dead clicks](#) · [FullStory dead clicks](#) · [MS synthetic monitoring playbook](#) · [Google SRE testing](#). Tooling: [Schemathesis paper](#) · [Schemathesis repo](#) · [RESTler](#) · [Dredd \(archived\)](#) · [Microcks CNCF incubation](#) · [Optic post-mortem](#) · [Pact provider docs](#) · [Quickstrom PLDI 2022](#) · [ShardStore SOSP 2021](#) · [AWS systems correctness / PObserve](#) · [MongoDB eXtreme Modelling](#) · [Keptn archived](#) · [Tracetest](#) · [Falco](#) · [Bach & Bolton](#) · [SRE SLOs](#). False success and evidence: [Confident Closing](#) · [agent tests 2602.07900](#) · [Panickssery](#) · [Capers Jones DRE](#) · [Fagan 1976](#) · [Calikli & Bener 2013](#) · [Google flaky tests](#) · [Microsoft flakiness ICSE-SEIP 2019](#) · [in-toto test-result predicate](#) · [GitHub actions/attest](#) · [Rekor](#).

Parts Library and Criteria Format

The minimum set of reusable functional parts, the acceptance-criteria format that keeps their checks composable when assembled into one application, the composition rules, and what remains unsolved — with real format examples and a two-part worked proposal.

1. The recommended parts list

1.1 What the census found

Fourteen enumerable sources were cross-tabulated: internal-tool platforms (Retool's ~127 components, OutSystems Forge with its download counts, Appsmith, Budibase, NocoBase, Softr, Mendix), SaaS starter kits (ShipFast, SaaS Pegasus, Laravel Jetstream, Vercel's Next.js SaaS starter, Wasp/Open SaaS, create-t3-app), and Supabase as the platform-as-parts-bin. The WordPress popular-plugins list — the largest natural experiment in what people bolt onto applications, with public active-install counts — corroborates the frequencies: Contact Form 7 at 10M+ active installs, WooCommerce 7M+, WPForms 5M+, WP Mail SMTP 4M+ (wordpress.org).

Two blocks appear essentially everywhere: **authentication** (11 of 14 sources; all seven starters ship the identical login/registration/session/2FA quartet — Jetstream's own description is "login, registration, email verification, two-factor authentication, session management" ([docs](#))) and **forms** (13 of 14). One block is an entire product category: **CRUD tables over a database with an admin surface** is what Retool, Appsmith, Budibase and NocoBase fundamentally are. The rest of the majority tier: **roles/permissions** (8), **payments/billing** (7 — every customer-facing starter, near-zero in internal-tool templates), **transactional email/notifications** (8), **file upload/storage** (7), **charts/reporting** (7), and **workflow/approvals/background jobs** (7 — with the striking datum that 22 of Budibase's 23 templates are approval flows ([templates](#))).

1.2 The recommended list

Core nine — in, because they recur across effectively every source:

1. **Auth** (login, registration, sessions, password reset, optional SSO/2FA)
2. **Forms** (data collection with validation)
3. **Records** (CRUD over a datastore, list/detail/edit, the admin panel case included)
4. **Permissions** (roles, per-role read/write/action gates)
5. **Notify** (transactional email, in-app, optional SMS/push)
6. **Files** (upload, storage, retrieval, presigned access)
7. **Reports** (charts/dashboards over the records)
8. **Flow** (approvals, background jobs, scheduled tasks)
9. **Billing** (checkout, subscriptions, webhooks) — *conditional: present in every customer-facing source, absent from internal tools; ship it as a first-class part but expect roughly half of assembled apps to exclude it*

Second shelf — in the library but not the core, because they appear in a minority of sources and each is a whole service category when done properly: teams/multi-tenancy; search (long tail as a shipped block — 2 of 14 — though universal as a bolt-on service); calendar/scheduling (note Cal.com is an entire company shipping this one part); audit log; content/SEO pages; AI chat (5 of 14 and rising).

Out, with the reasoning: device capabilities, maps, kanban, PDF generation, i18n, feature flags, impersonation — each appears in one to three sources. They are real demands (OutSystems' Ultimate PDF has 25,096 downloads ([Forge](#))) but each added part carries the full cost of criteria authoring, configuration schema and composition testing while covering a shrinking slice of apps.

1.3 Where the set stops, and what is always custom

The stopping rule the evidence supports has two triggers, and the vendors state them themselves. OutSystems names the moment to leave the blocks: when you need "your own algorithms to solve your specific use cases" ([custom-code post](#)) — its platform "promotes the graceful 'stepdown' to standard 3GL languages" at exactly that point ([evaluation guide](#)). Bubble's documented hard limits give the second trigger: 300-second workflow timeouts, 50,000-record sorted searches, 10,000 elements per page ([hard limits](#))

— when the unit of work no longer fits one block's envelope, composition stops and custom build starts.

Always business-specific, in the vendors' own words: ShipFast covers the integrations so you can "spend your time building your startup" — the startup is not in the box (shipfast.com); Pegasus "handles the foundation of your application" — foundation, not application (aaspegasus.com). The never-templated residue is consistent across all sources: **the domain data model, the proprietary algorithm (pricing, matching, scoring — the thing the business actually is), and org-specific approval semantics** — even Budibase, whose entire catalogue is approvals, ships them as thin starting points because every company's process differs.

Verdict: the parts list is the easy half. Nine core parts plus a conditional Billing cover the recurring functional surface of business applications; the census sources agree with each other to a degree that suggests this is settled industry knowledge. The set stops at proprietary logic and at envelope limits, and both boundaries are documented by the platforms themselves.

2. The criteria format

2.1 What exists, and the zero-interpretation test

Every shipping format was judged against one question: given only the artifact, can a machine produce pass/fail with zero human or model interpretation?

Gherkin fails it, and this needs saying plainly because it is the industry's default answer. A `.feature` file is not executable. Cucumber's own docs: "When Cucumber executes a Gherkin step in a scenario, it will look for a matching step definition to execute" ([step definitions](#)) — the meaning of every step lives in glue code someone writes per application. Gauge and Robot Framework (for app-specific keywords) share the disease.

The formats that pass are the ones with a *closed* vocabulary — every keyword defined by the runtime, nothing left to a glue author. Hurl is the cleanest existence proof ([asserting-response](#)):

```
GET https://example.org/api/cats
HTTP 200
[Asserts]
jsonpath "$.cats" count == 49
duration < 1000
```

Karate passes for its HTTP surface (closed keyword set, first-class retry: `* configure retry = { count: 10, interval: 2000 }` then `retry until response.id == 1` ([polling docs](#))). StepCI and Maestro pass. Playwright passes *once written* but is code — authoring is the interpretation step. Pact's contract JSON passes for replay, but its provider states are named preconditions with hand-written handlers. Schemathesis/Microcks pass but express only *shape* criteria — they cannot say "the order was actually created."

The newest and most relevant standard is Arazzo (OpenAPI Workflows, v1.1.0 published 17 May 2026 — a real spec, early adoption, runner ecosystem still thin ([spec](#); [APIScout survey](#))). Its Criterion Object is the best existing skeleton:

```
successCriteria:
  - context: $response.body
    condition: $[?count(@.pets) > 0]
    type: jsonpath
```

— *what to look at* (`context`), *the decision rule* (`condition`), *the rule's language* (`type`), plus step-level `timeout` , `retryAfter` , `retryLimit` , and namespaced outputs (`$steps.loginStep.outputs.sessionToken`). But it deliberately stops at the API surface: no UI channel, no invisible-outcome channel.

2.2 Your four-field hypothesis, corrected

You proposed trigger + observable outcome + where observed + decision rule. The formats' own required fields say that is **necessary but not sufficient** — **two fields are missing and one needs splitting**:

1. **Precondition / starting state** — every composable format forces it: Gherkin `Given` , Pact `given('I have a list of dogs')` , Arazzo `dependsOn` + `inputs` , Playwright project `dependencies` + `storageState` . Without it a criterion is only unambiguous relative to unstated world-state.
2. **Trigger** — confirmed.

3. **Observation channel + address** — your "where," made explicit: the formats force you to name the *channel* (HTTP response, DOM, span, mailbox), not just a location.
4. **Extraction query** — split out from "where" by every mechanical format (JSONPath, XPath, span attribute, header name).
5. **Decision rule from a closed matcher set** — confirmed, with "closed set" as the load-bearing property.
6. **Time budget** — *missing from your list, and every serious format converged on forcing it*: Playwright auto-retries assertions for 5 seconds by default; Maestro waits up to 7; Arazzo has `timeout / retryAfter / retryLimit`; Hurl has `--retry`. Without a time budget, "the email is sent" is undecidable — there is no moment at which failure is declared.

2.3 Invisible outcomes — your suspicion confirmed

This is the weakest area in existing work, exactly as you expected. No declarative format has first-class email, webhook, queue or database-row outcome channels. In practice these are checked by *interposing infrastructure* driven by imperative code: email goes through a capture service and is asserted via its API (Mailosaur:

`messages.get(serverId, { sentTo: ... })`), which "automatically waits for the first email to arrive that matches" ([docs](#)); self-hosted Mailpit ships "a REST API for integration testing" ([repo](#)); webhooks through an interposed receiver (`stripe listen --forward-to ...` — with Stripe's own warning that event *ordering* is not guaranteed and therefore not assertable ([docs](#)); queues and rows by polling a database or admin API (Karate's JDBC helper plus `retry until`).

The single structural exception: **Tracetest** reduces every *instrumented* internal effect to one declarative channel — OpenTelemetry spans — with selectors and polling profiles:

`selector: span[name = "POST /pokemon/import"] / attr:http.status_code = 200` ([docs](#)). The price: the app must be instrumented, and your telemetry schema becomes your criterion vocabulary. Its OSS maintenance is a watch item.

The design consequence for your library: the observation channel cannot be an afterthought of the criteria format — it must be part of the *part's own definition*. A part that sends email must ship, as part of its contract, the requirement that the assembled app expose a mail-capture endpoint in test tenancy. The part declares the channel; the assembly provides it; the criterion addresses it.

2.4 The proposed format

Fields, then two worked examples. This is a *proposal* assembled from shipping pieces (Arazzo's criterion object, Pact's named states, Tracetest's channel concept, Playwright's auto-wait defaults) — labelled as such.

Each part ships a `criteria.yaml` containing: `part` (name + version), `config_schema` (the questions this part needs answered, JSON-Schema typed), `provides[]` (named state capabilities this part can establish, with outputs — e.g. `authenticated_session` yielding `session_token`), `requires[]` (capabilities that must be provided by other parts or the assembly, including observation channels), `controls[]` (every control: role + accessible name template, endpoint, states — your existing surface record), and `criteria[]`. Every criterion carries the six fields, plus two the composition problem forces:

- `active_when` — a predicate over the part's *configuration*, so "password-reset email arrives" exists only when `config.password_reset == true`. **No shipping format has this**; tag-based test selection (Cucumber tags) is the nearest ancestor and it gates suites, not criteria.
- `id` namespaced as `<part>/<instance>/<criterion>` — collision-proof by construction, following Arazzo's `$steps.*` namespacing and Salesforce's namespace-prefix lesson ([LWC guide](#)).

Worked example one — visual outcome (Auth part, login control):

```
part: auth@2
criteria:
  - id: auth/{instance}/AC-01
    active_when: always
    precondition:
      requires: [seeded_test_user]          # capability provided by assembly's
test tenancy
    trigger:
      surface: "{routes.login}"
      action: click
      control: { role: button, name: "Sign in" }
      after: { fill: { email: "{test_user.email}", password: "
{test_user.password}" } }
    observe:
      channel: http_response
      address: "POST {api_base}/auth/session"
      extract: { status: ".", body_field: "$.session.state" }
      decide:
```

```
- status == 200
- body_field == "active"
budget: { timeout_ms: 5000, retries: 2 }
evidence: [har_entry, trace_zip_action, response_body]
```

Worked example two — invisible outcome (Auth part, password-reset email):

```
- id: auth/{instance}/AC-04
  active_when: config.password_reset == true      # criterion exists only if
the feature does
  precondition:
    requires: [seeded_test_user, mail_capture]    # the part DECLARES the
channel it needs
  trigger:
    surface: "{routes.login}"
    action: click
    control: { role: link, name: "Forgot password" }
    after: { fill: { email: "{test_user.email}" }, submit: true }
  observe:
    channel: mail_capture                        # provided by assembly:
mailpit/mailosaur API
    address: "mailbox:{test_user.email}"
    extract: { subject: "$.subject", link: "$.html.links[0].href" }
    decide:
      - subject matches "(?i)reset"
      - link startswith "{app_base}/reset/"
    budget: { timeout_ms: 30000, poll_ms: 2000 }  # email is eventual; the
budget is the criterion
    evidence: [captured_message_json]
```

Verdict: no existing format suffices alone, but nothing here requires invention — only assembly. The six-field tuple is confirmed by the shipping formats' own required syntax; the two additions (`active_when` , capability-typed preconditions with declared observation channels) are the parts that exist nowhere and that your composition problem makes mandatory.

3. The composition rules

Derived from what the shipping formats do, and from the documented seam failures of platforms that got it wrong.

Rule 1 — Shared state travels as named capabilities with typed outputs, never as ambient assumptions. The Pact pattern (a state *name* as shared vocabulary between independently owned sides — `providerStates: [{name, params}]` ([provider docs](#))) merged with the Arazzo pattern (outputs consumed as `$steps.login.outputs.sessionToken`). Auth *provides* `authenticated_session`; Billing and Records *require* it. The checker topologically orders criteria by the provides/requires graph — exactly Playwright's `setup-project/ storageState` pattern ([auth docs](#)), lifted into declaration.

Rule 2 — Namespacing is structural, not conventional. Two lessons: Salesforce built mandatory namespace prefixes because flat naming breaks composition ([LWC guide](#)); the web-components world didn't, and its global `customElements` registry plus async upgrade produces documented havoc ([Carniato](#)). Neither Playwright nor Testing Library gives any test-id namespacing guidance — that silence is a finding. So: every part instance gets an id; every control's accessible name and test-id is prefixed with it; every criterion id is `part/instance/criterion`; two instances of the same part on one page are distinct by construction.

Rule 3 — Configuration gates criteria, and the gate is declared, not inferred. Each part's `config_schema` answers your question 12 (which provider, is reset on, who holds keys — the custodian fields from your gate's R6 belong here); `active_when` predicates make each criterion's existence a function of configuration. The checker resolves configuration *first*, then materialises the active criterion set — a criterion for a disabled feature is not skipped, it *does not exist*, which keeps pass-counts honest.

Rule 4 — Ordering comes from the dependency graph plus the trace, never from timestamps. `dependsOn` at the workflow level (Arazzo), provides/requires at the state level, and — where cross-part causality must be proven (Flow picked up what Forms submitted) — span-hierarchy assertions, which is the one thing Tracetest ships that nothing else has ([span-order](#)).

Rule 5 — Each part's envelope is declared, because the documented seam failures are envelope failures. The failure catalogue from the census: naming/registry collision, shared-dependency contention (WordPress's official troubleshooting doc institutionalises deactivate-all-and-bisect for "two or more plugins trying to use the same resources" ([docs](#))), substrate mutation (Shopify moved to sandboxed theme app extensions explicitly because injected code broke themes ([shopify.dev](#))), upgrade version skew (Mendix's

10 → 11 guide: React 19 removes `findDOMNode` so "all widgets require updating" ([docs](#))), and resource-envelope overflow (Bubble's 300s/50k limits mean two individually working blocks fail when composed at scale). The corresponding rule: a part declares its version-pinned dependencies, its resource envelope, and that it never mutates shared substrate; the assembly checker verifies the declarations don't conflict *before* the browser opens.

Verdict: every composition rule above is a generalisation of something one shipping system already does — and every seam failure in the catalogue is a system that skipped one of them. The rules are not speculative; the *combination* is.

4. What is unsolved — bluntly

1. **Nobody has shipped criteria that travel with a component and re-verify it inside the assembled application.** The near-misses each miss by exactly one dimension: Storybook play functions plus portable stories are executable criteria that physically travel with the component and run "in a live browser" via the test-runner ([docs](#)) — but they validate the component *in isolation*, not the app's wiring around it. Salesforce UTAM ships declarative JSON page objects with the platform's components — but they contain locators and interaction methods with *zero assertions* ([utam.dev](#)). Angular's component harnesses: same shape, "the assertions remain the consumer's job." Arazzo is mechanically evaluable and composable — but API-only, and its runner ecosystem is admitted to be "still forming" ([APIScout](#)). Web Platform Tests attach executable criteria to *specs*, not components. No documented reason was found for the gap — no post-mortem of an attempt; the evidence reads as *nobody has tried the full combination*, not as *tried and failed*.
2. **Config-conditional criteria exist nowhere.** No shipping format can say "this criterion exists iff this configuration is set." Tag-based suite selection is the nearest thing and it is the wrong granularity. `active_when` is a proposal in this report, not a citation.
3. **The invisible-outcome channel has no standard.** Interposed infrastructure per channel (mail catcher, webhook receiver, DB poller) driven by glue, or Tracetest's

spans with instrumentation coupling. Making channels a declared part-requirement (§2.3) is this report's answer; nobody's shipped it.

4. **Test-id and accessible-name namespacing has no authority to cite.** Playwright and Testing Library are silent; the proposal here (instance-prefixing) follows Salesforce's precedent but will be your convention, not an industry one.
5. **Webhook ordering is not assertable** at the receiver — Stripe says so of its own events. Criteria over webhook-driven flows must assert *sets and eventual states*, never sequences.
6. **Two census gaps:** Bubble and Mendix marketplace catalogues (and install counts) are JS-rendered and could not be enumerated — the parts table leans on the other twelve sources; and no primary-source Gartner/Forrester low-code figure survived verification, so none appears.

If someone has already built this: they have not. The verifiably closest assemblages are Storybook's play-function ecosystem (criteria travel, isolation only), UTAM (declarative, no assertions), and Arazzo with Redocly's runner (mechanical and composable, API-only, young). The combination — behavioural parts, six-field mechanical criteria, declared channels, config gating, instance namespacing, verified in the assembled app in a real browser — is unoccupied. Given your existing gate and the census in §1, the missing pieces are the two things labelled *proposal* above, and both are small.

Sources

Parts census: [Retool components](#) · [OutSystems Forge](#) · [Appsmith templates](#) · [Budibase templates](#) · [NocoBase handbook](#) · [Softr](#) · [ShipFast](#) · [SaaS Pegasus](#) · [Jetstream](#) · [Vercel SaaS starter](#) · [Wasp](#) / [Open SaaS](#) · [create-t3-app](#) · [Supabase](#) · [WordPress popular plugins](#) · [Bubble hard limits](#) · [Bubble reliability post-mortem](#) · [Mendix 10 → 11](#) · [OutSystems evaluation guide](#) / [custom code](#) · [WordPress troubleshooting](#) · [Shopify theme app extensions](#) · [Salesforce LWC namespaces](#) · [Carniato on web components](#) · [Stripe newsroom](#) (vendor) · [TechCrunch](#), [Stripe/Lemon Squeezy](#) · [Fortune](#), [Okta/Auth0](#) · [Novu](#) · [Cal.com](#). Formats: [Arazzo v1.1.0](#) · [Redocly on Arazzo](#) · [APIScout](#) · [Cucumber step definitions](#) · [Gherkin reference](#) · [Playwright assertions](#) / [actionability](#) / [auth](#) / [locators](#) · [Karate keywords](#) / [polling](#) / [DB](#)

· [StepCI](#) · [Hurl](#) · [Maestro](#) · [Tracetest](#) / [span order](#) · [Schemathesis](#) · [Microcks](#) · [Pact](#) / [pact-js](#) / [provider states](#) · [Gauge](#) · [Robot Framework](#) · [WebDriver BiDi](#) · [Mailosaur](#) · [Mailpit](#) · [Stripe webhooks](#) · [Awaitility](#) · [Storybook play functions](#) / [test-runner](#) / [portable stories](#) · [WPT](#) · [Custom Elements Manifest](#) · [Angular harnesses](#) · [UTAM](#) · [Testing Library](#) · [Open Feature](#) · [Curtis](#), [Component Contracts](#) (proposal).

The Spec Writer's Questions

Seventy-eight questions reverse-engineered from eight research documents: universal questions for every spec, riders per part, composition questions per assembly, and the detectors that flag what is not a part — each traceable to the finding it prevents and the gate rule it feeds.

1. Universal questions — asked for every spec

Dimension 1 — Who

U-01. Who uses this, and what are they trying to get done in one sentence each?

Why: [brief] Unstated requirements are followed ~41% of the time; the user's job is the requirement most often left unstated. · *Fills:* `intent.user` · *Gate:* R1

U-02. Is there more than one kind of user (admin vs member vs anonymous)? Name each.

Why: [enf] States a test account cannot reach are unverifiable; every role must be enumerated to be checked. · *Fills:* `substance.permissions` · *Gate:* R1

Dimension 2 — Entry

U-03. What is the exact URL or entry point, and what does the very first screen show before any data exists?

Why: [gate] Empty states are one of the five dimensions speakers always omit as obvious. · *Fills:* `substance-surfaces[0]` · *Gate:* R1

U-04. How does a brand-new user get from arrival to their first successful action — every screen in between?

Why: [parts] Every starter kit ships onboarding because every app needs it and every speaker skips it. · *Fills:* `substance-surfaces` · *Gate:* R1

Dimension 3 — Surfaces

U-05. List every screen. For each: its route, its purpose in one line.

Why: [gate] Surfaces are the largest hole class; the dead-buttons failure began as an unlisted control. · *Fills:* `substance-surfaces` · *Gate:* R1

U-06. For every control on every screen: what endpoint does it call — or is it deliberately display-only?

Why: [gate] R5: a control with neither is a rejection, not a warning. There is no third option. · *Fills:* `substance-surfaces[]-controls` · *Gate:* R5

U-07. Will two copies of the same component ever appear on one page? If yes, what distinguishes their names?

Why: [parts] Flat naming breaks composition (Salesforce namespaces exist for this; web components' global registry is the counter-example). · *Fills:* `substance-surfaces[]-controls[]-label` · *Gate:* R5

Dimension 4 — Actions

U-08. For each control: what does the user see on success, on failure, while loading, and when there is nothing to show?

Why: [gate] The four states speakers never mention; [parts] Playwright/Maestro auto-wait exists because loading is universal. · *Fills:* `substance-surfaces[]-controls` · *Gate:* R1

U-09. Which actions change something outside this app (send, charge, delete, post)? For each: is it reversible, and how?

Why: [gate] R11; [nova] retries re-execute — an irreversible action without rollback doubles on retry. · *Fills:* `substance-actions` · *Gate:* R11

U-10. Which actions must a human approve before they run?

Why: [nova] Park-by-default for side effects; approval is a field, not a vibe. · *Fills:* `substance.actions[].needs_approval` · *Gate:* R11

Dimension 5 — Data

U-11. What is stored? For each thing: its fields, where it lives (exact store), and who can read and write it.

Why: [gate] R12; [enf] a criterion over a row needs the row's address to go and look. · *Fills:* `substance.data` · *Gate:* R12

U-12. What happens to the data when a record's parent is deleted — cascade, orphan, or forbid?

Why: [parts] Lifecycle is in every CRUD platform's data model and no speaker's description. · *Fills:* `substance.data[].lifecycle` · *Gate:* R1

Dimension 6 — External

U-13. List every outside service. For each: what it is for, what credential it needs, WHO supplies that credential, and WHERE they paste it.

Why: [gate] R6 — credential custody is the classic silently-assumed answer. · *Fills:* `substance.externals` · *Gate:* R6

U-14. For each external: what should the app do when that service is down — block, degrade, or queue?

Why: [enf] 'With the backend stopped...' criteria exist only if the degraded behaviour was ever decided. · *Fills:* `substance.externals[].on_outage` · *Gate:* R1

Dimension 7 — Identity

U-15. How does the system know who the user is — and how long does that hold (session length, remember-me, logout)?

Why: [parts] Auth is the most universal part (11/14 sources) and session policy is its most-skipped config. · *Fills:* `parts.auth.config` · *Gate:* R1

Dimension 8 — Permissions

U-16. For each role: name one thing it **MUST** be able to do and one thing it **MUST NOT** — per screen it touches.

Why: [enf] Authorisation matrices are machine-checkable only for enumerated role-endpoint pairs; unenumerated = unverifiable. · *Fills:* `substance.permissions` · *Gate:* R1

Dimension 9 — Errors

U-17. For the three most likely failures (server down, bad input, no permission): what exact words does the user see?

Why: [gate] Errors are a top-five hole; exact strings make the criterion mechanical ('shows "Cannot reach server" within 5s'). · *Fills:* `substance-surfaces[]-states.error` · *Gate:* R3

Dimension 10 — Empty states

U-18. Before any data exists, what does each list, table and dashboard show, and what does it tell the user to do?

Why: [gate] Empty is a state, not an accident; unspecified empty states become blank panels. · *Fills:* `substance-surfaces[]-states.empty` · *Gate:* R1

Dimension 11 — Money and limits

U-19. What are the ceilings — spend, records, file size, requests per minute, retry attempts, timeouts?

Why: [parts] Bubble's documented envelope limits (300s workflows, 50k sorted records) are where composed blocks silently break; declare yours. · *Fills:* `bounds` · *Gate:* R1

U-20. What is the build budget (money, time or tokens), and what should happen when it is hit?

Why: [gate] Budget is a required field; [nova] hard stops with partial results beat silent overruns. · *Fills:* `bounds.budget` · *Gate:* R1

Dimension 12 — Environment

U-21. Where does this run — host, process, port — and how is it started and restarted?

Why: [gate] Environment is required; [enf] a verifier needs a reachable URL, not a description. · *Fills:* `bounds.environment` · *Gate:* R1

U-22. Is there a test tenant or test account the verifier may use on the live system, and what marks its traffic as synthetic?

Why: [enf] Destructive controls cannot be click-proven on production without a test tenant + cleanup contract; the synthetic-traffic marker is the documented pattern. · *Fills:* `bounds.environment.test_tenancy` · *Gate:* R10

Dimension 13 — Rollback

U-23. If the deployment breaks, what is the named way back — and who may pull that lever?

Why: [gate] R11 at spec level; 'git revert' is an answer, 'we'll figure it out' is not. · *Fills:* `bounds.rollback` · *Gate:* R11

Dimension 14 — Acceptance

U-24. For each thing you just said, what observation proves it — a command and its output, a response body, a row, a message in a mailbox?

Why: [enf] The six-field tuple: a criterion needs precondition, trigger, channel+address, extraction, decision rule, and time budget. · *Fills:* `gate.acceptance` · *Gate:* R3

U-25. For every outcome that is NOT on screen (email, row, webhook, queued job): where exactly does a script go to look, and how long may it wait?

Why: [parts] The invisible-outcome gap is the weakest area of all existing formats; the channel must be declared or the criterion is undecidable. · *Fills:* `gate.acceptance[].observe` · *Gate:* R3

U-26. Which criteria genuinely need a human eye? (Fewer is better; each must name the exact evidence the human looks at.)

Why: [gate] human:true criteria are counted separately; [enf] the human judges diffs and named evidence packs, never whole pages. · *Fills:* `gate.acceptance[ ].human` · *Gate:* R3

Dimension 15 — Out of scope

U-27. Name at least three things a reasonable person might assume is included that you are NOT building.

Why: [gate] R9 — everything not listed is assumed in scope; an empty boundary is never true. · *Fills:* `intent.out_of_scope` · *Gate:* R9

U-28. Of the things now in scope: which are load-bearing (failure blocks release) and which are nice-to-have?

Why: [nova] Load-bearing gates escalation and retry policy; unlabelled criteria all gate equally, which means none really do. · *Fills:* `gate.acceptance[ ].load_bearing` · *Gate:* R3

2. Per-part riders — asked when the part is selected

Questions marked **[channel]** establish an observation channel the assembled app must provide so the criterion is verifiable — the invisible-outcome fix.

Auth

A-01. How do users sign in — email+password, magic link, Google/Microsoft SSO, or several? List all.

Why: [parts] Every starter ships the same quartet; the chosen subset changes every criterion. · *Gate:* R1

A-02. Is self-registration open, invite-only, or admin-created?

Why: [parts] Changes the entry flow and the seeded-test-user precondition. · *Gate:* R1

A-03. Is password reset on? If yes: what address sends the email, and what does its subject line say?

Why: [parts] active_when in action: the reset-email criterion exists only if this is true; the subject string makes it mechanical. · *Gate:* R3

A-04. How long does a session last, and what does the user see the moment it expires mid-task?

Why: [parts] Session policy is auth's most-skipped config; expiry mid-form is the classic unhandled state. · *Gate:* R1

A-05. Is 2FA required, optional, or off?

Why: [parts] Jetstream ships it as a toggle; a toggle unanswered is a criterion undefined. · *Gate:* R1

A-06. After login, where does each role land?

Why: [gate] The success observation of the login criterion is this exact redirect. · *Gate:* R3

A-07. [channel] For verification: what seeded test user exists per role, and who maintains those credentials?

Why: [enf] Every login criterion's precondition; states a test account can't reach are unverifiable. · *Gate:* R10

Records

R-01. Name each record type and its fields — every field, its type, and which are required.

Why: [parts] The domain data model is always business-specific; it cannot be defaulted, only asked. · *Gate:* R12

R-02. Who can create, see, edit and delete each record type? (Owner-only? Team? Everyone?)

Why: [enf] The enumerated authz matrix is the only checkable form of permissions. · *Gate:* R1

R-03. Is delete soft (recoverable) or hard? If hard, say so out loud.

Why: [gate] R11 — hard delete is irreversible and needs its rollback named or accepted. ·
Gate: R11

R-04. How many records before the list needs search, filters or pagination — and which of those are you asking for?

Why: [parts] Envelope limits: composed blocks break at scale thresholds nobody declared (Bubble's 50k sort cap). · *Gate:* R1

R-05. Can records be exported? In what format, and does the export include everything or a subset?

Why: [parts] Export is in every CRUD platform and almost no spoken description. ·
Gate: R1

Forms

F-01. For each form field: what makes an input invalid, and what exact message does the user see?

Why: [gate] Validation strings make error criteria mechanical; 'proper validation' is a banned-word answer. · *Gate:* R4

F-02. What happens on submit — where does the data go, and what does the user see next?

Why: [gate] R5/R12 — the endpoint and the storage location are the two mandatory addresses. · *Gate:* R5

F-03. Can a half-finished form be saved or is it lost on navigation? Say which.

Why: [parts] Draft state is a universal silent assumption in both directions. · *Gate:* R1

F-04. Can the same form be submitted twice by double-click or refresh? What prevents a duplicate?

Why: [nova] Idempotency keys exist because retries double side effects; forms are where users retry. · *Gate:* R11

Permissions

P-01. List the roles. For each pair (role, screen): allowed or not? For each pair (role, action): allowed or not?

Why: [enf] Only enumerated pairs are checkable; the matrix IS the spec. · *Gate:* R1

P-02. Who assigns roles, and can a user ever change their own?

Why: [parts] Privilege escalation by self-service is the classic silent default. · *Gate:* R1

P-03. What does a user see when they hit something they may not do — invisible, disabled, or an error page?

Why: [gate] The deny-path is a surface state; three different builds satisfy 'has permissions' unless this is pinned. · *Gate:* R3

Notify

N-01. Which channels — email, in-app, SMS, push — and for which events exactly?

Why: [parts] Novu ships all four; each event×channel pair is a separate criterion. · *Gate:* R1

N-02. What address or number do messages come FROM, and who owns that domain's sending reputation (DKIM/SPF setup)?

Why: [parts] WP Mail SMTP's 4M installs exist because unsent email is the default; custody question per R6. · *Gate:* R6

N-03. Can users turn notifications off, per channel or per event?

Why: [parts] Preference centres are in every notification service; unasked means unbuilt or uncriterioned. · *Gate:* R1

N-04. [channel] For verification: where may the checker read sent messages (mail-capture endpoint in test tenancy), and how long may delivery take?

Why: [parts] The invisible-outcome gap; the mailbox address and the time budget make 'the email arrives' decidable. · *Gate:* R3

Files

FI-01. What file types and maximum size are accepted, and what does the user see when they exceed them?

Why: [parts] Every upload service leads with these two limits; the rejection message is the criterion string. · *Gate:* R1

FI-02. Where do files physically live (bucket/store, region), and who can fetch a file's URL — is it public, signed, or authenticated?

Why: [gate] R12 plus the most common real-world leak: public buckets nobody decided on. · *Gate:* R12

FI-03. When a record is deleted, do its files go too?

Why: [parts] Lifecycle again; orphaned files are the default outcome of silence. · *Gate:* R1

Reports

RP-01. For each chart or number on the dashboard: what is the exact query behind it, in words?

Why: [enf] A handler that runs and computes the wrong answer passes unless the criterion pins the answer; pin it at spec time. · *Gate:* R3

RP-02. How fresh must the numbers be — live, minutes, or daily?

Why: [enf] The time budget field; 'live' and 'daily batch' are different architectures and different criteria. · *Gate:* R3

RP-03. What does the dashboard show when there is no data yet — and is a zero a zero or a blank?

Why: [gate] Empty states, dashboard edition; blank-vs-zero is a real decision. · *Gate:* R1

Flow

W-01. For each approval or background job: what starts it, who or what approves it, and what happens on approve and on reject?

Why: [parts] 22 of Budibase's 23 templates are this shape; org approval semantics are always business-specific and must be asked, never defaulted. · *Gate:* R1

W-02. How long may a job run or wait before it is considered stuck, and what happens then — retry, alert, or park?

Why: [nova] Every loop needs a Rail-side bound; [parts] Bubble's 300s timeout is the envelope example. · *Gate:* R1

W-03. If the same job fires twice, is that harmless? If not, what makes it fire exactly once?

Why: [nova] Idempotency; queue redelivery is normal, not exceptional. · *Gate:* R11

W-04. [channel] For verification: where can a script see a job's state (queued/running/done) — an admin endpoint, a table, or a span?

Why: [parts] Invisible-outcome channel for the queue; Tracetest-style spans or a status endpoint, but it must be named. · *Gate:* R3

Billing

B-01. Which provider — Stripe, Lemon Squeezy, other — and whose account? Who holds the API keys and where are they pasted?

Why: [gate] R6 custody; [parts] provider choice changes every webhook and criterion. · *Gate:* R6

B-02. One-off payments, subscriptions, or both? If subscriptions: which plans, prices and intervals, exactly?

Why: [parts] The starters split precisely here; a criterion cannot assert an unnamed price. · *Gate:* R1

B-03. What happens in the app the moment payment succeeds — and what happens if the webhook confirming it never arrives?

Why: [parts] Stripe documents that event order is not guaranteed; assert eventual states, never sequences. · *Gate:* R3

B-04. Refunds: who can issue one, from where, and what does it change in the app?

Why: [gate] R11 — a charge is a side effect; the refund is its rollback and must be named.
· *Gate:* R11

B-05. Failed payments and dunning: retry how many times, then what — downgrade, lock, or email?

Why: [parts] Every billing service has a dunning setting; unset means undefined app behaviour on the most common billing event. · *Gate:* R1

B-06. [channel] For verification: is provider test-mode available, and what is the webhook endpoint the checker may hit with test events?

Why: [enf] `stripe listen` /test-mode is the interposed channel; without it billing criteria cannot run against the live system safely. · *Gate:* R10

B-07. Tax and receipts: does the provider handle tax (merchant-of-record) or do you, and must a receipt email go out?

Why: [parts] The Stripe/Lemon Squeezy consolidation is exactly this split; a receipt email is an invisible outcome needing a channel. · *Gate:* R1

3. Composition questions — asked once per assembly

C-01. Which parts are in this assembly, and does any appear twice? Give each instance a short name.

Why: [parts] Instance namespacing is structural, not conventional; two Forms on one page must be distinct by construction. · *Gate:* R1

C-02. What does each part need from another (login before billing? records before reports?)? State every such dependency.

Why: [parts] Provides/requires is the composition rule; undeclared dependencies become ordering bugs. · *Gate:* R1

C-03. Is any part's configuration in conflict with another's (auth says invite-only, billing says self-serve signup)?

Why: [brief] Constraint conflicts collapse compliance; catch the impossible pair at spec time. · *Gate:* R1

C-04. Do two parts write the same data? Who wins?

Why: [parts] Shared-substrate mutation is a documented seam failure (Shopify's sandboxing exists because of it). · *Gate:* R12

C-05. Summed envelopes: do the parts' combined limits fit the declared ceilings (jobs × runtime, records × queries, files × storage)?

Why: [parts] Two individually working blocks fail when composed at scale; sum the envelopes before building. · *Gate:* R1

C-06. One user journey that crosses at least three parts, end to end: walk it. Where does state hand over?

Why: [parts] The seams are where composition breaks; a walked journey surfaces the handovers no per-part question can. · *Gate:* R3

4. Custom-build detectors — the answers that mean 'this is not a part'

X-01. Is there a calculation, matching rule, score or price that is YOUR way of doing it — the thing the business actually is?

Why: [parts] OutSystems' own stepdown rule: 'your own algorithms' means leave the blocks. Vendors agree the algorithm is never in the box.

If yes: Mark as custom scope with its own acceptance criteria; do not force into a part.

X-02. Does any approval flow have rules beyond who-approves (thresholds, delegation, escalation chains)?

Why: [parts] Org approval semantics are always business-specific; parts ship thin defaults only.

If yes: Custom flow logic; the Flow part hosts it but its rules are authored, not configured.

X-03. Does any single unit of work exceed a part's envelope (minutes-long jobs, >tens-of-thousands of records in one view)?

Why: [parts] The envelope stopping rule; composition stops where the block's ceiling is.

If yes: Custom build for that unit; keep parts for the rest.

X-04. Does the customer's description keep using a word no part has (their domain noun)? What is its data shape and behaviour?

Why: [parts] The domain model is the never-templated residue; the unmatched noun is its tell.

If yes: It is the Records part's schema at minimum, custom logic at most — ask R-01 about it first.

How it runs

The gap scan (Spec Builder step 3) asks these in order: universal, then the rider for each selected part, then composition, then the detectors. Any question the conversation already answered is marked covered with its transcript line; anything else becomes an **[ASK]** carrying the question text verbatim — silence is never an answer. The bank is data, not prose: adding a part means adding its rider entries, and the gate rule ids keep every question honest about which mechanical check it feeds. The channel questions are the ones that make invisible outcomes decidable: they force the spec to name where a script goes to look and how long it may wait, before anything is built.